

Manual

MES Development Suite AIS: AIP and CTWIN MDS-AIS 8.1

Version 1.4.23049

Last changed on: 01.09.2020

Copyright

©Copyright 2020 All rights reserved.

SAP® and R/3® are registered trademarks of SAP AG.

WINDOWS® is a registered trademark of Microsoft Corporation.

MPDV® and HYDRA® are registered trademarks of MPDV Mikrolab GmbH.

ORACLE® is a registered trademark of ORACLE Corporation, California, USA.

Copying and distribution of this documentation or any part thereof, for any purpose or in any form, is prohibited without prior written permission from MPDV Mikrolab GmbH.

The information contained in this documentation is subject to change without prior notice.

Contents

1	Overview – AIP / CTWIN.....	6
1.1	Features.....	6
2	Local Configuration File ctaip.ini	7
2.1	Basic configuration	7
3	Local Configuration File ctaipbut.ini	11
4	Local configuration file ctaipplay.ini.....	17
4.1	Formulas used in grid layout	22
4.2	Translations in grid layout.....	24
4.3	Configuration of basic screens	24
4.3.1	Available fields for the dialog configuration of basic screens	26
4.4	Integrate dynamic dialogs in information dialog	27
4.5	TextViewer in dynamic dialogs	29
5	Central Configuration File hytnrcfg.ini	31
5.1	Layout configuration	34
6	Local Configuration File keyboard.ini	36
7	Dynamic Dialogs - Workflow	39
8	Dynamic Dialogs	45
9	Dynamic Dialogs - Fields	54
10	Dynamic Dialogs - Function Keys	71
11	User Exit Reference CTWIN/AIP	78
11.1	Introduction	78
11.1.1	Storage.....	79
11.1.2	Processing	80
11.2	User Exits on the Terminal	81
11.2.1	CTWIN	81

11.2.2	CTWIN + AIP	96
11.2.3	AIP	119
11.3	DIALOG – Script Processing	121
11.3.1	CTWIN + AIP	121
11.3.3	AIP	133
11.3.4	Tips and tricks with the dialog control	135
11.4	Script Functions/Variables	137
11.4.1	Notes on script variables	137
11.4.2	Script functions	141
11.4.3	Notes on the script functions	173
11.5	Special Fields of Application	179
11.5.1	Assignment of a script function to a key without DDLG	179
11.5.2	How to use the functions GSrce, VSrce	180
11.5.3	Important note when working with numbers	180
11.5.4	Important note for < IF > queries	180
11.5.5	Update grid at the push of a button	181
11.5.6	Recording of MSS signals	181
11.5.7	Call script when leaving "Log on/off person" using ESC	182
11.5.8	Implementing a debug output	182
11.5.9	Dynamic assignment of a button configuration in the OP info (AIP)	183
11.5.10	Call scripts from DNC (only AIP)	183
11.5.11	Setting the display time of an LR error message (only AIP)	184
11.5.12	Editing the DNC communication	184
11.5.13	Script events online/offline	185
11.5.14	Setting offline status for tests	185
11.5.15	Show comments in machine status log	185
11.5.16	Script events with CAQ calls (only AIP)	186
11.5.17	Script events with line breaks in order and machine list	186
11.5.18	Read first row from list file	186
11.5.19	Processing of long 2D bar codes	187
11.5.20	Reloading ANR list script request (only AIP)	187
11.5.21	Assign function keys in the order sequencing list	187
11.5.22	Script event when changing cell in the machine list	188
11.5.23	Focus in dialog grid (only AIP)	188
11.5.24	Script event when loading additional info	189
11.5.25	Script events when navigating in the dialog grid	189

11.5.26 Extended customizing of label printing.....	190
11.6 Terminal Dialog List Files (in alphabetical order)	191
11.6.1 Static lists ().....	191
11.6.2 Temporary lists ()	193
11.7 Exceptions via programmed standard development	195
11.7.1 Exception: Sending of ID < EGG:GUT >.....	195

1 Overview – AIP / CTWIN

1.1 Features

You can use the MES Development Suite to change and extend the data collection and the data display on the shop floor clients AIP and CTWIN.

The document **MDS-AIS_81_AIP_CTWIN** describes the functions that the MES Development Suite Business Applications & Services provides to change and extend the data collection and the data display on the shop floor client AIP or CTWIN.

- Using configuration files, you can change the layout of the shop floor client AIP/CTWIN. The configuration files are available as INI files.
- Using the dialog configuration on the MOC, you can change and define the dialogs and workflows to enter and display data.
- The shop floor clients AIP and CTWIN provide user exits that you can use to implement dynamic actions in the data collection. The user exits are implemented in a script language. The script language is similar to the programming language Visual Basic and easy to learn.
- Using deployment mechanisms, you can automatically deploy the customer-specific configurations and user exits to the shop floor clients.

The document **MDS-AIS_81_AIP_CTWIN** is the reference manual of the functions provided. To learn all about the MES Development Suite Business Applications & Services, MPDV offers specific trainings. MPDV recommends to attend this training to be able to successfully use this product.

2 Local Configuration File ctaip.ini

The most important hardware and system settings are defined for each terminal in the CTAIP.INI file of the c:\ctaip directory.



Changes to the configuration file ctaip.ini are only enabled after rebooting the terminal software.

2.1 Basic configuration

Entry	Comment
Section [system]	

Entry	Comment
Section [system]	
Usr=21	Distinct terminal number
Hypath=d:\hydra\ Hypath=/usr/hydra/	HYDRA server path: Windows NT: -> DOS notation, the drive is the local drive of the server on which HYDRA or xMES is installed Unix: -> Unix notation
Hostname=192.9.200.24	Internet address of the server
Offlinetimeout=600	In offline mode, the interval after which online access should be attempted the next time. The interval is specified in seconds
Showcursor=on	Show or hide mouse pointer in terminal application: on: mouse pointer active off: mouse pointer inactive
Loadfile=ctnet\win\ctaip.txt	Configuration file for downloading the application from the server. The path is relative to the server directory (i.e. within "hypath")
Watchdog=on	ON: Watchdog is activated OFF: Watchdog is not activated
Demo=off	'on': Offline demo mode; always off in the production environment
parameters=-t	The -t parameter switches off the virtual keyboard
TMOUT_C=xxx	Timeout for CONNECT to the server If not specified, default = 10 seconds → Increase to 20 seconds for routing
TMOUT_S=xxx	Timeout for SEND to the server If not specified, default = 10 seconds → Increase to 20 seconds for routing
TMOUT_R=xxx	Timeout for RECEIVE of the server If not specified, default = 120 seconds

Entry	Comment
TMOUT_F=xxx	Timeout for FILESERVER operations to the server If not specified, default = 10 seconds → Increase to 20 seconds for routing
Section [barcode]	Configuration of customized barcode prefixes.
BarKenn90=MNR4 ... BarKenn99=ANR3	BarKenn90 > defines the prefix (here: 90); The ID from the dialog (= acronym) is assigned.
Section [comports]	
Com1=0 com2=MSS com3=BAR Com3=LEGIC Com4=RFLESER	Assignment of serial interfaces to the connected devices MSS – machine interface BAR, LEGIC, RFLESER – various reading devices
Section [MSS-INIT]	
ZAEHLER= 1 2 3 4 5 6 7 8	Assignment of physical inputs of the MSS (machine interface) to logical counters (ZAEHLER) according to configuration: The first connector (labeled “0” on the MSS) corresponds to the logical counter no. 1 Please note: MSS1 has only 8 inputs If digital inputs are also to be used with MSS1, configuration should be changed as follows: ZAEHLER= 1 2 3 4 IN= 5 6 7 8
IN= 9 10 11 12 13 14 15	Assignment of physical MSS inputs to logical inputs as per configuration: The ninth connector (labeled “8” on the MSS) corresponds to the logical input no. 1
CHARGE= 5 6	For batch recording: Inputs for automatic batch changes In this case, the connectors 5 and 6 are the inputs for the automatic batch change.
MSSZyklusBerechnung=ON MSSZyklusReferenz=0	Activates reading out of cycle time values from the machine interface (MSS). Reference for cycle calculation (smallest time unit of the MSS). The default value is 0, if the parameter is not specified. 0 corresponds to 100 ms. 2 corresponds to 20 ms.
CalculateCycle=ON	The terminal itself calculates the actual cycle. If the connected control does not provide a determined actual cycle, then a calculated actual cycle can be displayed: This calculated value is also available for DS100 terminals. Please note: The calculation cannot provide exact values.
WochenEnde_ProdCheck=ON	This function prevents the weekend automatic from affecting the “production” status and the workplace from being set to status 999. ON is set by default In case WochenEnde_ProdCheck=OFF, the automatism switches to status 999.

Entry	Comment
sFrom999ToNotAttributed=OFF	Only affects HYDRA-MDE machines with operation mode = "no monitoring". If the weekend automatic function is enabled this option prevents the machine from switching to the "not assigned" status when the weekend automatic ends. Reasons: The "not assigned" status may not be set manually for machines that are configured with the "no monitoring" option. The "not assigned" status is normally only set for machines with operation mode = "cyclic monitoring" or "monitoring by operating signal".
Section [ext. software]	
Button=Editor WindowName=Editor SearchParts=On	Configuration of the button in the top line: A previously started program can be called to the foreground at the push of a button. Button: button caption WindowName: Name of the program (e.g. from the taskbar). SearchParts=On: parts of WindowName are sufficient SearchParts=Off: WindowName must be entered completely. The option "SearchParts=On" is recommended for programs such as MSWord that change the title bar subject to the document that is currently being loaded.
ProgFileName=c:\Programme\wincmd\Wincmd32.exe	The program that is started if the program mentioned above cannot be called to the foreground.
AutoStart=on	This option starts the program (ProgFileName) when starting the terminal program.
Section [PDV]	
Modus=PDV Modus=PDV,BDE	Operation as interface IOP↔DOS terminal (standard) Operation as shop floor terminal with PDV
Terminals=121,122,123	CTDOS terminals "connected" by LAN. Only required with Mode=PDV
PDVTerminalDir=c:\hsrv\spool\ PDVIOPTDir=c:\IOPSim\	Directory for communication with CTDOS terminals Directory for communication with the IOP
For debug purposes only: InfoFenster=100 SlowDown=600	Number of lines of the "current" window (presentation of last PDV actions) Slow down, to make events "visible"
;Supported barcodes	
FieldWNRBarcodeOnly=Y	If this entry is set the tool number may only be entered using a scanner.
FieldNestBarcodeOnly=Y	If this entry is set the cavity number may only be entered using a scanner.
FieldNummBarcodeOnly=Y	If this entry is set the number may only be entered using a scanner.
FieldKNRBarcodeOnly=Y	If this entry is set the badge number may only be entered using a scanner.
BarcodeWNR=	This field specifies which acronym is entered into the tool number field by the scanner.

Entry	Comment
BarcodeNest=	This field specifies which acronym is entered into the cavity number field by the scanner.
BarcodeNumm=	This field specifies which acronym is entered into the number field by the scanner.

3 Local Configuration File ctaipbut.ini

Buttons are configured for specific terminals in the file ctaipbut.ini in the terminal directory c:\ctaip.

The button pages of the main view and the OP info dialog may be configured in the configuration file ctaipbut.ini.

The server directory \hydra\ctnet\win\aip includes complete INI files pertaining to the HYDRA standard. Any deviations from that are developed in specific, customized directories e.g. \hydra\1\custom\aip\trp_901.



The relevant, empty file (e.g.: ctaipbut.ini) is created here. All sections e.g. [ANR-ALL-Page1] are copied to this file. Then configuration takes place in this file.

After restarting the terminal, files from the main directory \hydra\ctnet\win\aip are merged with files from the customized directory \hydra\1\custom\aip\trp_901. Then the merged file is transferred to the local terminal directory C:\aip.



All sections including the string "page" are imported.

Entry	Comment
Definition of sections [LST-MODUS-PageX.]	General schematic structure of a button page Definition of a section LST = List identifier of the button page (MNR, ANR, LIST3) MODUS = Mode of the machine LN = MPL – Mode DN = DLL – Mode LR = RF – Mode (reel-based manufacturing) LS = RS – Mode (cutting reels) LC = Handling unit (packing station) or XX = <MPL_MOD>[1] + <TYPE>[] YY = Value from the MNR.LST column <MNRBTN.MODUS> Otherwise if no applicable entry is found for the machine mode, the section ...ALL will be used (if available) X = Button page The definition specifying the mode a machine is running on is implemented in the AIP application program.

Entry	Comment									
Sample configuration [MNR-...-Page1] 1=A_AN, L, log OP on 2=BLANK, L 3=\$MPL-PAL\$PAL_AN,L,log pallet on, 4=%BART:PZE=J%PZE,R,PZE,PZE. BMP <u>Please note:</u> In one section the numbering of entries has to be consecutive 1...n. A gap in numbering indicates the completion of a page!	<u>General structure:</u> x=<Function>,<Alignment>,<ButtonName>,<Icon> e.g. 1=A_AN,L,log OP on,AGAN.PNG - Function A_AN - Alignment L or R (from the first "R" on always "R") - ButtonName Log OP on - Icon optional icon name (PNG, resolution 24x24 px) <u>Special functions:</u> \$...\$ (e.g. \$MPL-PAL\$) License check fails → Button is deleted %...% (e.g. %BART:PZE=J% .) Check field with value in (T)terminal(K)label → only show if they match BLANK Insert distance between buttons									
Configuration of functions using wildcards x=A_AN*,L, log OP on x=A_UN*,R, interrupt OP	The dialog to be opened is located as described below if buttons are configured using wildcards ID A_UN* - calling dialog: A_AN - Determination of the machine type - Supplementing the dialog based on the machine type - Check whether or not the dialog is available if this is the case - calling dialog: A_AN_MPL - evaluation of the posting type (only with A_AN) - Supplementing the dialog based on the <i>posting type</i> - Check whether or not the dialog is available if this is the case - calling dialog: A_P_AN_MPL - Calling up the located workflow or dialog If the function is <A_UN*> or <A_AB*>, it will be checked whether or not the OP to be logged off is a merged OP. - If this is the case, <A_UN*> is changed into <SA_UN> or <A_AB*> into <SA_AB> If the virtual column <MNRDLG.SUFFIX> includes a value, it will always be used (if available). <table><tr><td>ButtonFkt</td><td>MNRDLG.SUFFIX</td><td>Dialog</td></tr><tr><td>e.g. A_AN*</td><td><XYZ></td><td>A(_P)_AN_XYZ</td></tr><tr><td>A_TR*</td><td><ABC></td><td>A_TR ABC</td></tr></table>	ButtonFkt	MNRDLG.SUFFIX	Dialog	e.g. A_AN*	<XYZ>	A(_P)_AN_XYZ	A_TR*	<ABC>	A_TR ABC
ButtonFkt	MNRDLG.SUFFIX	Dialog								
e.g. A_AN*	<XYZ>	A(_P)_AN_XYZ								
A_TR*	<ABC>	A_TR ABC								

Entry	Comment
Further standard buttons: P_SPERRE VIEW ICON PDV_ISTW WF_BDE_KOM SA_AN DNC MINIMIZE	Block production status Switching of the basic view: List view ↔ presentation of individual machines Calling up icon view (only possible if configured in the machine configuration) Calling up the actual value view of PDV Input of BDE comments Log merged operation on Calling up the DNC startup screen Minimizing of the terminal program (as of V2.0.2.23) → Windows 7 requires the compatibility mode XP
USER1...USER9	User-defined buttons to show and start external software The programs are configured in ctaip.ini within the section [ext. software]
Button IDs for the machine info [MNR-ALL-Page2] 1=M_INFO.INFO, L, show information 2=M_INFO.PERS, L, staff 3=M_INFO.MSPROT, L, machine status log	Consequently, the relevant info dialog including the selected page is opened in the foreground (with focus). Switching to other pages is allowed. M_INFO may be used to show the info page in the foreground (with focus): M_INFO=M_INFO.INFO
Button IDs for OP info: [ANR-ALL-Page3] 1=A_INFO.DOKU, L, documents 2=A_INFO.HILF, L, production resources and tools 3=A_INFO.KOMP, L, components 4=A_INFO.BMK, L, RPA 5=A_INFO.FORT, L, progress 6=A_INFO.NOTE, L, notes A_INFO.TEXT1, L, User text1 A_INFO.PICT1, L, User image1 A_INFO.SCRINF1, L, User scriptInfo1 A_INFO.DIALOG1, L, User dialogs	Consequently, the relevant info dialog including the selected page is opened in the foreground (with focus). Switching to other pages is allowed. A_INFO may be used to show the information page in the foreground: A_INFO= A_INFO.INFO Direct call of user-defined pages configured in ctaip.ini in the section [OP info]. Example: Dialog1=WF_BDE_KOM_LIST,BDE comments → A_INFO.DIALOG1, L, BDE comments

Entry	Comment
Configuration of a function with different modes Just as is the case for the configuration of the ANR/MNR info tabs, a function can be configured with different modes. 1=RES_WART.MNR,L,machine maintenance 2=RES_WART.RES,L,resource maintenance 3=RES_WART,L,other maintenance	In the configured examples, the dialog < RES_WART > is called with the below-mentioned modes. 1 < MNR > 2 < RES > 3 without mode The values can be read out as follows in the terminal script. VPar("BTN.FKT") Function + mode VPar("BTN.FUNC") Function VPar("BTN.MODE") Mode
Available button sections and buttons for pages of the OP info dialog: [A_INFO-Page1] [A_INFO.DOKU-Page1] 3=AI_VIEW,R,open document 4=AI_VIEW_CLOSE,R,close document [A_INFO.HILF-Page1] [A_INFO.KOMP-Page1] [A_INFO.BMK-Page1] [A_INFO.FORT-Page1] [A_INFO.NOTIZ] [A_INFO.DEFAULT-Page1] Recommended for all pages: 1=AI_CLOSE,L,close OP information	Overview Document view Production resources and tools Components Resource performance accounts Progress bar Notes (as of ADE 7.3) Configuration of a default page (is used if no section is defined for the tab). The IDs may also be used for the keys within the dynamic dialog (field "function").

Entry	Comment
Available button sections and buttons for pages of the machine info dialog: [M_INFO-Page1] [M_INFO.PERS-Page1] 2=P_AN,R,log person on 3=P_AB,R,log person off 4=P_AAB,R,log everyone off [M_INFO.MSPROT-Page1] [M_INFO.DEFAULT-Page1] For all pages: 1=MI_CLOSE,L,close machine information	Overview Staff logged on Machine status log Configuration of a default page (is used if no section is defined for the tab).

Entry	Comment
Definition of sections [ButtonPanel]	General section for the configuration of global settings for all used button panels → 2 in main view (MNR , ANR) → (W)ork(F)low
functionkey_visible=on	Shows function keys (e.g. "F3") in button panels in order for the selection to be performed using function keys (default = off).
radiobuttonkey_visible=on	Presentation of function keys in radio group boxes of a workflow (default = off).
functionkey_pze_visible=on	Display of function keys in PZE module (default = off).
Definition of sections [LIST3-ALL-Page1]	General section for the configuration of functions of the configurable third list of the main screen. INFO:
as of CTAIP V# 2.0.2.33 ..=~<VISLIST-ID>~,L,,<BITMAP>	The different types of the "3rd list" are configured in the machine label. The layout of a "3rd list" is defined in the "hytnrcfg.ini". → All used lists have to be configured with their identifier „“ as follows. → When changing machines, the "3rd list" may be hidden/shown and buttons for the "3rd lists" that are not configured may be disabled, if necessary.
The characters "~" (or previously "S", should no longer be used) have been designed to identify third list buttons. Correct processing/updating (disabled/enabled) is only given in the third grid list of the main screen.	
1=~M~,L,,PALETTE20x20.BMP	Entry for "material list" → "[VISLIST3(M)]" from "hytnrcfg.ini"
2=~P~,L,,PERSON20x20.BMP	Entry for "list of persons" → "[VISLIST3(P)]" from "hytnrcfg.ini"
3=~R~,L,,RESS20x20.BMP	Entry for "MNR_AMAT.LST" → "[VISLIST3(R)]" from "hytnrcfg.ini" with the configured Bitmap
4=~A~,L,,NUM.BMP	Entry for "material list" → "[VISLIST3(A)]" from "hytnrcfg.ini"
5=~G~,L,,PERSON20x20.BMP	Entry for "list of persons GWP" → "[VISLIST3(G)]" from "hytnrcfg.ini"

4 Local configuration file ctaipplay.ini

The layout is configured for specific terminals in the file ctaipplay.ini in the terminal directory c:\ctaip.

This file is used for the configuration of grids in AIP.

The server directory \hydra\ctnet\win\aip includes complete INI files belonging to the HYDRA standard. Any deviations are developed in specific, customized directories e.g. \hydra\1\custom\aip\tgrp_901.



Create the corresponding, empty file (e.g. ctaipplay.ini) in this directory. Modified sections are copied to this file. Make the configurations in this file.

After restarting the terminal, files from the main directory \hydra\ctnet\win\aip are merged with files from the customized directory \hydra\1\custom\aip\tgrp_901. Then the merged file is transferred to the local terminal directory C:\aip.

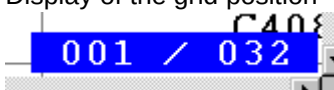


Changes to the configuration file ctaipplay.ini will not take effect until the terminal software has been restarted.

Entry	Comment
Section [OP info]	
Deaktiviert=AG_Bmk,AG_Fort	- Indicated info pages are not shown - AG_Info (OP info) cannot be disabled - entries affected by sorting are not disabled.
Sortierung=AG_TechInfo,*	- Order of info pages in the icon list - if the list ends with " , * " the non-listed standard pages are added at the end. - Standard pages: AG_Info,AG_ZulInfo,AG_Bmk,AG_TechInfo,AG_Fort,AG_FertP ap
Sortierung=AG_Info,AG_AD_Satz,Text File1,TextFile2,Bild1,* TextFile1=C:\LOGOPAK\druck.txt,druck .txt TextFile2=C:\LOGOPAK\druckerg.txt,dr uckerg.txt Bild1=C:\aip\spool\screen1.bmp	Configuration of any texts and images that can be displayed in the OP info dialog.
Section [main]	
Nachkommastellen=0	Decimal places for quantities in the order/machine overview
Repaint_time=60	Cycle for updating the view (for machine list and machine info)
Buttonkonfigaktiv=0	reserved
layout=100	reserved

Entry	Comment
AuftragZusatzInfoFontSize=12	Font size of texts in the "OP info" dialog
AuftragZusatzInfoFontName=Courier New	Font type of texts in the "OP info" dialog
PopupSize->EmptyQueue=300	Empty popup window size for quick queue
PopupSize->ReloadPze=200	Reload popup window size for PZE configuration
SymbolAdditionalInfo=MBEZX	Display of any field from the machine list in the icon view between machine number and operation number The machine number is replaced by the configured field if lines and aggregates are configured.
SymbolSubstDesignation=MBEZX	The specified field replaces the machine number in the icon view.
Sections for list layouts	
[Personenliste]	List displayed when staff is logged on
[Bedienposition]	Predefined list of "operator positions"
[Maschinenstatusliste]	Predefined list of "machine statuses"
[Ausschussgruende]	Predefined list of "scrap reasons"
[Abweichungsgruende]	Predefined list of "deviation reasons"
[Auftragsliste]	Lower list in the main view
[Vorgabeliste]	Order sequencing list
[Schichtinfo]	List of shift info
[Maschinenliste]	Upper list in the main view
[Eingangslsliste] input batch list	List of input batches, e.g. when "logging on the OP" in batch mode
[Ausgangslsliste]	List of preceding batches when "changing output batches"
Syntax of table formatting	
GRID_FONT=Arial	Font type
GRID_FONTSIZE=9	Font size
GRID_COLOR=clBlack	Font Color
GRID_BACKGROUND=clWhite	Background color
GRID_FIXCOLS=0	Number of fixed columns
GRID_ORDER=MSZEIB GRID_ORDER=MSZEIB=-	Sorting Sorting in descending order Please note: If several criteria are indicated (separated by), only the first criterion can be sorted in descending order. All other criteria are sorted in ascending order. The following entry must be set in the configuration for the section so that the sorting is used in the display: ORDER=#USE#INI#ITEM# <u>Example for the section Sequencing List (Auto)</u> [WF@ANR] CMD=DLG=LIST;11 MOD=V MNR=<MNR> SECTION=Sequencing List (Auto) ORDER=#USE#INI#ITEM#
GRID_FILTER=MGRP=0	Table filtering Please note: NOT supported for script dialogs.

Entry	Comment
GRID_LIST_TYP=MNR GRID_LIST_TYP=ANR	The list type of the section is specified with this entry, if fields are displayed that need to be loaded additionally. This entry also enables the search when starting. The entry has to be entered above the IDs to be reloaded!!! All IDs that can be reloaded can be found in the file headers.txt in the "spool" directory of the terminal. It consists of four lines: 1. Fields that are always included in the machine list 2. Fields that can be reloaded for the machine list 3. Fields that are always included in the order list 4. Fields that can be reloaded for the order list
EXAMINE_CONTENTS_CHANGE=MGRP EXAMINE_COLOR_C1=cIWhite EXAMINE_COLOR_C2=cISilver	The font color switches from cIWhite to cISilver every time the MGRP value changes. Up to 8 colors can be defined.
EXAMINE_SCANEXPR1=MGRP=71 72 73 EXAMINE_SCANEXPR2=MGRP=96 97 101 EXAMINE_SCANCOLOR1=CIGreen EXAMINE_SCANCOLOR2=CIRed	The machine groups 71/72/73 are presented in green font color; the groups 96/97/101 are displayed in red font color. Up to 8 colors each can be defined.
EXAMINE_SCANBKEXPR1=BATTRIB=1 EXAMINE_SCANBKEXPR2=BATTRIB=2 EXAMINE_SCANBKCOLOR1=cIBlue EXAMINE_SCANBKCOLOR2=cILime	All lines with BATTRIB=1 are shown in blue background color; rows with BATTRIB=2 are displayed in lime. Up to 8 colors each can be defined.
EXAMINE_COLOR=TEXTCOLOR EXAMINE_BKCOLOR=HGRCOLOR	Specification of a column that includes the color value for the row (e.g.: 0-Black; 255-Red, 16777215-White)
EXAMINE_CELLBKLEVEL2=EGR:AUSP,EGR:AUSP,<1*cILime <=5*cIYellow >15*cIRed	Setting of the background color depends on whether the field value reaches different threshold values.
EXAMINE_ROWBKLEVEL1=SGR:REST,SGR:REST,<=0*cIRed <=5*cILime >15*cIYellow	Setting of the background color depends on whether the field value reaches different threshold values. The entire row is colored because of the threshold value.
Syntax of column definitions	
MNR=C8,80,R MGRP=N6,60,R AGR:AUS=N10.2,125,R MSDATB=dd.mm.yy,70,L MSZEIB=hh:mm,70,L SKDATB=dd.mm.yyyy,90,L SKZEIB=hh:mm:ss,80,L MSDAUER=ddd.iii,60,R	Alpha-numeric, 8 characters, 80 pixels, right-aligned Numeric, 6 characters, 60 pixels, right-aligned Decimal, 10 digits, 2 decimal places, .. Displayed in the form "23.03.98" (left-aligned) Displayed in the form "08:24" Displayed in the form "23.03.1998" Displayed in the form "08:24:39" Displayed in industrial time unit " 22,982"
AGR:BMK11=hhh:mm:ss,80,R,TESTHEADER	TESTHEADER: new column caption
ALIAS KOPIE=MNR=N8,120,R,TITEL	ALIAS new name is being introduced KOPIE= new identification MNR= ID in data file N8,120,R, Formatting TITEL column caption in table
ALIAS AKA=MNR[1..3]=N8,120,R,ARRAY[1..3]	The first three characters from MNR are displayed.

Entry	Comment
ALIAS ATTR=MNR(2)=N8,120,R,PARAMETER(2)	The second part separated by „ ; “ is displayed. Example: „ 12;20;130 “ → „20“
ALIAS U=(R*6.2831)=N10.3,60,L,Scope	Conversion of a value Syntax: see below
ALIAS SOLLZ={_INT((_DATETIME(SKDATE , SKZEIE)-_DATETIME(SKDATB , SKZEIB))*86400)}=hh:mm:ss,60,R,SOLLZ ;target time of shift	Complex calculations relating to several fields Syntax: see below
GRID_BROWSEROW=0	only the active row is colored yellow
GRID_CELLPAINT=ON	Requirements for coloring rows column by column
GRID_REFRESH=5000	Cycle for updating the display [ms] → lists are not reloaded from the server! Recommended if a constantly changing value is calculated using an ALIAS function.
GRID_POSITION=ON	Display of the grid position  Limited/no support when scrolling using scroll bars and page scrolling
GRID_CELLPAINT=ON EXAMINE_CELLBKCOLOR=WTK:STA, WTK:STA,0-clGreen 1-clBlue 2-clYellow 3-clRed can also be used with index: EXAMINE_CELLBKCOLOR1..8 As of 2.0.2.64: EXAMINE_CELLBKCOLOR1..20	One single column is colored in every cell subject to a value. 1st value: ID of the column to be colored. 2nd value: ID of the reference column 3rd value: Configuration (color for possible values) Notes: - The reference column MUST be shown in the list, if required with length 0 - The values are converted into capital letters when being compared.
As of ctaip V# 2.0.2.17 EXAMINE_CELLBKCOLOR=DMY,COLOR	Take over the color directly from the "color" column. The column <DMY> is shown in the color defined in the column <COLOR>

Entry	Comment
; Definition virt. column(1) EXAMINE_CELLVALUE1=CV1,REF1,S =DAT P=ZEI A=REST N=INFO ; Definition virt. column(2) EXAMINE_CELLVALUE2=CV2,REF2,1 0=MGRP 20=MNR 30=COLOR 40=MS TTXT ... ; Layout/Position virt. column(1) CV1=CELLVALUE,150,Z,Data ... ; Layout/Position virt. column(2) CV2=CELLVALUE,150,Z,M/C/T	Filling of a virtual "Case" column with values from different columns subject to the value of a reference column. 1rd value: Identification of the virtual "case" column 2rd value: Identification of the reference column 3rd value: Configuration Reference value + , = ' + display column Please note: the virtual "case" column needs to be configured as follows <Identifier>=<Key word>,<Width>,<Alignment>,<Caption> e.g. CV1=CELLVALUE,150,Z,YST
GRID_RANDOMSORT=ON	This options randomly sorts the list. Please note: If this option is active, any configured sorting will be ignored.
GRID_CLIPBOARD=<BUTTON>@<SELECT>@<DATA>	This option copies data from a table/grid into the clipboard.
Special entries	
[Auftragsliste] ALIAS Restlaufzeit=RLZ= C4,44,R,RLZ ALIAS SollErreicht=ZBV1= C1,12,R,S	Activation of the calculation & display of the remaining run time Activation of the calculation & display for target quantity reached (column 'S': '*' if reached)
[Maschinenliste] ALIAS StkProMin=IZYSM= N8,48,R,Stk/min	Activation of calculation & display of the produced pieces per minute
[layout pze] KundenBitmap=kunde.bmp DienstGangTaste=1,3 StdSchrift=Arial StdDateSize=30 StdStatusSize=26 StdSpdBttnSize=16 InfoSchrift=Courier New InfoSchriftSize=20 SmallStatusFontSize=16 DateTimeLayout=dd.mm.yyy hh:mm:ss	Configuration of the PZE terminal "Kundenbitmap=<File name>" file with customer logo When restarting the terminal, this file is copied from the server directory ".\ctnet\win\aip\etc\" into the application directory ".\etc\". „DienstGangTaste=1,3“ → Default [empty] By entering the function key numbers (1...4), a check specifying if the person is allowed to go on a business trip is performed during the posting. Configuration of the used font types/font sizes as well as the layout of the date and time display.

4.1 Formulas used in grid layout

Simple conversion of a value

Syntax: **ALIAS** <Alias>=(<formula>)=formatting

<formula>: [1/]<ID>[<Operator><Value2>]

<ID>: ID from list (The current value from the list is entered here in the formula)

<Operator>: + | - | * | / | ^

<Value2>: 2nd Operand

Extensive formulas:

Formulas that can also relate to several table fields can be recognized by braces.

Syntax: **ALIAS** <Alias>={<Formula>}

<Formula>: (<Operand1>[<Operator><Operand2>])

<Operand>: <Value> / <Function> / <Formula> / |KENN|

<Operator>: |+|-|*|/|^|

<Value>: Constant ('0'..'9','e','E','.', '-')

|Kenn|: reads out a value from the table

<Function>: _<Fname>(<Operand>[,<Operand>[,...]])

- _ **DATETIME** (<Date>,<Time>)
 - <Date>: mm/dd/yyyy
 - <Time>: ssss (Seconds of the day (0..86400))
 - ==> Real value as TDateTime
- _ **INT** (<Operand>)
 - returns a value without decimal places
- _ **REAL** (<Operand>)
 - returns a value with decimal places
- _ **ROUND** (<Operand>)
 - returns rounded value without decimal places
- _ **MOD** (<Operand1>,<Operand2>)
 - ==> Operand1 mod Operand2
- _ **ABS** (<Operand>)
 - absolute amount of a figure
- _ **EXP** (<Operand>)
 - e raised to the power of X (e: basis of the natural logarithm)
- _ **LN** (<Operand>)
 - natural logarithm (Ln(e) = 1)
- _ **FRAC** (<Operand>)
 - proportion of decimal places
- _ **LOG** (<Operand1>,<Operand2>)
 - LOG(N,X): logarithm to base N of X
- _ **MAX** (<Operand1>,<Operand2>)
 - the greater value of two values
- _ **MIN** (<Operand1>,<Operand2>)
 - the lesser value of two values
- _ **SQRT** (<Operand>)
 - ==> Square root of Operand1
- _ **PI** ()
 - ==> 3.14151926535...

The option "GRID_BROWSEROW=0" should also be set in order for the coloring to be recognized even if the row is selected.

Examples:

```
EXAMINE_CELLBKLEVEL1=MNR,MST,<=1*clLime|<=2*clYellow|>2*clRed
EXAMINE_CELLBKLEVEL2=FS,FS,<90*clLime|>=90*clYellow|>=100*clRed
EXAMINE_CELLBKLEVEL3=EGR:GUT,EGR:GUT,<(SGR:GUT)*clLime|>=(SGR:GUT)*clYellow
```

4.2 Translations in grid layout

Column contents can be configured to be translated and displayed by entering e.g. the configuration <XYZ=T10,100,L> instead of <XYZ=C10,100,L> in the configured grid columns. A <#> character must be prefixed for these "resource strings" to provide for better classification. This modification can be used in every INI file (hytnrcfg.ini,..) where grid layouts are configured.

Please note: The data do not include any translated values. In order for them to be displayed in e.g. dynamic dialog fields, an explicit translation must be performed using the VB script function <vbsTranslateDataValues("<columns>" , "<data row>") >.

Column contents can be configured to be translated and displayed by entering e.g. the configuration <PSPERRE=U1,100,L> instead of <PSPERRE=C1,100,L> in the configured grid columns. The entry for the "resource string" that depends on the field has the following structure:

	„#<Acronym>#<Value>“	
e.g.	„#PSPERRE#J“	"production lock enabled"
	„#PSPERRE#N“	„ “ (blank character)

This modification can be used in every INI file (hytnrcfg.ini,..) where grid layouts are configured.

Please note: The data do not include any translated values. In order for them to be displayed in e.g. dynamic dialog fields, an explicit translation must be performed using the VB script function vbsTranslateDataFields("<columns>" , "<data row>") >.

4.3 Configuration of basic screens

The dialogs/screens are configured using dynamic dialogs. For this reason, the following dialogs are always required:

MMINFO → Section referring to machines in the single machine view

MAINFO → Section referring to orders in the single machine view

60610		60611		60612		60614		NA43		GAP1		
Arbeitsplatz/Maschine	60612	Status	PRODUKTION									
Bezeichnung	ARB-320S	Beginn	14.08.10.2009									
Gruppe	SGM2	Dauer [Std:Min]	20:01									
Teiligkeit	1	Gutmenge	0.000000 ST									
Sollzyklus [Sek/Takt]	21	Ausschuss	0.000000 ST									
Istzyklus [Sek/Takt]	0.00	Takte	0.000									
Information anzeigen Produktionsstatus sperren Status ändern Arbeitsgang anmelden												
Arbeitsgang	S61003310100	Sollmenge	60000.000 ST									
Artikel	SAL-FMO011-K-	Gutmenge	228.000000 ST									
Artikelbezeichnung	Spiegelgehäuse	Ausschuss	4.000000 ST									
Bemerkung 1		Soll seit Anm.	--- ST									
Bemerkung 2		Gut seit Anm.	0.000000 ST									
Plandauer	350:00	Abweichung [%]	---									
		Fertigstellung	0%									
Information anzeigen Teilrückmeldung Arbeitsgang unterbrechen Arbeitsgang abmelden Hilfsmittel												
CTAIP V1.2.4.64 29.09.09 Server:10.10.60.70:3 TNR:126 Status: ON 09.10.09 10:02:57												

MINFO → Description of the machine information

Maschine 60612 Arbura Selogica 320S	
BDE-Kommentare	Angemeldete Personen
Arbeitsplatz/Maschine	60612
Bezeichnung	ARB-320S
Gruppe	SGM2
Teiligkeit	1
Sollzyklus [Sek/Takt]	21
Istzyklus [Sek/Takt]	
Status	PRODUKTION
Beginn	14.08.10.2009
Dauer [Std:Min]	20:03
Gutmenge	0.000000 ST
Ausschuss	0.000000 ST
Takte	0.000
Maschineninformation schließen	

AINFO → Description of the order information

The heights of the individual components of the basic screens and, as a result, the positions of the button bar are configured in the *ctaipplay.ini* file using the below-mentioned parameters:

Section [MainView1]	Configuration of the basic screen
MachineGridHeight=415 OrderGridHeight=500 ButtonBarHeight=50	Height configuration of components for the basic screen (machines, order grid, button bar) The configured heights are scaled to the current height. Consequently, the total sum of entered heights does not play a role.
Section [MainView2]	Configuration of the single machine view
MachineGridHeight=50 MachineInfoHeight=415 OrderInfoHeight=355 ButtonBarHeight=50	Single-row grid to select the machine Information on the machine Information on the order Height of both button bars The configured heights are scaled to the current height. Consequently, the total sum of entered heights does not play a role.

4.3.1 Available fields for the dialog configuration of basic screens

A script function that fills the fields according to the customer's requirements is currently not available (14 September 2009).

In general, the fields of the machine list and the order list are available. "MNR." or "ANR." must be prefixed for identification purposes.

Known quantity fields are formatted to match the configured number of decimal places.

Some fields are calculated. The following fields are additionally available:

Acronym	Description
ANR.SOLL_SEIT	Target quantity since login The value is determined locally at the terminal. This is only useful for MDE machines. However, the order must be logged on locally after restarting the terminal.
ANR.ABWEICH	Deviation [%] Comparison of "target quantity since logon" and "actual quantity since logon"
MNR.SZY	Target cycle As of AIP 2.0.2.85: Field is transferred including "internal decimal places". The number of characters displayed is determined by the field of the dialog configuration.
MDE.IZY	Actual cycle The machine's current actual cycle - only if MDE processing is active for the machine at this terminal.
MNR.MSZEIB	Start time of the current status
MNR.MSDATB	Start time of the current status
MNR.MSDAUER	Duration of the current status
ANR.BEARBZ	Planned duration
MNR.MSTTXT	Status text
MNR.TLG	Partitioning Calculated based on the orders running at the machine.
ANR.FERTIG	Progress bar
TNRSPERRE	Translated text for the production lock (corresponds to the configuration "TNRSPERRE=U1,150,L,Hinweis" in ctaipplay.ini) (the value J/N from the list can be found in MNR.TNRSPERRE)

4.4 Integrate dynamic dialogs in information dialog

Dynamic dialogs can be integrated in information dialogs. The toolbar of the information dialog is then shown below the toolbar of the dialog:

Maschine 60610 Engel

BDE-Kommentare	Angemeldete Personen	Statusprotokoll	Beschreibung
Auftrag Arbeitsplatz 60610			
Datum/Zeit Maschine Kommentar	[07.10.2009 09:34:26] [60612] [444444444444]	Person Auftrag [S61003310100] [00050014 Albert,Claudia]	
Datum/Zeit Maschine Kommentar	[07.10.2009 09:33:47] [60612] [werqwerqwerwer]	Person Auftrag [S61003310100] [00040563 Griebner,Maria]	
Datum/Zeit Maschine Kommentar	[07.10.2009 09:33:21] [60612] [Kommentar 2]	Person Auftrag [S61003310100] [00040256 Pernikova,Lisa]	
Datum/Zeit Maschine Kommentar	[07.10.2009 09:32:54] [60612] [Kommentar (in ctwin erstellt)]	Person Auftrag [S61003310100] [00000667 Schulz,Christian]	
Datum/Zeit Maschine Kommentar	[21.01.2009 17:12:47] [LACK02] [FARBE LEICHT ABWEICHEND]	Person Auftrag [S30002000300] [00000667 Schulz,Christian]	
Datum/Zeit	[21.01.2009 14:26:07]	Auftrag [S30002000200]	
Seite 1 • Seite 2			
Abbrechen		Kommentar verfassen	
Maschineninformation schliessen			

Definition of a dialog for the machine info (ctaipay.ini):

[M-Info]

;;Dialog<n>=<DLG>,<Tab

Caption>

Dialog1=WF_BDE_KOM_CHK,BDE

comments

Sortierung=Dialog1,*

Up to 10 dialogs can be configured (Dialog1...Dialog10). The entry "Sortierung=..." causes the tab with the new dialog to be put at the beginning.

Configuration of the toolbar of the info dialog for the dialog (ctaipbut.ini):

[M_INFO.DIALOG1-Page1]

1=MI_CLOSE,L,close machine information

If the "cancel" key is defined in the dialog as described in the above example, it will have no effect here.

It must be noted that the tabs are displayed over several rows if too many tabs have been inserted. Consequently, the tabs are moved downwards. Relevant areas might reach beyond the display area. To avoid this, the configurable caption texts of the tabs should be as short as possible.

4.4.1.1.1 Restrictions

A dynamic dialog is generated, once it is called. Output data (machine, order) are already available.

The dialogs integrated in the information dialogs are already set up when starting the application. Consequently, placeholders included in text fields (e.g. machine <MNR> <MBEZK>) cannot be converted. The function cannot just be started later, as the placeholders have already been overwritten during initialization.

4.5 TextViewer in dynamic dialogs

BDE-Kommentare

Auftrag S30008880100 Arbeitsplatz 60614

Datum/Zeit	[07.10.2009 09:34:26]	Auftrag	[S61003310100]
Maschine [60612]	Person [00050014 Albert,Claudia]		
Kommentar	[444444444444]		
Datum/Zeit	[07.10.2009 09:33:47]	Auftrag	[S61003310100]
Maschine [60612]	Person [00040563 Griebner,Maria]		
Kommentar	[werqwerqwerwer]		
Datum/Zeit	[07.10.2009 09:33:21]	Auftrag	[S61003310100]
Maschine [60612]	Person [00040256 Pernikova,Lisa]		
Kommentar	[Kommentar 2]		
Datum/Zeit	[07.10.2009 09:32:54]	Auftrag	[S61003310100]
Maschine [60612]	Person [00000667 Schulz,Christian]		
Kommentar	[Kommentar (in ctwin erstellt)]		
Datum/Zeit	[21.01.2009 17:12:47]	Auftrag	[S30002000300]
Maschine [LACK02]	Person [00000667 Schulz,Christian]		
Kommentar	[FARBE LEICHT ABWEICHEND]		
Datum/Zeit	[21.01.2009 14:26:07]	Auftrag	[S30002000200]

Abbrechen Kommentar verfassen

Dialog configuration:

"General" tab:

Positioning by field X,Y

Size by unit X,Y

Felder ändern

Dialog WF_BDE_KOM_CHK Typ AIPDEF Dlg-User 0 Verweis 4341

Allgemein Format Funktionen Optionen

Aktiv an

☒ immer

☐ nur Konsole

☐ nur Terminal

Position

Text X 0 Y 0

Feld X 10 Y 50

Einh. X 770 Y 350

Feld-Nr. 3

Text

Einheit

Info Grid Kommentarliste

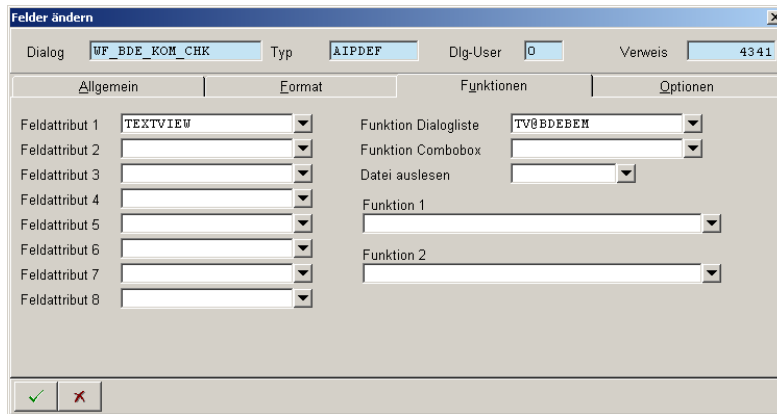
Kennung BEM

Kennung Index

✓ ✗

"Format" tab: selection of "grid"

"Functions" tab:



The field attribute "TEXTVIEW" defines the exact type of the "grid" pre-selection.

The field "dialog list function" includes the section of the ctaipplay.ini file in which further configurations are defined.

Ctaipplay.ini:

```
[TV@BDEBEM]
CMD=DLG=LIST; 61 | MNR=<MNR> | ANR=<ANR> |
FILE=BdeBem.lst
TEXTFILE=BdeBem.txt
CONVERSION=BDEBEM
FONTNAME=Arial
FONTSIZE=10
```

CMD: Command for loading the list. The placeholders <ANR>, <MNR> are replaced by the current values. The values must be available in the dialog and pre-assigned to default values.

FILE: This file is loaded by the above-mentioned command

TEXTFILE: This file is displayed

CONVERSION: Rule for the conversion from FILE to TEXTFILE (up to now BDEBEM available only). The file is only copied if nothing is entered.

FONTNAME, FONTSIZE: configuration of the font in the TextViewer

5 Central Configuration File hytnrcfg.ini

This file includes different configurations for all or single terminals at a central place.

Each section is available in a generally accepted version

[section 0].

However, entries included in this section can be overwritten by entries in a terminal-specific section [section <TNR-USER>]

<TNR-USER> = HydraUser = Terminal number + 2000 e.g. 2010,2101,..) for exactly one terminal/HYDRA User



The hytnrcfg.ini file is loaded from the server every time the terminal is started.

Section / Entry	Comment
	→→→
[Tnr configuration 0]	
FollowExternStatus=on	Transfer of machine statuses when reloading machine list Useful if status change is set by PDM or another terminal
[Terminal->Installation 0]	
InstallFonts=on	If this is set to "off" fonts will not be installed during the restart. ON=DEFAULT
OnlyInstallFontsAfterDownload=false	"InstallFonts=on": If true then fonts will only be installed directly after a download. If false then fonts will be installed every time the terminal is restarted. (false = DEFAULT)
InstallTvicport=on	If "off" the LPT driver "tvicport.sys" will not be installed. It is required for HYDRA-ZKS. ON = DEFAULT
[Terminal->USR 0]	

AttachedApplication=First	This configuration checks whether or not an application is connected in Windows that matches the file extension of the document to be displayed from the OP info dialog. If there is such an application, it will be used for displaying the document. If there is no connection, viewers configured in ctaip.ini (→ [ext. software]) and internal viewers will be used. In case, an extension is completely unknown it is attempted to display it as text Different settings may be configured: First → search for connected application first AfterUserViewer → If a UserViewer is configured this one overrides the connected application (also applies for ExcelViewer, WordViewer and PowerpointViewer) Last → Only if no ctaip.ini assignment is found for the file extension, then the connected assignment will be searched for (default). Off → Connected application is never searched.
HTTPBrowser=standard	Viewing of documents (via OP info): If documents are configured with a path of the <u>type</u> "http", the file will not be downloaded to the terminal, but the link will only be transferred to a browser. The default browser for the terminal is htmview3.exe, as this one can be operated by touchscreen. If this entry is set, the default browser configured in Windows will be used.
SupressErrorMessage=70012	Suppress message "material is not planned"
[SignatureRecording->User 0]	
ManualBadgeInput=true	This configuration specifies whether or not the field "user" can be edited in the terminal (by default: no editing) true → activates keyboard input for the "user" field in the terminal

Transparency=255	The signature dialog can also be transparent. 255 → Signature dialog is 0 % transparent (not transparent) 1 → Signature dialog is 99% transparent (maximum transparency) (Default = 155) Available as of CTAIP (V# 2.0.2.25) / CTWIN (V# 7.2.5.99)
ShowPosition=TR	The position of the signature dialog can be adjusted as follows: TL Top – Left TM Top – Middle TR Top – Right ML Middle – Left MM Middle – Middle (Default) MR Middle – Right BL Bottom – Left BM Bottom – Middle BR Bottom – Right Available as of CTAIP (V# 2.0.2.25)
USE_SERVICE_ACCOUNT=1	0 (default) SSO: do not use ServiceAccount (requires the terminal to be started with the "user" domain (SSO)). Please note: ServiceAccount=1 can only be used if all users are in the "root" domain. SubDomain users are not supported.
SIGNATURE_1_USER_TYPE=REPORTING_USER_READONLY	REPORTING_USER_READONLY The tab identifying users via the Windows user is activated and assigned to "user" by default. The "user" field is read-only. This requires, however, that in the HYDRA HR master the "SSO" option is set for all users logging in. Otherwise, successful authentication is impossible. REPORTING_USER_CHANGEABLE The tab identifying users via the Windows user is activated and assigned to "user" by default. The "user" field can be modified. This requires, however, that in the HYDRA HR master the "SSO" option is set for all users logging in. Otherwise, successful authentication is impossible.

SIGNATURE_1_LOGON_TYPE=HYDRA	"" / Not set / "EMPTY" There is also an alternative login procedure. HYDRA The tab identifying users via the Windows user is blocked. The HYDRA user must be used for identification purposes. This requires, however, that in the HYDRA HR master all users logging in are created and that the "SSO" option is not set. Otherwise, successful authentication is impossible. ACTIVEDIRECTORY The tab identifying users via the HYDRA user is blocked. The Windows user must be used for identification purposes. This requires, however, that in the HYDRA HR master the "SSO" option is set for all users logging in. Otherwise, successful authentication is impossible. MIXED_BUT_UNIQUE Either the HYDRA or Windows login procedure is available, subject to whether or not the "SSO" option is set for the registered user in the HYDRA HR master. "SSO" enabled → Windows only "SSO" disabled → HYDRA only
SIGNATURE_2_LOGON_TYPE=HYDRA	Identical to SIGNATURE_1_LOGON_TYPE (see above)
ExtendedSignatureRecording=true	Used for signatures with the terminal in the area of quality data collection.

5.1 Layout configuration

Entry		Comment
Section and/or	[terminal configuration 0] [terminal configuration 2XXX];	(general configuration) (2XXX terminal-specific configuration)
AUTO-CONFIRM-UHR-ERROR-MESSAGE=TRUE		In case of an error in reading the clock (e.g. after coming out of standby mode), this configuration makes sure that the time is accepted without having to confirm a dialog. Afterwards the terminal time will be synchronized with the server time using a PDM command.
SUPPRESS-MAXIMUM-NUMBER-OF-MACHINES-WARNING=ON		As of ctaip V# 2.0.2.23 Prevents the warning after restarting the terminal if more than 32 machines are assigned to the terminal (static/dynamic). (Default = OFF)

Entry	Comment
NetRuntimeMode=2	As of ctaip V# 2.0.2.50: Alternative calculation of the target quantity since logon: The net run time is not calculated from the times when the production lock is enabled (PSperre=green) but only from the shift times less the shift breaks. Consequently, it can also be displayed, even if the terminal program has been restarted.
Section [QRD-PRINTER->TICKET 0] ;(general configuration) [QRD-PRINTER->TICKET 2xxx] ;(2XXX configuration for a specific terminal)	
COMPLETE-ABSENCE-OF-LOCAL-MNR- DATA-FOR-EVENT=< Events >	Reloads the machine row for the configured <Events>, if it is not available locally => This configuration might be required/necessary for a group workplace without machine assignment.
COMPLETE-ABSENCE-OF-LOCAL-ANR- DATA-FOR-EVENT=< Events >	Reloads the order row for the configured <Events>, if it is not available locally ➔ This option has been implemented to access order data within the master data, e.g. when logging an order on.
COMPLETE-...-EVENT=< Events > COMPLETE-...-EVENT=#ALL# COMPLETE-...-EVENT=A_AN A_P_AN	Explanation on the configuration of <Events> ➔ Using <#ALL#> the row (ANR/MNR) that is not available is reloaded for any event. ➔ <A_AN A_P_AN> restricts reloading of information to the specified events. The ID <DLGFAM> is preferred to the ID <DLG> in order to identify the <Event>.

6 Local Configuration File keyboard.ini

Settings for the virtual keyboard are configured for specific AIP terminals in the keyboard.ini file of the directory c:\ctaip.



Changes to the configuration file will not take effect until after the terminal software has been restarted.

Logic enabling the virtual keyboard:

The AIP terminal shows the keyboard for fields where data can be entered. The keyboard is positioned as described below:

Logic for positioning of the virtual keyboard:

As soon as the activation command has been sent, the keyboard receives the coordinates for the center as well as information on the height and width of the control element to which it is added.

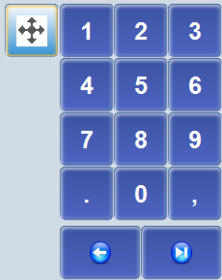
At first, an attempt is made to attach the keyboard directly below the control element. If the lower margin is not sufficient, an attempt is made to place the keyboard directly above the control field. If the space above the control element is not sufficient for the keyboard, an attempt is made to position the keyboard at the bottom margin of the screen. In case the space above the control panel is three times as large as the one below the control element, the keyboard will be positioned at the top margin of the screen.

These are the priorities for horizontal alignment:

- to the right of the control
- to the left of the control
- to the screen margin that is farther from the control

If the "VirtScreenSize" option is enabled, the virtual keyboard is not aligned within the virtual screen but still within the real screen. Consequently, the keyboard may also reach beyond the terminal program.

Entry	Comment
Section [User]	General settings

Entry	Comment
Key1=del,8,Delete Key2=<,37,Cursor left Key3=>,-39,Cursor right Key4=<<,36,Cursor at the beginning Key5=/	Definition of additional customer-specific keys displayed in the upper row:  Keys are configured with the following syntax in tasten32.ini within the section [User]: Key<i>=<CH>[,<Code>[,<Comment>]] Key<i>: Key1..Key5 <CH>: Characters displayed on the key and the code of which is sent successfully tested characters: \$=)/?'`{[]}#.;<>_*~:€äüöÖ²³@ª¿®Çüéâäæ/£ø£®½¼©¥ăµ Characters that lead to errors while testing: !"\$%(&,' <Code>: Code that is sent instead of the character code <Comment>: any comment
ButtonShift=ON	Show key "shift" The key has been designed for switching between upper case and lower case letters
ContextSensitive=off	Starting from version 2.0.1.5 of keyboard.exe (30 April 2012), the virtual keyboard automatically shows or hides letters. This depends on whether the current field is numeric or alphanumeric. This button disables the function so that the virtual keyboard is always "opened" ("dropped down"). 

Entry	Comment
HideTime=10 HideTime=0	Starting from version 2.0.1.5 of keyboard.exe (30 April 2012), there is a new button on the virtual keyboard allowing to disable/hide the keyboard for a configurable period of time (by default=10 sec.). Once switched to a new field, the virtual keyboard will immediately appear at the new position even if the hide period has not yet expired. The "hide" button is removed by configuring HideTime=0. 
HideMode=1 HideMode=2 Trace=1	HideMode=0 (by default) means the virtual keyboard appears, once another dialog (e.g. a message indicating "... is being loaded...") is opened and the input focus returns to the dialog. 1: The virtual keyboard is hidden over the indicated period, even though another AIP dialog is opened. The virtual keyboard is shown immediately after switching to another field of the dialog. 2: Like (1) but the virtual keyboard remains hidden even after switching to another field of the dialog. The virtual keyboard is only shown before the HideTime has expired, if the dialog is closed and reopened. Trace=1 enables logging (prot_kbd.txt) for the virtual keyboard. (AIP 2.0.3.25 / keyboard 2.0.2.3)
ScaleMultiplier=0.7	Additional scaling factor reducing or increasing the keyboard. This makes it possible to cover as less information as possible and to keep maximum user friendliness. This setting is useful, in particular, for large screens with low resolution and vice versa. (keyboard.exe V2.0.1.5)
KEYMODE=QWERTZ	Alphanumeric mode based on the German typewriter keyboard (keyboard.exe V2.0.2.2)
Section [SKIN]	General settings
[SKIN] Directory=..\..\..\tastatur\ skins Name=mpdv Saturation=0 Hue=0 SkinButton=on	Configuration of the skin (normally, default values are sufficient – the section is not required) Shows a button enabling and disabling the skin

7 Dynamic Dialogs - Workflow

Overview

Menu	System administration → Terminals → Workflow
Transaction code	ddconfw
Function authorization	ddconfw

Purpose

You can use the dialog configuration to change the AIP input dialogs in a quick and efficient manner according to the user's requirements.

The system delivery includes a basic dialog configuration. You can edit and change this configuration using the functions described in the following.

The function "Dynamic Dialogs - Workflow" defines the order of tabs in a complex dynamic dialog.



The complete functionality of the dialog configuration includes a lot of options, but it is also very complex. For this reason, we recommend to change the dialogs only after consultation with MPDV and only by experts.

Integration

To define the tab order, the fields and the buttons of dialogs, you must not only use the application "Dynamic dialogs - Workflow", but also the applications "Dynamic dialogs", "Dynamic dialogs - Fields" and "Dynamic dialogs - Function keys".

Requirements

The dialog that you want to configure must already exist in the "Dynamic dialogs".



Some of the functions require the development license MDS-AIS to be fully available. The restricted functions are marked in the document using "(*)".

Basics of the functions without development license:

- Existing data can be changed. Only data for default dialogs with user 0 must not be changed without development license.
- You cannot create new data without development license, except fields in existing dialogs.

- You can copy the existing data to terminal groups or terminals. Without development license, you cannot copy to default dialogs with user 0.

Selection criteria

The application provides the following selection criteria:

Workflow

Selection by workflow

Type

You can select from different dialog types:

- AIPDEF – default dialog
- AIPTNR – terminal dialog
- AIPTGRP – dialog for terminal group

Dlg user

Selection by terminal number or terminal group

Toolbar

Insert (*)

Creating a new workflow

Edit (*)

Editing a workflow



(*) Without development license, you cannot edit workflows. Editing a workflow is the same as creating a new workflow. You need a developer license to create new workflows.

Delete

[Deleting a workflow](#)

Copy

[Copying a workflow](#)

In addition to the standard function calls, the following function calls are available:



Dynamic dialogs

Function authorization: ddconf.*

The function "Dynamic Dialogs" calls the application [Dynamic dialogs](#).



Test dialog

Function authorization: ddconf.test

The function "Test dialog" opens the selected [Workflow](#) to test all settings and the functionality.



Save dialog configuration

Function authorization: none

The function "Save dialog configuration" saves the current dialog configuration.



Activate dialogs

Function authorization: ddconf.activate

The function *Activate dialogs* [activates the dynamic dialogs](#). The dialogs are then available on the terminals.

Field description

General

Workflow

Identifier of the workflow

The identifier can be assigned to a button on the AIP, for example. This button then opens the respective workflow dialog.

Type

AIPDEF:	default workflow
AIPTNR:	terminal workflow
AIPTGRP:	workflow for terminal group

User

User number restrictions

The default workflows have Dlg user "0" and type "AIPDEF".

Dialog

If you perform the function in the workflow using the server, the dialog identifier (DLG) specified in this field is transferred to the dialog data.

By specifying the dialog identifier via the "Dialog" field, you can define customer-specific workflows that send default dialogs to the server.

Title

Title of the workflow dialog. Using the notation <XXX>, you can define placeholders for important local information on the AIP. The title is then displayed as header in the workflow dialog.

Keep forced dialog sequence

If the field "Keep forced dialog sequence" is enabled, the order of the dialog steps is set and cannot be changed, i.e. you can only go to the next dialog step (tab).

User defined 1

Additional configuration options:

BUTTONHEIGHT=50

In case of workflows with several tabs, this configuration specifies the initial height of the button bar before scaling.



If you configure an alternative button height, the dialog field positions do not change automatically.

User defined 2...3

The fields "User defined 2...3" are currently not used.

Comment

You can use the field "Comment" to configure a description of the workflow.

Steps

Step 1...10

Name of the dialog configuration for dialog step 1...10. The dialog steps are displayed and performed in the specified order. You can use the different dialog steps in any workflow.

Script 1...10

W: The script of the workflow is run (and not the script of the dialog step).

S: The script of the dialog step is run (and not the script of the workflow).

Copying dynamic dialogs (workflow)

You can use the function "Copy" to copy complete workflow configurations of a dialog.

Function selection

Copy entire configuration

You can use the function "Copy entire configuration" to copy the complete workflow configuration, e.g. from the default configuration (AIPDEF 0) to a terminal (e.g. TNR nnn, with nnn= terminal number). If you copy the workflow configuration, the default configuration is still used for all terminals that do not have an own configuration. And later on you can change the custom configuration without affecting other users. If you use the mode "Copy entire configuration", the input fields "Workflow from" and "Workflow to" are hidden.

Copy workflow

If you use the function "Copy workflow", only the workflow entries for a selected dialog (workflow) are copied.



(*) Without a development license, you can only copy entire workflows or the entire configuration. You cannot copy to default workflows for user 0 without a development license.

Deleting dynamic dialogs (workflow)

If you use the function "Delete", you can delete the entries that include workflow data. The dialogs themselves are not deleted.



(*) Without a development license, you cannot delete default dialogs for user 0.

Testing dynamic dialogs

Use the function "Test dialog" to call the dialog. You can test its functionality or the separate steps.

Activate dialogs

If you use the function *Activate dialog*, the configured dynamic dialogs are available on the server and can then be downloaded to the terminals. Without activation, the dialogs and the possible changes are saved on the system, but the terminals still download the version activated last.

You can activate dialogs for single terminals or for terminal groups. When you activate a dialog, you specify the activation type and the user number that can be a terminal or a terminal group.

Type	Value of field User	Description
AIPTNR, TNR	Terminal number	Activates all dialogs for the terminal. If a dialog is not explicitly configured for the terminal number, the dialog AIPDEF/DEF with user 0 is

		activated. The activation for user 0 provides the default dialogs.
AIPTGRP, TGRP	Terminal group	Activates all dialogs for the terminal group. The system only activates the dialogs that are explicitly configured for the terminal group.

In general, dialogs that are configured for a terminal have a higher priority than dialogs for terminal groups. If dialogs are activated for a terminal, the dialogs of the respective terminal group are ignored.

If no dialogs are activated for the terminal group or for the terminal number, the terminal loads the dialogs that are activated for user number 0.

Porting notes: from AIP to AIP2 / workflow with one workflow step

The difference in the terminal script processing in AIP and AIP2 in case of a workflow with only one workflow step is as follows:

Example:

Workflow: [U_TST]
with only one dialog step: [WF_U_TST]

In the **AIP**, all "dynamic dialog user exits" have been performed as **workflow** script ("U_TST").

In the **AIP2**, the following activities are performed independent of the dynamic workflow/dialog configuration:

- The user exit "DynDlgInit_" is always executed as **workflow** script ("U_TST").
- All other user exits are called in the **dialog** tab script ("WF_U_TST").

As of the AIP2 version 8.2.1.10, you can use the following workflow configuration to specify the same processing in the workflow script ("U_TST") as in the AIP:

"Step 1" "WF_U_TST" (STEP:1= WF_U_TST)
"Script" "W" (WFSCR:1=W)

8 Dynamic Dialogs

Overview

Menu	System administration → Terminals → Dynamic dialogs
Transaction code	ddconf
Function authorization	ddconf

Purpose

You can use the configuration of the dynamic dialogs to change the AIP input dialogs in a quick and efficient manner according to the user's requirements.

The system delivery includes a basic dialog configuration. You can edit and change this configuration using the functions described in the following.

You can use the function "Dynamic dialogs" to configure and customize the dialogs on the AIP. With this function, the general dialog parameters and specific AIP options are specified.

The application *Dynamic dialogs* also includes the configuration of fields and function keys. You can therefore easily configure the input dialogs in this one application.



The complete functionality of the dialog configuration includes a lot of options, but it is also very complex. For this reason, we recommend to change the dialogs only after consultation with MPDV and only by experts.

Integration

To define the tab order, the fields and the buttons of dialogs, you need not only use the application *Dynamic dialogs*, but also the applications *Dynamic dialogs - Workflow*, *Dynamic dialogs - Fields* and *Dynamic dialogs - Function keys*.

Requirements

Some of the functions require the development license MDS-AIS to be fully available. The restricted functions are marked in the document using "(*)".



Basics of the functions without development license:

- Existing data can be changed. Only data for default dialogs with user 0 must not be changed without development license.

- You cannot create new data without development license, except fields in existing dialogs.
- You can copy the existing data to terminal groups or terminals and then change them. Without development license, you cannot copy to default dialogs with user 0.

Selection criteria

The fields **Dialog**, **Type** and **User** are provided as selection criteria.

Toolbars

The application *Dynamic dialogs* provides several toolbars.

Toolbar *Main page*

This toolbar includes functions to edit dialogs. The different functions are described [below](#).

Toolbar *Dynamic dialogs - fields*

The toolbar *Dynamic dialogs - fields* includes functions to edit [fields](#).

Toolbar *Dynamic dialogs - function keys*

The toolbar *Dynamic dialogs - function keys* includes functions to edit [function keys](#).

Functions of toolbar *Main page*

Insert (*)

Use the function *Insert* to create new dynamic dialogs in the system.

[Copy](#) (*)

Use the function *Copy* to copy complete configurations, single dialogs or parts of a dialog.



(*) Without development license, you can only copy complete dialogs or complete dialog configurations. Without development license, you cannot copy to default dialogs with user 0.

You require a development license for all other functions.

Edit

Use the function *Edit* to edit existing dynamic dialogs in the system.

Delete (*)

The function *Delete* deletes the selected dialogs including fields and buttons.

An undo function does not exist.



If you delete default dialogs (AIPDEF and DEF with user 0) (*), we strongly recommend to backup the dialogs before deletion.



(*) Without development license, you cannot delete default dialogs for user 0.



Test dialog

Function authorization: ddconf.test

Use the function *Test dialog* to call the dialog. You can test its functions, fields and function buttons.



Activate dialogs

Function authorization: ddconf.activate

The function *Activate dialogs* [activates the dynamic dialogs](#). The dialogs are then available on the terminals.



Save dialog configuration

Function authorization: none

The function *Save dialog configuration* saves the current dialog configuration on the server.



Enable simple dialogs

Function authorization: ddconf.actsdlg

Use the function *Enable simple dialogs* to activate the simplified dialogs that can be used when needed. These dialogs show all data that must be entered on one page.



Workflow

Function authorization: ddconfw.*

The function *Workflow* calls the application [Workflow](#).

Field description

Dialog

Dialog ID

Type

Dialog type:

- AIPDEF/DEF – standard dialog
- AIPTNR/TNR – terminal dialog
- AIPTGRP/TGRP – dialog for terminal group

User

According to type: terminal number or terminal group.



Note for the fields **Type** and **User**: there are some effects that are described in section **Activating dynamic dialogs**.

Key text

The field *Key text* is not used.

Resolution

You can use the field *Resolution* to scale the dialog and the controls (default = empty).

Short text

Use the field *Short text* to configure the tab title of the dialog. This text is shown when the dialog is used as tab in a workflow.

Long text

Use the field *Long text* to configure the dialog title of the dialog. This text is only shown if the dialog is a one-page dialog without workflow configuration.

Function 1

Use the field *Function 1* to select the function that is called when the dialog is opened.

If an entry starts with "FKT=", the function specified is called in the dialog script using the function `DynDlgFunctions_<DLG>()`.

Optionally, a preassignment of field *yield* is possible when *A_AB* or *A_UN*

in function 1 (when dialog is opened) or 2 (when ANR is exited):

- **SET_RESTME**
Remaining quantity = target quantity - yield – scrap (up to now) – scrap (in current dialog)
- **SET_RESTM2**
Remaining quantity = target quantity – yield

Other entries are still available for reasons of downward compatibility (CTWIN), but are not used in current software versions.

Function 2

Use the field *Function 2* to select the function that is called just before the dialog is closed.

Entries starting with "FKT=" are possible here. MPDV recommendation: with script functions called on closing a dialog, trigger these script functions via the script of the relevant function key because the context is known then.

Comment

The field *Comment* is not used.

Height

Use the field *Height* to configure the height of the dialog window.

Only relevant with CTWIN dialogs and POPUP windows (see options 2).

Width

Use the field *Width* to configure the width of the dialog window.

Only relevant with CTWIN dialogs and POPUP windows (see options 2).

Key

If you configure a "+" in field *Key*, the dialog ID is shown in the window title of a one-page dialog (e.g. "Log on OP and person" > "Log on OP and person <A_P_AN>"). This information is useful for non-German customers and for the support.

Key ID

The field *Key ID* is not used.

Activation

The field *Active* specifies if the dialog is active or not active. Cannot be modified.

AIP options**Licenses**

Use the field *Licenses* to optionally define one or several licenses, separated by semicolon.

- If the field *Licenses* is empty, the dialog step is always active.

- If licenses are entered in field *Licenses*, then the dialog step is only displayed if at least one of the licenses is available (OR conjunction).

Otherwise, the dialog step is **not displayed**.

Example:

DNC-BP;WRM-BP

Static condition

You can define one or several conditions via AND conjunction in field *Static condition*. The condition refers to the values of acronyms. For each acronym, you can enter one or several valid values, separated by semicolon.

Static conditions do not change in the course of a dialog. Static conditions are only evaluated when the workflow is opened.

If no condition is specified, the dialog step is always active.

If the condition is not fulfilled, the dialog step is **not displayed**.

Syntax for conditions:

Example of a value request:

MNR.MGRP=100

The machine group must be 100.

Example of array access/comparison

```
TNR.PARAM3 [ 5 ] =5
```

The fifth character must be 5 (counting starts with 1).

```
TNR.PARAM3 [ 3 . . 5 ] =345
```

The characters 3 to 5 must be 345.

Example of negated conditions, several values are allowed

```
XXX<>12;34 & YYY=34;56
```

This condition is true if the content of <XXX> does not equal "12" or "34" and the content of <YYY> is equal to "34" or "56".

Example of programmed functions

```
PRG:EMPTY->ABC
```

The condition is true if the acronym ABC is empty or the acronym does not exist.

```
PRG: [NOT] EMPTY->ABC
```

The condition is true if the ID ABC exists and is not empty.

Dynamic condition

You can define one or several conditions via AND conjunction in field *Dynamic condition*. The condition refers to the values of acronyms. For each acronym, you can enter one or several valid values, separated by semicolon.

Dynamic conditions refer to acronyms that you can enter or change in the dialog. They are evaluated when the system changes to the next workflow tab.

If no condition is specified, the dialog step is always active.

If the condition is not fulfilled, the dialog step is **deactivated**.

The syntax of dynamic conditions is the same as the syntax of static conditions.

Forced fields

You can use the field *Forced fields* to configure fields that are required for the processing but that are not configured as hidden fields. You configure several field acronyms using semicolon (e.g. KNR;PNR; . .). The field acronyms are then available in the dialog buffer.

User defined 1

Additional configuration options:

```
BUTTONHEIGHT=50
```

In case of workflows or dialogs with one tab only, this configuration specifies the initial height of the button bar before scaling (default 30).



The dialog field positions do not automatically change.

User defined 2

Additional configuration options:

Configuration of a pop-up window with the configured height and width before scaling.

Example: POPUP:150:50:clYellow

Syntax	pop-up window	POPUP	key	word
	X position	150	X position top left corner (default 5)	
	Y position	50	Y position top left corner (default 5)	
	color	clYellow	color of dialog background (default \$A0FFFF)	

Copying dynamic dialogs

You can use the function *Copy dynamic dialogs* (button *Copy* in the toolbar) to copy complete configurations, single dialogs or parts of a dialog.

Function selection

Copy entire configuration

You can use the function *Copy entire configuration* to copy a complete configuration, e.g. from the default configuration (AIPDEF 0) to a terminal (e.g. AIPTNR nnn, with nnn= terminal number) or to a terminal group (AIPTGRP nnn, with nnn = terminal group). If you copy the configuration, the default configuration is still used with all terminals that do not have their own configuration. And later on you can change the custom configuration without affecting other users. In this mode, the input fields *Dialog from* and *Dialog to* are hidden.

Copy complete dialog

Use the function *Copy complete dialog* to call the following three copy operations in one operation.

Copy dialog without buttons and fields (*)

Use the function *Copy dialog without buttons and fields* to copy the basic or dialog information of the dialog. Fields and function keys are not copied.

Copy buttons of a dialog (*)

Use the function *Copy buttons of a dialog* to copy only function keys. The target dialog must exist before the copy operation.

Copy fields of a dialog (*)

Use the function *Copy fields of a dialog* to copy only fields. The target dialog must exist before the copy operation.



When you copy dialogs, you can specify type and user for the source (*From*) and the target (*To*). If the complete configuration is copied, the input fields of *Dialog from* and *Dialog to* are hidden.



(*) Without development license, you can only copy in mode *Copy entire configuration* or *Copy complete dialog*. Without development license, you cannot copy to default dialogs with user 0.

Activate dynamic dialogs

If you use the function *Activate dialog*, the configured dynamic dialogs are available on the server and can then be downloaded to the terminals. Without activation, the dialogs and the possible changes are saved on the system, but the terminals still download the version activated last.

You can activate dialogs for single terminals or for terminal groups. When you activate a dialog, you specify the activation type and the user number that can be a terminal or a terminal group.

Type	Value of User	Description
AIPTNR, TNR	Terminal number	Activates all dialogs for the terminal. If a dialog is not explicitly configured for the terminal number, the dialog is activated with type AIPDEF/DEF and user 0. The activation for user 0 provides the default dialogs. The activated dialogs are stored as files on the server: Schema: \\<Server>\<InstDir>\<SystemNr>\spool\aip<User>.* Example: \\MyServer\mip3\3\spool\aip10.* or: Schema: \\<Server>\<InstDir>\<SystemNr>\spool\<User>.* Example: \\MyServer\mip3\3\spool\10.*
AIPTGRP, TGRP	Terminal group	Activates all dialogs for the terminal group. The system only activates the dialogs that are explicitly configured for the terminal group. The activated data does not include default dialogs. The activated dialogs are stored as files on the server: Schema: \\<Server>\<InstDir>\<SystemNr>\spool\aiptgrp<User>.* Example: \\MyServer\mip3\3\spool\aiptgrp900.* or: Schema: \\<Server>\<InstDir>\<SystemNr>\spool\tgrp<User>.* Example: \\MyServer\mip3\3\spool\tgrp10.*

When you **load the dialogs from the terminal**, only the activated configurations are used. The following priority applies:

1. If dialogs are activated for the terminal, only the dialogs activated for the terminal are loaded. No other dialogs are loaded.
2. If dialogs are activated for the terminal group, only the dialogs activated for the terminal group are loaded. No other dialogs are loaded.
3. If no dialogs are activated for the terminal or the terminal group, then the default dialogs are loaded. The default dialogs are the dialogs activated for terminal/AIP terminal 0.

Enable simple dialogs

In the system, simplified dialogs are available that may be used if required. On one page, these dialogs show all data that must be entered. The dialogs are activated for the default terminal user AIPDEF using the function *Enable simple dialogs*.

This setting affects the following input dialogs:

- Log on order → A_AN
- Log on order + person → A_P_AN
- Interrupt order → A_UN
- Finish order → A_AB
- Post part quantities (partial confirmation) → A_TR



The activation of the simplified dialogs can only be performed once.
You can only undo the activation if you manually change the workflow.



With older installations: It is possible that with the simple dialogs "Log on order (A_AN)" or "Log on order + person (A_P_AN)" the data transfer to the dialog fields does not work after selection of an operation. Here, the configuration in the ctaipplay.ini must be corrected.

In section [WF@ANR], you must enter the current standard file ctaipplay.ini in line DATAFIELDS.

9 Dynamic Dialogs - Fields

Overview

Menu	System administration → Terminals → Dynamic dialogs, Tab <i>Dynamic Dialogs - fields</i>
Transaction code	ddconf
Function authorization	ddconf

Other option:

Menu	System administration → Terminals → Dynamic dialogs - fields
Transaction code	ddconf
Function authorization	ddconf

Purpose

You can use the configuration of the dynamic dialogs to change the AIP input dialogs in a quick and efficient manner according to the user's requirements.

The system delivery includes a basic dialog configuration. You can edit and change this configuration using the functions described in the following.

You can edit the dialog fields in two places: in the tab integrated in the application *Dynamic dialogs* and in the application *Dynamic dialogs - fields*. You can change existing fields (e.g. positioning), create new fields or delete fields.



The complete functionality of the dialog configuration includes a lot of options, but it is also very complex. For this reason, we recommend to change the dialogs only after consultation with MPDV and only by experts.

Integration

To define the tab order, the dialogs and the buttons, you need not only use the application *Dynamic dialogs - fields*, but also the applications *Dynamic dialogs - Workflow*, *Dynamic dialogs* and *Dynamic dialogs - Function keys*.

Requirements

The dialog must exist in the *Dynamic dialogs* function.



Some of the functions require the development license MDS-AIS to be fully available. Basics of the functions without development license:

- Existing data can be changed. Only data for default dialogs with user 0 must not be changed without development license.
- You cannot create new data without development license, except fields in existing dialogs.
- You can copy the existing data to terminal groups or terminals and then change them. Without development license, you cannot copy to default dialogs with user 0.

With the fields of the dynamic dialogs, all functions are available without development license. But without development license, you cannot create, edit or delete default dialogs for user 0.

Selection criteria

The application provides the following selection criteria. The selection criteria are read-only. The values entered in the previous application are automatically preassigned.

Dialog

Selection by dialog

Type

Selection by dialog types:

- AIPDEF/DEF – standard dialog
- AIPTNR/TNR – terminal dialog
- AIPTGRP/TGRP – dialog for terminal group

User

Selection by terminal number or terminal group

Editing functions

The application "Dynamic dialogs - fields" only provides the usual editing functions: insert, edit and delete.

If you use the application *Dynamic dialogs*, you can change to the toolbar tab *Dynamic dialogs - fields*. This toolbar provides the usual editing functions and additionally the detail application *Edit fields*. Using this editing application, you can easily manage and edit the fields of a dialog.

Detail application *Edit fields*

In the detail application *Edit fields*, you can right-click the table view to open a **context menu**. The context menu provides the following functions:

New row

Inserts a new row in the grid for the definition of a field.

Several new rows

Inserts the specified number of rows in the grid.

Copy row

Copies the currently select row and inserts it in the grid.

Delete row(s)

Deletes the currently selected row(s) from the grid.

Swap fields

The selected rows are swapped. You can select one of the two methods:

- **Swap positions**
Swaps the X and Y positions of texts, fields and units of the two selected entries.
- **Swap field numbers**
The field numbers and the tab order of the fields are swapped.

Align fields

Automatically aligns function keys in the X or Y direction.

Move fields

Moves one or several fields in the X or Y direction. Buttons are moved using the specified offset ("Move by").

Apply fields from other dialog

Takes over several fields from the selected dialog.

Field description

Tab General**Activated**

This option is only available for reasons of downward compatibility. Always enable the option.

Field no.

Consecutive number of input field in dialog. Specifies the tab order in the dialog.

Text

Text label of field (on the left of the field).

Unit

Text for unit (on the right of the field).

Information

Information text displayed in tooltip if focused with mouse (mouseover).

Identifier

Identification of the data field in the dialog data string, e.g. ANR → |ANR=123456780100|

ID index

Index of field identification for similar data fields, e.g. |EGR:GUT=12340|.

Tab Position

X pos. text

X position of field label

Y pos. text

Y position of field label

X pos. field

X position of data field

Y pos. field

Y position of data field

Unit X pos.

X position of text for unit

Unit Y pos.

Y position of text for unit

Tab Format

Alignment

Alignment of text in input field.

"L" left left-aligned

"R" right right-aligned

Category

INPUT Field is an input field.

TEXT Alphanumeric text

GRID Table

OPTION Option field

RADIO Selection group (as of CTAIP: see "field attribute 1-8" → "EXTENDED")

Input type

Input type of data field.

ALPHA	alphanumeric
NUMERISCH	numeric without decimal places
FLIESS	numeric with decimal places
DATUM	date
ZEIT	time (00:00:00 - 23:59:59)
DAUER	time period (00:00:00 - 9...9:59:59)

Alternatively, a you can edit a preconfigured field type for user fields in the input type using the selection list, for example ANR. Only MPDV can change the field type for user fields.

Length

Total length of input field in characters.

Formatting

If a field type for user fields is entered in the "Input type" field, the formatting is done using the formatting rules defined for the field type.

If you use the simple input types (FLIESS,...), you can specify in field *Formatting* how the contents are displayed. Exceptions: types NUMERISCH and ALPHA; you cannot store a format for these types.

Sample configurations of input fields of the different input types:

FLIESS	###,### or -###,### or #####	with or without algebraic sign. The decimal separator must be "," or ".".
DAUER	hhhh:mm:ss	maximum 99999:59:59, h: hours with leading zeros
	-dddd:mm:ss	maximum 9999:59:59 + sign,
	d:	hours without leading zeros,
	-:	sign allowed.
	dd,iiii:	industrial format
DATE	dd.mm.yyyy	Default for date fields (need not be specified)
TIME	hh:mm:ss	Default for time indications
	hh:mm	Time without seconds (with leading zeros)
		(must be specified (mandatory))

Allowed characters

You can restrict the characters that can be used. The restriction only applies for the category "INPUT". If no characters are entered, there is no restriction.

e.g. A-Za-z0-9

Default

You can store a default value that is preassigned to the input field.

e.g. 1.5.

From

Lower limit of default (only relevant for EINGABE category)

To

Upper limit of default (only relevant for EINGABE category)

Optional field 1

Only relevant for OPTION category

Optional field 1: Value, if active

Optional field 2

Only relevant for OPTION category

Optional field 2: Value, if inactive

Radio button

Labels and values for radio buttons (only relevant for RADIO category),

For example: J:Yes:F7;N:No:F8;V:Maybe:F9

1st Value: Return value (identification = X), if selected

2nd Value: Text next to radio button

3rd Value: Function key (optional)

On the AIP, the function keys are automatically assigned to the dialog buttons using the sequence displayed. If the function key of the radio button is additionally configured as function key of a dialog button that triggers actions or is automatically assigned on the AIP, then the dialog button takes priority over the radio button.

Tab Functions**Field attribute 1 to 8**

Field attributes for input fields

- You can use "@AKRONYM/KENNUNG@" to configure an alternative identification to initialize dialog variables.

Application example: (dialog "TRANRLIST")

Dialog field/ID "MATPUF". The column "MPUFF" of the MNR.LST file includes the value initializing the field.

The field will be initialized properly when opening the dialog, if the field "MATPUF" is configured with field attribute "@MNR.MPUFF@". An additional terminal script is not required.

- If the field attribute "DATA.LEN=xxx" is configured, the input length of a field can be changed. The display length is not affected. Example: DATA.LEN=40 (input length 40).
- MANUELL – Input field is visible and editable
- STATUS – Input field is visible but not editable
- NULL - Input field may remain empty and is assigned with default value
- FOCUS - Input field is focused when the dialog is shown
- FOCUSNEXTFIELDONBARC – After editing the field using a serial barcode, the subsequent field is focused.
- BARCODE - Data can only be entered via barcode (keyboard locked)
- READONLY - This field cannot be edited.
- SETVALUE - if configured, the value included in the "default" field is used as the default value, e.g. for statuses (ID "MST") in the "log on OP" dialog. Without this identification, the default value is not written into the field. Note the following on the processing:
If a field is configured several times with SETVALUE (i.e. the same field is included in more than one tab of a workflow), the configuration with the lowest tab index "takes priority".
If a field is configured several times with SETVALUE on one workflow/tab page, the configuration with the greatest field index (no.) takes priority.
- UPPERCASE – Data input is converted into capital letters.
- PASSWORD – Input without visible characters ('***') - transmission is not encrypted
- PWD - Input without visible characters ('***') - transmission is performed with Blowfish encryption
- PWDRSA - Input without visible characters ('***') - transmission is performed with RSA encryption (only AIP2 as of V# 8.2.1.10 and hypdm32.dll V# 8.2.1.24)
- EMPTY – For NUMERIC fields only: if the field value is deleted, an empty input field is displayed instead of value 0.
- UNSELECT – prevents the whole field content from being selected, if the field is focused.
- DIALOGLISTE - modal list dialog that can be called (button behind input field)
Dialog list function must be filled (see below)
- COMBOBOX – Field contains combobox; Combobox function must be filled. (Is not used on the terminal).

General field attributes:

- AUTO - Input field is not visible on the terminal, but field ID is assigned
- LABELFONT – Font type and background color of label are used to display the input field. In combination with the NOBORDER parameter, an input field can be created that is displayed as a label.
- NOBORDER - The input field is displayed without depth effect.
- FIELDLABELFONT – The input field font and background color do not depend on the label. Font type and background color are displayed as configured in the file "dialog.ini" (section "layout", values of "FieldLabelFont") (as of CTAIP).

- **COLORLABELFONT** – The input field font and background color do not depend on the label. Font type and background color are displayed as configured in the file "dialog.ini" (section "layout", values of "ColorLabelFont") (as of CTAIP). For further information, refer to the section "Further descriptions", paragraph "[Field attribute <COLORLABELFONT>](#)".
- **FIELD** – shows variable "data" labels. Select the category "TEXT". The "identification" field must be filled (only CTWIN).
- **EXTENDED** – If configured in the field attribute, you can configure further properties with category <RADIO> in the following fields as of AIP:
 - Number of columns in field "length"
 - Width in field "X position of unit"
 - Height in field "Y position of unit"
- **AUTOTAB** – If the field is completely filled with characters, the cursor goes to the next input field. Example: If a barcode reader is connected via keyboard, the scan can take up several input fields (only CTWIN).

Field attributes of category GRID:

- **<XXX>_GRID** – Definition/function of table (<XXX> = variable; setting only by MPDV)

Entry	Description
AG_GRID	Table with running operations at the machine selected (A_AUT_HU, A_AUT_MPL, A_AUT_RF).
C_MG_BLZ_GRID	Table with quantity info on the input batches for the operation of the machine selected (C_MG_BLZ).
CA_INFO_GRID	Table with batches preceding the operation selected (C_VLOS_MPL, C_VLOS_RF, C_VLOS_S)
CE_ASW_GRID	Table with components of the operation selected (CE_ASW_RF).
CE_GRID	Table of components/input batches of the operation selected with processing (A_AN_[MPL,RF,S], A_P_AN_[MPL,RF], CE_WL[MPL,RF,S]).
CE_INFO_GRID	Table of components/input batches of the operation selected for display (CA_WL_MPL).
FHM_GRID	Table of the resources used at the selected machine (RES_WL).
KOMP_VERB_GRID	Table with components of the selected operation to enter discrete consumption (A_VERB).
PAL_GRID	Table to display the batches of a pallet (C_PALETTE).
SCRIPT_GRID	Table for the display of variable contents. The configuration/processing is defined in the relevant dialog script.
WF_GRID	Table for the display of variable contents. Here, the configuration/processing is performed via the configuration in the section (field <i>Dialog list function</i>) in the layout file (ctwinlay.ini/ctaiplay.ini).

Dialog list function

Function for the dialog list, which can be called on the terminal for dynamic dialogs.

Available dialog lists (entry in the LISTE field; case sensitive):

MNR_LISTE:	List of operations running at the machine
VAG_LISTE:	Order sequencing list (only CTWIN)
VMST_LISTE:	Machine status list
VGGRD_LISTE:	List of deviation reasons
VAGRD_LISTE:	List of scrap quantity reasons
VNCH_LISTE:	List of rework reasons (as of ADE 7.2/ MW2.0)
VPRB_LISTE:	List of problem quantity reasons (as of ADE 7.2/ MW2.0)
VLPKZ_LISTE:	List of premium indicators
VBPOS_LISTE:	List of operator positions
ZLO_LISTE:	List of material buffers (MPL)
TPE_LISTE:	List of transport units (MPL)
HZTYP_LISTE:	List of material types (MPL)
REQRES_LIST :	List of the allowed and assigned resources of a required resource (only AIP2)

Combobox function

Leave this field empty with current configurations. This option is only available for backward compatibility reasons and was used for selection lists on the clients from older system versions (before MW 3).

Select file

File used to read the list (only relevant in combination with the *Combobox function*).

Function 1

Function started upon entering the field, called by parameter list in/out.

Function 2

Function started upon leaving the field, called by parameter list in/out

Options

Blocked

If the field is blocked, it won't be made available by the AIP when the dialog is activated. Therefore, it is unknown in the AIP.

Visible

Invisible fields are processed in the AIP but not displayed. Invisible fields can be used to send fixed acronyms from the AIP to the server. A target value should be added to invisible fields and the field attribute SETVALUE.

Visible/invisible with dialog control

The meaning of this field/control changes whether the option *Visible* is set or not.

- 'Visible' set > Not visible when dialog control is enabled
- 'Visible' not set > Visible when dialog control is enabled

Depending on how the "*Visible*" option is set, you can use identifiers to specify when the field/control is displayed or not.

For further information, refer to the section "Further descriptions", paragraph "[Dialog control](#)".

DB table 1

DB tables, reserved for customization.

DB field 1

DB fields, reserved for customization.

DB table 2

DB tables, reserved for customization.

DB field 2

DB fields, reserved for customization.

User defined 1

Additional configuration options:

If your customer documentation includes no other specification, enter AUTO in this field.

User defined 3

Additional configuration options:

If you configure "FLD.LEN=xxx", you can configure a display length of a field that deviates from the standard, e.g. FLD.LEN=17 (display/input length 17 digits).



Customers often need this configuration to enter longer, customer-specific batch numbers. The input length of the default batch number is configured in the "length of batch no." field in the basic parameter settings.

Further descriptions

Dialog control

Configurations

A field *Dialog control* is provided with the following configurations:

- Machine/workplace configuration
- Order type configuration
- Terminal configuration (not yet available via GUI)

In field *Dialog control*, you enter an "identifier", which controls the further processing.

These identifiers are used in the application *Dynamic dialogs - fields* in tab *Options* in the fields -Visible J/N

- Visible with dialog control/Not visible with dialog control

to integrate the required control of the input fields.

Functionality

An input field/control in a dynamic dialog is displayed, if it is

- visible = J

or

- visible = N, but the *Visible with dialog control* field includes a (sub) string transferred by the *Dialog control* field of one of the configurations. Several "dialog control identifiers" can be specified, separated by semicolon.

An input field/control is not displayed, if it is

- visible = N

or

- visible = J, but the *Not visible with dialog control* field includes a (sub) string transferred by the *Dialog control* field from one of the three configurations. Several "dialog control identifiers" can be specified, separated by semicolon.

Example:

- Machine 4711, Field *Dialog control* = M1
- Order type 0, Field *Dialog control* = A1

In the application *Dynamic dialogs - fields*, the configuration is as follows:

- [] Visible
- Visible if [A1;M]

The input field or control is displayed if either M1 or A1 or both is true. This means: The posting is made for a machine where the field *Dialog control* includes M1 and/or an order/OP is logged on where the order type is configured with A1 in field *Dialog control*.



The fields that are hidden via dialog control are not sent to the server via dialog strings.

Field attribute <FIELDPAGER>

With a FIELDPAGER, the AIP generates several input fields for a configured field in the entry dialog depending on a list on the AIP. This can be used to collect scrap with several scrap reasons for partial confirmations and to delete and interrupt operations. The FIELDPAGER then generates automatically an input field for a valid scrap reason on any machine.

The multiple entry is only suitable if you need not display more than 80 elements.

The following fields are used from the dialog configuration:

Identifier

The identification gets the prefix and no „\$CT.“ and no index. Example:

Scrap: „\$CT.AUS:“

Rework: „\$CT.NCH:“

Open quantity: „\$CT.PRB:“

If the machine status is entered hierarchically, the identification is MST.

Positions

Position Field	FPOSX and FPOSY	= Top left corner of the input object
Position Unit	EPOSX and EPOSY	= Width and height of the input object

Alignment

"Left"

Category

„Grid“

Length

Input length of multiple fields, e.g. 8

Create an input format

The input format by default is integer. If inputs with decimal places are required, the input format can be the same as for normal input fields with the input type "FLIESS" and the formatting "####.##" (7 digits with 2 decimal places) or via a configured input type. The input types are restricted to numeric field types (with or without decimal places).

Field attribute 1

„FIELDPAGER“.

Dialog list function

Preconfigured fieldpager from a PAGER - section in the file ctaipplay.ini, for example "PAGER-AGRD.LST" (see below)

User defined 1

„AUTO“

The fields are generated using the font/size configurations designed for workflow fields (see "Dialog.ini")

- ➔ LabelFont... (Name, Size, Style, Color) (for multiple input field identifiers)
- ➔ FieldFont ... (Name, Size, Style, Color) (for multiple input field)

You can configure the available FIELDPAGER in a PAGER-section of the file ctaipplay.ini. The following configuration are prepared in the file ctaipplay.ini in the standard.

Section	Purpose
PAGER-AGRD.LST	Collection of scrap with several scrap reasons in case of partial confirmations, interruption and termination of operations. The FIELDPAGER generates a valid scrap reason for the input field at the machine.
PAGER-AGRD-NCH.LST	Collection of rework quantities with several reasons for partial confirmations or interrupting and terminating operations. The FIELDPAGER generates a valid rework reason for the input field at the machine.
PAGER-AGRD-PRB.LST	Collection of open quantities with several scrap reasons for partial confirmations or interruption and termination of operations. The FIELDPAGER generates a valid reason for open quantities for the input field at the machine.
MST-BUTTON-PAGER	Hierarchically input of the machine status.

Example for "Multiple scrap input".

Entry	Comment
Section [PAGER-AGRD.LST]	Configuration in workflows (dynamic dialogs) ➔ Field attribute:1 = FIELDPAGER ➔ List = PAGER-AGRD.LST
FILE=agrd.lst	Name of file used to generate multiple input fields
INI=	INI/configuration file including grid layout definition (Default = ctaipplay.ini)
SECTION=WF-PAGERPANEL-AGRD.LST	Section that includes the definition of the grid layout. The section "WF-PAGERPANEL-AGRD.LST" is required because the sorting does not work properly with ORDER sorting if file contents are unsorted. (Numeric sorting 1,2,3,4,5,10,11,22,... / alphanumeric 1,10,11,2.22.3.4.5....)

Entry	Comment
FILTER=MNR=<MNR> & ART=A	Possible filter to generate multiple input fields from the file <FILE>
ORDER=GRTXT	Possible sorting to generate multiple input fields from the file <FILE>
LabelColumn=GRTXT	Configuration of the column from the file <FILE> that has been designed as identifier for multiple input fields. (Default GRTXT)
LABELBEFOREFIELD=true	Optional: Identifier input field (Default=False)
LABELHEIGHTFAKTOR=1.25	Optional: Factor for calculating the height of a multi input field for positioning (default = 1.25)
LABELCHARCOUNT=20	Optional: Number of characters displayed for the label text of a multiple input field (see <i>LabelColumn</i>) (default=20)
IDCOLUMN=GR	Optional: Configuration of the column from the file <FILE> that includes the "KeyValue" of the row. (Default GR)
MODE=... SHOWIDCOLUMN=1 SHOWKEYLABEL=1 BUTTON-PAGER=1 or BUTTON-RESULT=1 BUTTON-RESULT=0 BUTTON-SKIN=BUTTON_BIG 	Extended options - shows (<IDCOLUMN>) in label (Default=0) - shows hotkeys 'a' .. 'z'(default=0) - item is shown as button (default=0) - button closes the dialog (default=1/active) ..as of V# 2.0.3.45 - the dialog remains open. The user exit „DynDlgFunctions_<dlg>“ with the function "ON(<KENN>)CLICK" is executed. - skin for button (default=BUTTON_BIG)
INCREMENT-BUTTON-COLOR=clSilver MODE=.. INCREMENT-BUTTON=TRUE	As of CTAIP V# 2.0.3.10 If the mode "INCREMENT-BUTTON=TRUE" is set, you can increment the value by 1 if you click on the label of the input field. Use the configuration „INCREMENT-BUTTON-COLOR=clSilver“ to change the label background. Default is "clSilver".

Entry	Comment
Section [WF-PAGERPANEL-AGRD.LST]	Configuration of a grid layout for the generation of multiple input fields
GR=N10,100,R	Definition that enables numeric sorting.
GRTXT=C10,200,L	Standard configuration for alphanumeric sorting.



If you use user exits at the AIP as part of the MES Development Suite, bear in mind that the DynDlgFieldchange event is not normally triggered for the fieldpager when a field is changed. This is required to calculate a sum field that might exist.

If the field attribute "**FIELDCHANGE**" is added to the Fieldpager, the event is also triggered for its input fields. You must insert the following line in DynDlgFieldchange_XYZ in your user exits to ensure that totals are still calculated.

Example for Fieldpager with ID \$CT.AUS::

```
Sub DynDlgFieldChange_A_TR
  Select Case VDlg("DLG.FLD")
    Case "$CT.AUS:"
      DLGVAR=Item("$CT.AUS:SUM",VDlg("$CT.AUS:SUM"))
  End Select
End Sub
```

Field attribute <COLORLABELFONT>

You can use the field attribute <COLORLABELFONT> to display texts with a special font. (see "Dialog.ini" -> ColorLabelFont[Name,Size,...])

The default field color is < blue > and can be used for the following dynamic dialog fields

GRID	- for Grid/Image/Shape/... Header
IMAGE	
SHAPE	
FIELDPAGER	

TEXTVIEW

METER

TEXT	- for text display
Input	- for field label

Use {c~color} to switch the color.

For example: "Log on {c~clblack} <ANR> {c~clblue} order"
 → "Log on AU0010001AG01 order"

The color configuration may also be used for the workflow caption. Here, the default font color is black.



Translation: If you want to translate the new texts/labels, you must add the texts to the relevant translation file and translate them (standard: ctaip.mld, custom: ctaipkd.mld).

10 Dynamic Dialogs - Function Keys

Overview

Menu	System administration → Terminals → Dynamic dialogs, tab "Dynamic dialogs - function keys"
Transaction code	ddconf
Function authorization	ddconf

Other option:

Menu	System administration → Terminals → Dynamic dialogs - function keys
Transaction code	ddconfb
Function authorization	ddconfb

Purpose

You can use the dialog configuration to change the AIP input dialogs in a quick and efficient manner according to the user's requirements.

The system delivery includes a basic dialog configuration. You can edit and change this configuration using the functions described in the following.

You can edit the function keys of dialogs in two places: in the tab integrated in the application "Dynamic dialogs" and in the application "Dynamic dialogs - function keys". You can change existing function keys (e.g. positioning), create new function keys or delete others.



The complete functionality of the dialog configuration includes a lot of options, but it is also very complex. For this reason, we recommend to change the dialogs only after consultation with MPDV and only by experts.

Integration

To define dialogs; fields and the order of tabs, you must not only use the application "Dynamic dialogs - function keys", but also the applications "Dynamic dialogs - workflow", "Dynamic dialogs" and "Dynamic dialogs - fields".

Requirements

The dialog that you want to configure must already exist in the "Dynamic dialogs".



Some of the functions require the development license MDS-AIS to be fully available. The restricted functions are marked in the document using "(*)".

Basics of the functions without development license:

- Existing data can be changed. Only data for default dialogs with user 0 must not be changed without development license.
- You cannot create new data without development license, except fields in existing dialogs.
- You can copy the existing data to terminal groups or terminals and then change them. Without development license, you cannot copy to default dialogs with user 0.

Selection criteria

The application provides the following selection criteria:

Dialog

Selection by dialog

Type

You can select from different dialog types:

- DEF – standard dialog
- TNR – terminal dialog
- TGRP – dialog for terminal group

User

Selection by terminal number or terminal group

Editing functions

The application "Dynamic dialogs - function keys" only provides the usual editing functions: insert, edit and delete.

If you use the application "Dynamic dialogs", you can change into the toolbar tab "Dynamic dialogs - function keys". This toolbar provides the usual editing functions and additionally the detail application "Edit function keys". Using this editing application, you can easily manage and edit the function keys of a dialog.

Detailed application "Process function keys"

In the detail application "Edit function keys", you can right-click the table view to open a **context menu**. The context menu provides the following functions:

New row (*)

The function "New row (*)" adds a new empty row to the grid to define a function key.

Several new rows (*)

The function "Several new rows (*)" adds the specified number of new empty rows to the grid.

Copy row (*)

The function "Copy row (*)" copies the currently selected row and adds it to the grid.

Delete row(s) (*)

The function "Delete row(s) (*)" deletes the currently selected row(s) from the grid.

Swap function keys (*)

Using the function "Swap function keys (*)", the selected rows are swapped. The following types of swapping exist:

- **Swap position**

With "Swap positions", the X and Y positions for function keys of the two selected entries are swapped (relevant CTWIN + ACTIONBUTTON).

- **Swap button no.**

With "Swap button no.", the button number and therefore also the tab order of the function keys is swapped.

Align buttons (*)

Using the function "Align buttons (*)", an automatic alignment of the buttons in the x or y direction is possible.

Move buttons (*)

Using the function "Move buttons (*)", you can move one or several function keys in the x or y direction. Buttons are moved using the specified offset ("Move by").

Apply function keys from other dialog (*)

Using the function "Apply function keys from other dialog (*)", you can take over several function keys from the dialog selected.

Field description

Button no.

The field "Button no. (*)" specifies the sequence number of the button in the dialog. This number specifies the tab order.

Return code

The field "Return code" defines the further processing after confirmation of the function key.

0: Dialog is closed and the dialog string is sent to the server.

1: Dialog is canceled.

7: Dialog is closed, the dialog string is returned, but not sent.

8: Dialog is not closed; the virtual keyboard must still be displayed.
(relevant with CTWIN)

9: Dialog is not closed.

Identifier

The field "Acronym" includes the value of the acronym for the return value; is returned as BTN=<acronym>

ID index

The field "Acronym index" includes the index for the return value in case of several similar data fields; is supplied with KENN, e.g. "..|BTN=BTN:UNDO|.."

Activated

This option is only available for reasons of downward compatibility. Select the option "Always" as of MW 3.x.

Key

Only CTWIN: You can use the field "Key" to configure the function key for the activation (hotkey) F1 to F12. By default, the key assignment is displayed on the button. The display is blocked if you place a "*" before the definition (e.g. KEY=*F1)

AIP: The function keys of the dialog are automatically assigned in the order of display to the function keys F1 to F12 of the keyboard.

Text

The field "Text" includes the button text (label) displayed.

X pos.

The field "X pos." specifies the X position of the button (top left corner of the button). (Relevant with CTWIN + ACTIONBUTTON).

Y pos.

The field "Y pos." specifies the Y position of the button (top left corner of the button). (Relevant CTWIN + ACTIONBUTTON).

Width

The field "Width" specifies the button width. (Relevant CTWIN + ACTIONBUTTON).

Height

The field "Height" specifies the button height. (Relevant CTWIN + ACTIONBUTTON).

Information

The field "Information" specifies the info text displayed as tooltip if the button is moused over.

Symbol

The field "Symbol" specifies the name of the assigned button icon.

(*): AIP: The available icons are included in the file pict.zip of the installation directory of the terminal.

Function

Function called via in/out.

For example:

Entry	Description
DLG=...	Calling a dialog
DLG=...;BREAK-ON-CANCEL	The dialog is not closed upon cancellation of the script dialog called, regardless of the "Return code" configured.
FKT=... (*)	Calling a script function
A_INFO	Calling the operation information (if an operation number is available in the dialog context).
A_AB_MPL A_UN_MPL A_TR	Calling the dialog "Log OP off", "Interrupt OP", "Partial confirmation" from the dialog "Interr/logoff/part.conf. OP" at an MPL machine.
A_AB_RF A_UN_RF A_TR_RF	Obsolete, not processed anymore.
C_VLOS	Calling the dialog "Show preceding batches" (C_VLOS_MPL,C_VLOS_RF)
CE_MLD	If you perform the function "Post batch", the dialogs - "Log off batch" (CE_AB,CE_AB_RF)" and - "Log on batch" (CE_AN,CE_AN_RF)" are used to change batches.
C_PAL_GEN	Generating a new batch number for a new pallet
CNR_ABF	Calling the dialog "Batch waste"
CNR_ADD	Execute function "PALTR.INSERT"
CNR_CHG	Calling the dialog "Modify batch length (C_CNR_LEN)" for update with "PALTR.UPDATE"
CNR_DEL	Execute function "PALTR.DELETE"
CNR-UNDO	UNDO function of a new batch number (only in dialog "Enter GR batch (C_GEN)")
DLG_CHECK	General function to check dialog input (only in dialog "Quantity balancing (C_MG_BLZ)")
ELW	Calling the dialog "Change input batch (CE_WL_MPL,CE_WL_RF)"
ELW_AB ELW_WL	Only in dialog "Quantity balancing (C_MG_BLZ)" Calling the dialog "Log off input batch (CE_AB,CE_AB_RF)" Performing the functions dialog "Log off input batch (CE_AB, CE_AB_RF)" and

	dialog "Log on input batch (CE_AN, CE_AN_RF)"
GRID_DOWN	General functions for navigating in tables (only CTWIN)
GRID_UP	next row
GRID_LEFT	previous row
GRID_RIGHT	column left
GRID_PAGEDOWN	column right
GRID_PAGEUP	next page
	previous page
PRINT	Calling the print (only CTWIN and in the dialogs "Form pallet (C_PAL_ASW)" and "Pallet (C_PALETTE)")
SEND	General function to perform server posting (only in the dialogs "Enter GR batch (C_GEN)" and "Repost batch (C_UMB)")
VERBRAUCH:RELOAD (consumption:reload)	Function "Reset (refresh consumption)" only in dialog "Component consumption posting (A_VERB)"
VERBRAUCH:START (consumption:start)	Calling the dialog "Component consumption posting (A_VERB)"
VTST	Showing/hiding the virtual keyboard (only required with CTWIN)

License

Optional: The field "License" includes the license required to activate the function key. If the field is empty, the key is always active.

User defined 1

The field "User def. 1" can include additional configuration options.

If you specify the value ACTIONBUTTON in field "User def. 2", you can configure the button layout in field "User def. 1".

Example: „clYellow,2,5,clBlack“

Syntax	color	clYellow	(default \$0080FF)
	layout	2=rectangle	(0=capsule (default), 1=ellipse, 2=rectangle)
	corner radius	5	(default 10, only with layout 2=rectangle)
	font color	clBlack	(default clBlack)

User defined 2

In field "User def. 2", you can configure the following configuration options:

FORM-VALIDATION

Before executing the button function, the system checks if the contents of the input fields are valid, as it does when the dialog is closed.

ACTIONBUTTON:

As with "FORM-VALIDATION", the system checks if the contents of the input fields are valid.

You are free to position the ACTIONBUTTONS in the dialog. You specify position and size using the fields "X pos.", "Y pos.", "Width" and "Height". You specify the layout in field "User def. 1".

Blocked

If the field "Blocked" is selected, the button is not displayed and not processed. If dialogs are activated, the button is not passed to the terminal.

11 User Exit Reference CTWIN/AIP

11.1 Introduction

This document describes the processing of terminal scripts.

On the terminal, two types of scripts are available:

- USER EXIT:
 - Customer-specific implementation of Hydra standard functions (e.g. generation of batch numbers).
 - Configuration of customer-specific evaluations (z.B. extension of the "OP info" on the terminal to start "Quick reports")
- DIALOG – script processing:
 - Dialog initialization (default assignment,...)
 - Dialog - control (FIELDCHANGE, FIELDEXIT, FIELDLISTE, GRIDINIT, FUNCTIONS)
 - Dialog - message (before and after sending dialog data to the server)

Note(s):

Translation of texts/postings with function "scrTranslate()"

Parameters for development in (ctwin.ini/ctaip.ini → system → parameters)

parameters= ... -AskForOverwriteScriptFiles -AlwaysReloadScript ...

- | | |
|-------------------------------|---|
| "-AskForOverwriteScriptFiles" | → Confirm before unpacking the terminal zip files
Prevents overwriting of locally modified scripts |
| "-AlwaysReloadScript" | → Downloads script files from the hard disk, if allowed.
(enables changes at runtime) |
| "-NeverOverwriteScriptFiles" | → No script update is performed. (without confirmation) |
| "-AskForRemoveDirectory" | → With confirmation before deleting the directories <ul style="list-style-type: none"> - .etc\var\ ; directory for partner extensions - .etc\local\ ; directory for customer extensions Prevents the deletion of local partners/customers extensions by deleting the respective directory |
| "-NeverRemoveDirectory" | → Above directories are not deleted; no confirmation |

11.1.1 Storage

ZIP files are only unpacked in live operation, once they have been successfully DOWNLOADED from the server.

11.1.1.1 CTWIN

File names and directories of default/customized terminal scripts must be named as follows.

	CTWIN	Description
MPDV	.lctnet\win\ai\pletc\tnr_mpdv.zip	MPDV standard (not used)
MPDV	.lctnet\win\ai\pletc\mpdv-tnr.zip	MPDV standard
CUST	.lcustom\userexit\tnr_<customer no.>.zip	Customizations with customer number
CUST	.lcustom\userexit\tnr_<project>.zip	Customizations with project abbreviation
CUST	.lcustom\userexit\<customer no.>-tnr.zip	Customizations with customer number
VAR	.lcustom\userexit\tnr_<project>@var.zip	Partner customizations with project abbreviation (as of v# 7.2.6.36)
VAR	.lcustom\userexit\tnr@var.zip	Partner customizations (as of v# 7.2.6.36)
LOCAL	.lcustom\userexit\tnr_<project>@local.zip	Customizations with project abbreviation (as of v# 7.2.6.36)
LOCAL	.lcustom\userexit\tnr@local.zip	Customizations (as of v# 7.2.6.36)

11.1.1.2 AIP

File names and directories of default/customized terminal scripts must be named as follows.

	AIP	Description
MPDV	.lctnet\win\ai\pletc\aip_mpdv.zip	MPDV standard (not used)
MPDV	.lctnet\win\ai\pletc\mpdv-aip.zip	MPDV standard
CUST	.lcustom\userexit\aip_<customer no.>.zip	Customizations with customer number
CUST	.lcustom\userexit\aip_<project>.zip	Customizations with project abbreviation
VAR	.lcustom\userexit\aip_<project>@var.zip	Partner customizations with project abbreviation (as of v# 2.0.2.49)

	AIP	Description
VAR	.lcustom\userexit\aip@var.zip	Partner customizations (as of v# 2.0.2.49)
LOCAL	.lcustom\userexit\aip_<project>@local.zip	Customization with project abbreviation (as of v# 2.0.2.49)
LOCAL	.lcustom\userexit\aip@local.zip	Customization (as of v# 2.0.2.49)

11.1.2 Processing

11.1.2.1 CTWIN

The terminal script files for the CTWIN are processed using the following priorities.

	PRI O	CTWIN	Description
MPDV	1.1	.l\system_mpdv.scr .l<dialog>_mpdv.scr	MPDV standard (not used)
MPDV	1.2	.l\mpdv-system.scr .l\mpdv-<dialog>.scr	MPDV standard
CUST	2.1	.l\system_<customer no.>.scr .l<dialog>_<customer no.>.scr	Customizations with customer number
CUST	2.2	.l\system_<project>.scr .l<dialog>_<project>.scr	Customizations with project abbreviation
CUST	2.3	.l<kdnr>-system.scr .l<kdnr>-<dialog>.scr	Customizations with customer number
VAR	3.1	.lvar\system_<customer no.>@var.scr .lvar<dialog>_<customer no.>@var.scr	Partner customizations with customer number (as of v# 7.2.6.36)
VAR	3.2	.lvar\system@var.zip .lvar<dialog>@var.zip	Partner customizations (as of v# 7.2.6.36)
LOCAL	4.1	.llocal\system_<customer no.>@local.scr .llocal<dialog>_<customer no.>@local.scr	Customization with customer number (as of v# 7.2.6.36)
LOCAL	4.2	.llocal\system@local.scr .llocal<dialog>@local.scr	Customization (as of v# 7.2.6.36)

11.1.2.2 AIP

The terminal script files for the AIP are prefixed by "aip_". The processing uses the following priorities.

	PRIORITY	AIP	Description
MPDV	1.1	.laip_system_mpdv.scr .laip_<dialog>_mpdv.scr	MPDV standard (not used)
MPDV	1.2	.laip_mpdv-system.scr .laip_mpdv-<dialog>.scr	MPDV standard
CUST	2.1	.laip_system_<customer no.>.scr .laip_<dialog>_<customer no.>.scr	Customizations with customer number
CUST	2.2	.laip_system_<project>.scr .laip_<dialog>_<project>.scr	Customizations with project abbreviation
VAR	3.1	.lvarlaip_system_<customer no.>@var.scr .lvarlaip_<dialog>_<customer no.>@var.scr	Partner customizations with customer number (as of v# 2.0.2.49)
VAR	3.2	.lvarlaip_system@var.zip .lvarlaip_<dialog>@var.zip	Partner customizations (as of v# 2.0.2.49)
LOCAL	4.1	.llocalaip_system_<customer no.>@local.scr .llocalaip_<dialog>_<customer no.>@local.scr	Customization with customer number (as of v# 2.0.2.49)
LOCAL	4.2	.llocalaip_system@local.scr .llocalaip_<dialog>@local.scr	Customization (as of v# 2.0.2.49)

11.2 User Exits on the Terminal

Customer-specific user exits are implemented in a customer-specific "system" script file. The customer-specific script uses general script functions of the standard "system" script file.

For notes on the storage and naming of the system script, refer to the section "1.2 Storage". For notes on the processing, refer to the section "1.3 Processing".

11.2.1 CTWIN

This section describes all user exits that are only available on the CTWIN.

11.2.1.1 UserExitCAQStatusChange

You use this user exit to process an inspection characteristic if the inspection status changes (checked, missed, due,...).

The variable UE:DAT (function VVar ("UE:DAT", "<ID>")) includes the characteristic data, as well as STATUSALT = color of old status, STATUSNEU= color of new status, START=specifies if the terminal is being restarted, Neuauftrag = order is currently logged on, and MNR for the machine number.

```
' Example 1: (Script to create a missed sample, in case of change from due to missed)
' *****
' *** User exit with status change of inspection characteristics
' *****
Sub UserExitCAQStatusChange
' *****
    dim PAN, PAU, AFO, StatusNeu, StatusAlt, Musterzug, Neustart, DLGStr
    PAN = VVar("UE:PAR", "PANNR")
    PAU = VVar("UE:PAR", "PAUNR")
    AFO = VVar("UE:PAR", "AFO")
    StatusNeu = VVar("UE:PAR", "STATUSNEU")
    StatusAlt = VVar("UE:PAR", "STATUSALT")
    Musterzug = VVar("UE:PAR", "FU:6")
    Neustart = VVar("UE:PAR", "START")
    '*** equipped with variables
    IF neustart = "0" THEN
        IF Musterzug = "1" THEN
            if (StatusNeu = "R") AND (StatusAlt = "G") THEN
                DD_SND = AddIt("DLG", "Q_PRB_GEN", "")
                DD_SND = AddIt("EVENT", "EVENT_OHNE_AUTO_MENGEN", "")
                DD_SND = AddIt("BER", VVar("UE:PAR", "BER"), "")
                DD_SND = AddIt("RECTYP", VVar("UE:PAR", "RECTYP"), "")
                DD_SND = AddIt("PANNR", VVar("UE:PAR", "PANNR"), "")
                DD_SND = AddIt("AFO", VVar("UE:PAR", "AFO"), "")
                DD_SND = AddIt("FU:1", VVar("UE:PAR", "UE:1"), "")
                DD_SND = AddIt("FU:2", VVar("UE:PAR", "UE:2"), "")
                DD_SND = AddIt("FU:3", VVar("UE:PAR", "UE:3"), "")
                DD_SND = AddIt("FU:4", VVar("UE:PAR", "UE:4"), "")
                DD_SND = AddIt("FU:5", VVar("UE:PAR", "UE:5"), "")
                DD_SND = AddIt("FU:6", VVar("UE:PAR", "UE:6"), "")
                DD_SND = AddIt("FU:7", VVar("UE:PAR", "UE:7"), "")
                DD_SND = AddIt("FU:8", VVar("UE:PAR", "UE:8"), "")
                DD_SND = AddIt("FU:9", VVar("UE:PAR", "UE:9"), "")
                DD_SND = AddIt("FU:10", VVar("UE:PAR", "UE:10"), "")
                DD_SND = AddIt("FU:11", VVar("UE:PAR", "UE:11"), "")
                DD_SND = AddIt("FU:12", VVar("UE:PAR", "UE:12"), "")
                DD_SND = AddIt("FU:13", VVar("UE:PAR", "UE:13"), "")
                DD_SND = AddIt("FU:14", VVar("UE:PAR", "UE:14"), "")
                DD_SND = AddIt("EINTTYP", "PROBE", "")
                DD_SND = AddIt("EINTNR", "", "")
                DD_SND = AddIt("MOD:VERSAEUMT", "1", "")
                DD_SND = AddIt("ANR", VVar("UE:PAR", "ANR"), "")
                DD_SND = AddIt("AUNR", VVar("UE:PAR", "AUNR"), "")
                DD_SND = AddIt("AGNR", VVar("UE:PAR", "AGNR"), "")
                DD_SND = AddIt("MNR", VVar("UE:PAR", "MNR"), "")
                DD_SND = AddIt("BEARB", "HYDRA", "")
                scrDDSnd
            END IF
        END IF
    END IF
' *****
End Sub
' *****

' Example 2: (script for the update to the measured value of a completed sample,
' starting from the missed status)
' *****
' *** User exit with status change of inspection characteristics
' *****
Sub UserExitCAQStatusChange
```

```

*****
dim PAN, PAU, AFO, StatusNeu, StatusAlt, Neustart, mnr, VersaeumteMin, LastDevSpAbs
PAN = VVar("UE:PAR", "PANNR")
PAU = VVar("UE:PAR", "PAUNR")
AFO = VVar("UE:PAR", "AFO")
StatusNeu = VVar("UE:PAR", "STATUSNEU")
StatusAlt = VVar("UE:PAR", "STATUSALT")
Neustart = VVar("UE:PAR", "START")
mnr = VVar("UE:PAR", "MNR")
'missed time in minutes
VersaeumteMin = VVar("UE:PAR", "VERSAEUMT_MIN")
'last device sample number from CPAUMW.INSERT or. CPAUMW.ABSCHLIESS
LastDevSpAbs = VVar("UE:PAR", "LAST_DEVSP_ABS")
if neustart = "0" then
    'Logging-Func
    scrLog("PAN, AFO, StatusAlt, StatusNeu = " + PAN + ", " + AFO + ", " + StatusAlt + ", " +
StatusNeu)
    'update to the measured value of a completed sample
    '(this measurement has completed the sample)
    'but only, if the completion started from status "missed"
    if (StatusAlt = "R") then
        DD_SND = AddIt("DLG", "CPAUMW.UPDATE", "")
        DD_SND = AddIt("CPAUMW.RECTYP", VVar("UE:PAR", "RECTYP"), "")
        DD_SND = AddIt("CPAUMW.BER", VVar("UE:PAR", "BER"), "")
        DD_SND = AddIt("CPAUMW.PANNR", VVar("UE:PAR", "PANNR"), "")
        DD_SND = AddIt("CPAUMW.PAUNR", VVar("UE:PAR", "PAUNR"), "")
        DD_SND = AddIt("CPAUMW.AFO", VVar("UE:PAR", "AFO"), "")
        DD_SND = AddIt("CPAUMW.DEVICE:STPRNR", LastDevSpAbs, "")
        DD_SND = AddIt("CPAUMW.DEVICE:TYP", "MASCHINE", "")
        DD_SND = AddIt("CPAUMW.DEVICE:ID", VVar("UE:PAR", "MNR"), "")
        DD_SND = AddIt("CPAUMW.WERTNR", "LAST", "")
        DD_SND = AddIt("CPAUMW.BEM:PRAEFIX", VersaeumteMin, "")
        scrDDSD
    end if
end if
*****
End Sub
*****

```

11.2.1.2 UserExitOnBtnIniPaint

You use this user exit to control the button bar with active <ctwinbut.ini>.

Parameters:

(1) Button bar is active / is to be drawn:

```

UE:PAR=BTN.INI=REPAINT|BTN.PAGE=1|BTN.IDX=7|A_AN_RS=#F1|#F1=A_AN_RS|..
|C_GEB_ASW=#F11|#F11=C_GEB_ASW|C_GG_ASW=#F12|#F12=C_GG_ASW|

```

(2) Active machine row:

```

UE:MNR=MNR=2600|MGRP=2600|MBEZK=VS 26| ... |FU:8=826|PDV=NR|

```

(3) Active order row (if an order is running):

```

UE:ANR=ROW.IDX=1|ANR=403216010100|AUNR=40321601|..|AUNR_KDAUPOS=10|

```

Result: (possible using <FKT.ID> or <#F[1..12]>)

```

(1) UE:RET=RET=*<@P_AUTOMATIK=clLime|
(2) UE:RET=RET=*<#F7=#FALSE# |

```

```

*****
Sub UserExitOnBtnIniPaint
'-----
dim fu33, mnr, sta
' SYS_SCRIPT_DEBUG
'-----

```

```

fu33 = VVar("UE:MNR","FU:33")
If VVar("UE:PAR","@P_AUTOMATIK") = "" Then
    Exit Sub ' FKT.BTN <@P_AUTOMATIK> not active
End If
If fu33 = "J" Then
    Mnr = VVar("UE:MNR","MNR")
    sta = GVars("#MNR#+mnr+#","AUTOMATIK")
    If sta = "" or sta = "J" Then
        UE_RET = Item("@P_AUTOMATIK","clLime")
    Else
        UE_RET = Item("@P_AUTOMATIK","clRed")
    End If
Else
    UE_RET = Item("@P_AUTOMATIK","#FALSE#")
End If
'-----
End Sub 'UserExitOnBtnIniPaint
'*****

```

Example: Color a button and change label text

```

...
    UE_RET = Item("@P_AUTOMATIK","clLime;Automatik (ON)")
...

```

For the button “@P_AUTOMATIK”, the color code “clLime” and the label text “Automatik (ON)” is passed in this code section separated by semicolon.

11.2.1.3 UserExitCAQSampleCreate

You call this user exit to create a sample. In the UE_RET, the dialog string to be sent is available and can be modified.

```

'*****
Sub UserExitCAQSampleCreate
'*****
    UE_RET = Item ( "CPANUMP.FU:1", VVar("UE:PAR", "EINTTYP"))
    UE_RET = Item ( "CPANUMP.FU:2", VVar("UE:PAR", "EINTNR"))
'*****
End Sub
'*****

```

11.2.1.4 UserExitCAQDueEvent

You call this user exit if a characteristic becomes due because of an event (machine status change, shift change, output batch change). In the user exit, the data of the characteristic that is now due and of the triggering event (DUE_EVENT) are passed. You can use the return parameter DUE=0 to prevent the setting of the due date.

11.2.1.5 UserExitCAQPAUCreate

You jump to this user exit once the inspection order has been created.

11.2.1.6 UserExitCAQAfterSampleCreate

You access this user exit once the sample and the respective measured value have been created. The data field UE:PAR includes the dialog string to create the sample, the data field UE:DAT includes the data of the current characteristic in the same way as this data has been available in the UserExitCAQSampleCreate. The data field UE:DLG includes the return string of the created sample and the return value of the created measured value in the acronym MW_SAVE_RET. If the measured value string has not been transferred to the server because of missing values or if an automatic measured value has not been created (PRUEFTYP of the characteristic <> attributive), then MW_SAVE_RET includes the value -1.

11.2.1.7 UserExitAfterCharacteristicChange

This user exit is used after each change of characteristic. All characteristic data of the characteristic from Merkmal.lst are passed to this user exit.

If specific criteria are true, the return value is RET=ACTION=DOCUMENT.

This user exit implements a customer-specific requirement

```

*****
Sub UserExitAfterCharacteristicChange
'-----
  dim sTempLfdNr, rc, Fehlergewichtung, bTempDoklisteZeigen
  dim sPfadOderUrl, erg, bTempAlleDokGeoeffnet
'-----
  'Init
  bTempDoklisteZeigen=False
  bTempAlleDokGeoeffnet=False
  Fehlergewichtung=VVar("UE:PAR", "ERRGEW")
  ' collect infos of rejected characteristics
  rc = GSrce("LOAD", Item("FILE", DIR_SPOOL + VVar("UE:PAR", "MWDLG:PAUDIR") + "\MerkDoku.lst"))
  If rc = "0" Then
    rc = GSrce("FIRST", "")
    If rc <> "#EOF#STORE#" Then
      Do
        '1. Show document list in ctwin
        if VSrce("AFO") = VVar("UE:PAR", "AFO") then
          if bTempDoklisteZeigen=False then
            '1a.
            if Instr(1,VVar("UE:PAR", "CMMNR"), "O_IN") = 1 then
              bTempDoklisteZeigen=True
              exit do
            end if
            '1b.
            if (Fehlergewichtung = "SICHER") AND (VSrce("DOKNR") => 70) then
              bTempDoklisteZeigen=True
              exit do
            elseif (Fehlergewichtung = "KRIT") AND (VSrce("DOKNR") => 80) then
              bTempDoklisteZeigen=True
              exit do
            elseif (Fehlergewichtung = "HAUPT") AND (VSrce("DOKNR") => 90) then
              bTempDoklisteZeigen=True
              exit do
            end if
          end if
        end if
        rc = GSrce("NEXT", "")
      Loop Until rc = "#EOF#STORE#"
      If bTempDoklisteZeigen=True then

```

```

    UE_RET = ""
    UE_RET = Item("RET", "ACTION=DOCUMENT")
End if
End If
End If
rc = GSrce("CLOSE" , "SAVE=FALSE")
'Open documents
rc = GSrce("LOAD" ,Item("FILE", DIR_SPOOL + VVar("UE:PAR", "MWDLG:PAUDIR") + "\MerkDoku.lst"))
If rc = "0" Then
    rc = GSrce("FIRST" , "" )
    If rc <> "#EOF#STORE#" Then
        Do
            '2. Open documents
            if (VSrce("AFO") = VVar("UE:PAR", "AFO")) OR (VSrce("AFO") = "") then
                ' Init
                bTempAlleDokGeoeffnet=False
                '2a. open all documents
                if Instr(1,VVar("UE:PAR", "CMMNR"), "O_IN") = 1 then
                    ' Preparing
                    if VSrce ("DOKTYP") = "TEXT" then
                        sTempText=""
                        sTempText= GibText
                        (VSrce("TEXT:1")+VSrce("TEXT:2")+VSrce("TEXT:3")+VSrce("TEXT:4")+VSrce("TEXT:5")+VSrce("TEXT:6")+
                        VSrce("TEXT:7")+VSrce("TEXT:8")+VSrce("TEXT:9")+VSrce("TEXT:10"))
                        if sTempText <> "" then
                            sTempLfdNr=StrFmtRight(IncStrDec(sTempLfdNr), 5, "0")
                            sPfadOderUrl=DIR_SPOOL + VVar("UE:PAR", "MWDLG:PAUDIR") + "CAQ_tmptxt" +
sTempLfdNr + ".txt"
                            'If file is available, delete first
                            if scrFileExists(sPfadOderUrl)="0" then scrFileDelete(sPfadOderUrl)
                            'physically create TEXT file
                            scrWriteDataIntoFile sTempText, sPfadOderUrl
                        end if
                    else
                        sPfadOderUrl=VSrce("DOKURL")
                    end if
                    'Open
                    TimerInterval="3500"
                    erg = scrAppExec (DIR_APP + "hyDocStarter.exe " + chr(34) + sPfadOderUrl + chr(34) +
" " + TimerInterval, "")
                    if erg = "#NOT#" then
                        scrMsgBox("hyDocStarter.exe konnte nicht gefunden werden")
                        exit do
                    end if
                    bTempAlleDokGeoeffnet=True
                end if
                '2b Documents according to failure weighting
                if bTempAlleDokGeoeffnet=False then
                    if (Fehlergewichtung = "SICHER") OR (Fehlergewichtung = "KRIT") OR (Fehlergewichtung
= "HAUPT") then
                        if VSrce("DOKNR") => 70 then
                            ' Preparing
                            if VSrce ("DOKTYP") = "TEXT" then
                                sTempText=""
                                sTempText= GibText
                                (VSrce("TEXT:1")+VSrce("TEXT:2")+VSrce("TEXT:3")+VSrce("TEXT:4")+VSrce("TEXT:5")+VSrce("TEXT:6")+
                                VSrce("TEXT:7")+VSrce("TEXT:8")+VSrce("TEXT:9")+VSrce("TEXT:10"))
                                if sTempText <> "" then
                                    sTempLfdNr=StrFmtRight(IncStrDec(sTempLfdNr), 5, "0")
                                    sPfadOderUrl=DIR_SPOOL + VVar("UE:PAR", "MWDLG:PAUDIR") + "CAQ_tmptxt" +
sTempLfdNr + ".txt"
                                    'If file is available, first delete
                                    if scrFileExists(sPfadOderUrl)="0" then scrFileDelete(sPfadOderUrl)
                                    'create TEXT file physically
                                    scrWriteDataIntoFile sTempText, sPfadOderUrl
                                end if
                            else
                                sPfadOderUrl=VSrce("DOKURL")
                            end if
                            ' Open
                            TimerInterval="3500"
                            erg = scrAppExec (DIR_APP + "hyDocStarter.exe " + chr(34) + sPfadOderUrl +
chr(34) + " " + TimerInterval, "")
                            if erg = "#NOT#" then
                                scrMsgBox("hyDocStarter.exe konnte nicht gefunden werden")

```

```

        exit do
    end if
end if
end if
end if
end if
rc = GSrce( "NEXT" , "" )
Loop Until rc = "#EOF#STORE#"
End If
End If
rc = GSrce("CLOSE" , "SAVE=FALSE")
'*****
End Sub
'*****

```

11.2.1.8 UserExitCAQInitialValueNumberField

This user exit implements a customer-specific requirement.

It is used to (pre-)populate the number field in the CAQ recording of measured values.

How it works

If the flag is set (hytnrcfg.ini → InitialValueNumberField=ON) and if the user exit UserExitCAQInitialValueNumberField is available, then RET=0|INITIAL_VALUE_NUMBER_FIELD=CNR is returned.

In case of **missing** or set flag (hytnrcfg.ini → InitialValueNumberField=**OFF**) and if the user exit UserExitCAQInitialValueNumberField is available, then

INITIAL_VALUE_NUMBER_FIELD=<default suggestion for number field> is returned.

```

'*****
Sub UserExitCAQInitialValueNumberField
'-----
    dim sTempCNR, sKzInitialValueNumberField
'Read Ini-File HyTnrCfg.ini - because of template customer
sKzInitialValueNumberField=""
sKzInitialValueNumberField = rsCfg("CAQ->Optionen", "InitialValueNumberField", "")
if UCASE(sKzInitialValueNumberField) <> "ON" then
    sKzInitialValueNumberField = "OFF"
else
    sKzInitialValueNumberField = "ON"
end if

'Batch number (=output batch number, ACRONYM CNR)
'      (=potential value of the suggested number field (initial value number Field))
UE_RET = ""
if sKzInitialValueNumberField="ON" then
    'Customer uses func.
    'Batch number is used to prepopulate the number field
    sTempCNR = VVar("UE:PAR", "CNR")
    'to differentiate if standard or custom
    UE_RET = Item("RET", "0")
else
    'Customer does NOT use func.
    'Standard preassignment of number field
    sTempCNR = VVar("UE:PAR", "VORSCHLAG_NUMMERNFELD")
end if
UE_RET = Item("INITIAL_VALUE_NUMBER_FIELD", sTempCNR)
'-----
End Sub
'*****

```

11.2.1.9 UserExitCAQBeforeSearchInspOrder

This user exit implements a customer-specific requirement.

You use this user exit to reassign a value to the parameter CPAU.ATKIDX

How it works

In this specific case, CPAU.ATKIDX is filled with the so-called test standard.

The user exit identifies and assigns the value for the test standard.

Call parameters

- String to search for an inspection order
- ANR = Order number
- Machine data (current order)
- Order data (current order)

Return

The user exit returns the modified string for the search of an inspection order.

```

'*****
'*** User exit before starting the search for inspection orders
'*****
Sub UserExitCAQBeforeSearchInspOrder
'*****
    Dim newValue
    ' --- Look for test standard (Prüf-Norm) in FU:13
    newValue = GetTestStandard(VVar("UE:PAR","ANR"))
    ' --- For testing
    ' scrMsgBox(newValue)
    ' --- Return RET with new Value for "CPAU.ATKIDX"
    UE_RET = Item("CPAU.ATKIDX", newValue)
'*****
End Sub
'*****

'*****
Function GetTestStandard(sANR)
'-----
Dim sFU_Default, sFU_Low, sFU_High, sValDefault, sValLow, sValHigh
Dim rValDefault, rValLow, rValHigh, sANR_Info, sFU_13, sValFU_13
'-----
    ' --- Init
    sFU_13 = "ANR_FU_13"
    ' --- Look for operation in list of running operations
    sANR_Info = scrQuickSearch(DIR_SPOOL+"anr.lst", "ANR="+sANR)
    ' --- Look for default value
    sValDefault = scrDDItem(sFU_13, sANR_Info)
    ' --- No entry found -> manually load list for op <sANR>
    If((sANR_Info = "") or (sValDefault = "")) and (SYS_OFFLINE <> "TRUE")) Then
        ' --- Read info for op
        ' --- Init
        LSTVARS = ""
        LSTVARS = "LST.FILE=" + "s_anr.lst"
        LSTVARS = "LST.CMD=" + "DLG=LIST;11|MOD=A|ANR="+sANR+"|AKRO="+sFU_13+"| "
        LSTVARS = "LST.FILTER=" + "ANR="+VDlg("ANR")
        ' --- Call list
        scrFktList
        ' --- Clear variable
        LSTVARS = ""
        ' --- Read orderline
        sANR_Info = scrQuickSearch(DIR_SPOOL+"s_anr.lst", "ANR="+sANR)
    End If
    ' --- Look for userfields
    sValFU_13 = scrDDItem(sFU_13, sANR_Info)
    ' --- Return appraised value
    GetTestStandard = sValFU_13
'-----
'
'    SYS_SCRIPT_DEBUG

```


End Function

11.2.1.10 UserExitCAQMNRCheckStatusChange

This user exit is used after each change of the machine inspection status.

When the user exit is called, the current machine inspection status and the previous machine inspection status are passed to the user exit.

Sub UserExitCAQMnrCheckStatusChange

```

dim MNRCheckStatusNeu, MNRCheckStatusAlt, aMNR
dim aChannel, aValueDischargeParts, aValueStopMachine
aMNR = VVar("UE:MNR", "MNR")
MNRCheckStatusNeu = VVar("UE:PAR", "MNR_CHECKSTATUS_NEW")
MNRCheckStatusAlt = VVar("UE:PAR", "MNR_CHECKSTATUS_OLD")
' Channel for OPC
aChannel = rsCfg("CAQ->Optionen", "OPC_CHANNEL", "")
' Value for discharging parts
aValueDischargeParts = rsCfg("CAQ->Optionen", "DISCHARGE_PARTS", "")
' Value for stopping machine
aValueStopMachine = rsCfg("CAQ->Optionen", "STOP_MACHINE", "")
if aChannel <> "" then
    ' did the checkstatus change?
    if MNRCheckStatusNeu <> MNRCheckStatusAlt then
        ' changing of checkstatus from "checked" to "due"
        if (MNRCheckStatusAlt = "W") AND (MNRCheckStatusNeu = "G") then
            if aValueDischargeParts <> "" then
                scrPCCVALUES("DLG=SETVAL|" + aChannel + "=" + aValueDischargeParts + "|")
            end if
        ' changing to "overdue"
        elseif (MNRCheckStatusNeu = "R") then
            if aValueStopMachine <> "" then
                scrPCCVALUES("DLG=SETVAL|" + aChannel + "=" + aValueStopMachine + "|")
            end if
        else
            end if
        end if
    end if
end if
End Sub

```

End Sub

11.2.1.11 UserExitCAQOnSearchingInspOrder

This user exit is used before the second run of CPAU.SUCHEN.

This way, you can attach further data to CPAU.SUCHEN. For example, you can attach a new acronym or assign another value to an existing acronym.

*** Userexit before starting the search for inspection orders (before searching ')

*** for inspection order)

Sub UserExitCAQOnSearchingInspOrder

```

'*****
Dim IstKomponentenListenAuftrag
IstKomponentenListenAuftrag=VVar("UE:PAR", "KOMPLST_AUFT")
' --- Return RET with new Value for "CPAU.MOD"
If IstKomponentenListenAuftrag = "TRUE" then
    UE_RET = Item("CPAU.MOD", "NOPANCREATE")
End If
'*****
End Sub
'*****

```

11.2.1.12 UserExitSwitchTopButton

The top buttons on the CTWIN have different functions. Which functions are assigned to these buttons depends on the modules and licenses that are available on the system. These functions are included in the main program of the terminal.

This user exit is called before and after execution of the internal functions. The user exit is also called if you change from PZE to BDE. According to the type of call, a respective mode is passed that can be requested using the function <<VVar("UE:PAR","MODE")>>:

BEFORE	Request before execution of the function
AFTER	Request after execution of the function
PZE->BDE	With change from PZE to BDE

Further parameters:

VVar("UE:PAR","BTN:FKT")	Function of the top button
VVar("UE:PAR","BTN:CAPTION ")	Button label text

Example:

After changing from the PZE dialog to the BDE, the lists (machines, orders, persons) are reloaded:

```

'-----
Sub UserExitSwitchTopButton
    Dim sMode,rc
    sMode = VVar("UE:PAR", "MODE")
    If sMode = "PZE->BDE" Then
        ' Reloading of BDE lists
        rc=scrExecute("RequestReload", "MNR, ANR, PNR")
    End If
End Sub
'-----

```

11.2.1.13 UserExitCAQAfterInvalidatingMeasurement

You run this user exit if a measured value in the CAQ recording of measured values has been set to invalid.

11.2.1.14 UserExitBeforeViewSelectedInfo

This user exit is called before an external viewer is opened (e.g. to display documents).

11.2.1.15 UserExitCAQNotifyDDSend

Notification interface for CAQ.

11.2.1.16 UserExitCAQOnReloadLists

Is used in QM-IDI.

The user exit is run if inspection order data is reloaded.

11.2.1.17 UserExitAGInfoGetCaption

Is used to fill the customer-specific OP info. If you want to change the layout of the machine overview, the terminal developer must extend the file "Controls.lst" using the tool "dlgsaver.exe".

Parameter:

(1) Fields of the OP info

```
UE:PAR:'DecimalPlaces=2|ShowMode=ShowSheet|
FIELD000=afSZY2,00:00 ,SZY2,dtMaschine|*SZY2=FIELD000|
FIELD001=afSZY1,14.40,SZY1,dtMaschine|*SZY1=FIELD001|
FIELD002=afTLG,5,TLG,dtMaschine|*TLG=FIELD002|
FIELD003=afMGRP,GS,MGRP,dtMaschine|*MGRP=FIELD003|
FIELD004=afMBEZK,M4,MBEZK,dtMaschine|*MBEZK=FIELD004|
FIELD005=afMNR,M4,MNR,dtMaschine|*MNR=FIELD005|.....
...FIELD021=afANRBEARBZEI,160:00,ANR_BEARBZEI,dtAuftrag|*ANR_BEARBZEI=FIELD021|'
```

(2) Active machine row:

```
UE:MNR=MNR=2600|MGRP=2600|MBEZK=VS 26|MBEZL=Vliesasse VS 26|... |FU:8=826|PDV=NR|
```

(3) Active order row (if an order is running):

```
UE:ANR=ROW.IDX=1|ANR=403216010100|AUNR=40321601|AGNR=0100|..|AUNR_KDAUPOS=10|
```

Result:

```
UE_RET='FIELD020=00:02|FIELD001=4.2|FIELD012=100|FIELD014=5|FIELD013=KT|FIELD015=KT|FI
ELD016=5.66|FIELD017=%|'
```

In "UE:PAR", all fields of the OP info are passed.

Syntax: <header>|<field description1>|<field description2>...

<Header>: DecimalPlaces=<AnzNK>|ShowMode=<SMode>
 <AnzNK>: configured number of decimal places (ctwinlay.ini→[main]→nachkommastellen)
 <SMode>: "ShowSheet": Call with data from machine and order list
 "UpdateSheet": Call to update with MDE data

<Field description>:
 FIELD<nnn>=<Type>,<progValue>,<Acronym>,<Source>|*<Acronym>=FIELD<nnn>

<Type>: Classification of the field type
 <progValue>: Value that was identified by the terminal program
 <Acronym>: Field name
 <Source>: Specification from the field configuration if data from machine or order are expected.

You must only specify the fields in the return string that you want to overwrite.

As an acronym can exist several times, the fields are numbered in consecutive order. You can directly call fields with a unique identification using the added acronym assignment. Fields with an acronym that exists several times, must be processed by searching all fields one after the other.

11.2.1.18 Example 1: Replace field calculation

```

Sub UserExitAGInfoGetCaption
  UE_RET=""
  Idx=1
  s=VVar("UE:PAR","DecimalPlaces")
  If s<>"" Then
    iDecimalPlaces=CInt(s)
  Else
    iDecimalPlaces=2
  End If
  sShowMode=VVar("UE:PAR","ShowMode") ' UpdateSheet/ShowSheet
  ' ***** Remaining run time *****
  sIdent=VVar("UE:PAR","*ANR:RLZ")
  sEntry=VVar("UE:PAR",sIdent)
  If sEntry<>"" Then
    s=VVar("UE:ANR","ANR_RLZ") ' sec
    If s<>"" Then
      sNewValue=scrFormatDuration(s)
      UE_RET=Item(sIdent,sNewValue)
    End If
  End If
  ' ***** Target cycle *****
  sIdent=VVar("UE:PAR","*SZY1")
  sEntry=VVar("UE:PAR",sIdent)
  If sEntry<>"" Then
    If sShowMode="UpdateSheet" Then
      s=VVar("UE:MNR","MDESZY") ' Target cycle from MDE
    Else
      s=VVar("UE:MNR","SZY") ' Target cycle from MDE
    End If
    r=scrStr2Real(s) ' sec/1000St
    If r<>0 Then
      r=60000/r ' cycles/min
      sNewValue=RealToStrNK(r,1)
      UE_RET=Item(sIdent,sNewValue)
    End If
  End If
  ' ***** Actual cycle *****
  sIdent=VVar("UE:PAR","*SZY2")
  sEntry=VVar("UE:PAR",sIdent)
  If sEntry<>"" Then
    s=VVar("UE:MNR","MDEIstZyklus")
    If s<>"" Then

```

```

r=scrStr2Real(s)                ' sec/hour
If r<>0 then
    r=60/r                      ' cycles/min
End If
sNewValue=RealToStrNK(r,1)
UE_RET=Item(sIdent,sNewValue)
End If
End If
' only fill the following fields from order/machine list!
If sShowMode="ShowSheet" Then
    ' ***** Batch size [KT] *****
    sIdent=VVar("UE:PAR","*SGR:GUTS")
    sEntry=VVar("UE:PAR",sIdent)
    If sEntry<>"" Then
        sGUTS=VVar("UE:ANR","SGR:GUTS")
        sNewValue=RealToStrNK(scrStr2Real(sGUTS),0)
        UE_RET=Item(sIdent,sNewValue)
    End If
    ' ***** Unit *****
    s=VVar("UE:ANR","AGE:S")
    If s<>"" Then
        sIdent=VVar("UE:PAR","*AGE:S1")
        sEntry=VVar("UE:PAR",sIdent)
        If sEntry<>"" Then
            sNewValue=s
            UE_RET=Item(sIdent,sNewValue)
        End If
        sIdent=VVar("UE:PAR","*AGE:S2")
        sEntry=VVar("UE:PAR",sIdent)
        If sEntry<>"" Then
            sNewValue=s
            UE_RET=Item(sIdent,sNewValue)
        End If
    End If
End If
' ***** unit [%] for scrap *****
sIdent=VVar("UE:PAR","*AGE1")
sNewValue="%"
UE_RET=Item(sIdent,sNewValue)
' scrMsgBox(sIdent+"="+sEntry+"-->"+sNewValue)
' SYS_SCRIPT_DEBUG
End Sub ' UserExitAGInfoGetCaption

```

11.2.1.19 Example 2: Change caption and colors

Example:

- optimized access with "*ANR"
- Hide/show fields + change background/font color/caption

```

Sub UserExitAGInfoGetCaption
' scrLog( " UserExitAGInfoGetCaption = " + SYS_DT )
x = VVar("UE:PAR","*RLZ")
ag = VVar("UE:ANR","AGNR")
If x <> "" And ag <> "" Then
    rlz = VVar("UE:ANR","RLZ")
    If rlz <> "" Then
        r1 = scrGetPart(rlz,";",1)
        r2 = scrGetPart(rlz,";",2)
    End If
End If

```

```

r3 = scrGetPart(rlz,";", "3")
If vbsValidDate(r2) And vbsValidTime(r3) Then
    sf = Int(VZei("0"))-Int(r3)
    df = Int(now*86400) - (Int(vbsDateTime(r2,"0")*86400) + Int(r3))
    xf = Int(Round((now - vbsDateTime(r2,r3))*86400))
    r0 = CStr(Int(r1)-xf)
Else
    r0 = r1
End If
' *** Formatting of RRT with STD (Hrs.) script function in format
"HHH:MM"
    UE_RET=Item(x,scrFormatDuration(r0))
    ' UE_RET=Item(VVar("UE:PAR","*ATK"),Vdat("0"))
    UE_RET=Item(VVar("UE:PAR","*ATK"),scrFormatDuration(r1)+",
"+CDurationToTimeStr(r1,"HMS"))
    UE_RET=Item(VVar("UE:PAR","*ATKBEZ"),scrFormatDuration(r0)+",
"+CDurationToTimeStr(r0,"+HMS"))
    ' *** "" + LABEL + "=" + bkColor + ";" + FontColor + ";" + Caption ***
    If scrStr2Real(r1) < 0.0 Then
        UE_RET=Item("*"+"RLZ_A","clRed;clWhite")
    Else
        UE_RET=Item("*"+"RLZ_A","clBlue;clYellow")
    End If
    UE_RET=Item("#CTRL:SHOW","_RLZ;RLZ_A")
Else
    UE_RET=Item(x,"---")
    UE_RET=Item("#CTRL:HIDE","_RLZ;RLZ_A")
End If
Else
    x = VVar("UE:PAR","*MBEZK")
    If x <> "" Then
        ' *** #CTRL:SHOW / #CTRL:HIDE = LABEL + ";" + LABEL + ";" + .. ***
        UE_RET=Item("#CTRL:SHOW","MNR_M;_MGRP")
        If VVar("UE:MNR","MNR") = "M505" Then
            ' *** "" + LABEL + "=" + bkColor + ";" + FontColor + ";" + Caption ***
            UE_RET=Item("*"+"MNR_M","clWhite;clBlue")
        End If
    End If
    UE_RET=Item("*"+"_MGRP","clWhite;clBlue;"+scrTranslate("Gruppe","")+"/"+scrTr
anslate("Takte",""))
    Else
        UE_RET=Item("*"+"MNR_M","clWhite;clBlack")
        UE_RET=Item("*"+"_MGRP","clBtnFace;clBlack;Gruppe")
    End If
End If
'SYS_SCRIPT_DEBUG
End Sub 'UserExitAGInfoGetCaption

```

Arbeitsplatz/Maschine	M506	Arbeitsplatz/Maschine	M505
Bezeichnung	M506	Bezeichnung	M505
Gruppe	063502	Gruppe/Takte	063502
Teiligkeit	32	Teiligkeit	4
Zykluszeit [Sek/Takt]	17	Zykluszeit [Sek/Takt]	14
Zykluszeit [Std:Min/Takt]	00:00	Zykluszeit [Std:Min/Takt]	00:00
Arbeitsgang	0000001319750060	Arbeitsgang	AST-TST-00010100
Artikel	0616725	Artikel	- 03:51 , -03:51:08
Artikelbez.	LANZETTE PRO II VK:12	Artikelbez.	- 03:59 , - 03:59:29
Bem. 1		Bem. 1	
Bem. 2		Bem. 2	
Plandauer [Std:Min]	694:26	Plandauer [Std:Min]	01:18
		Restlaufdauer [Std:Min]	- 03:59

11.2.1.20 Example 3: Planned duration in [days:hrs]:

```

...
Dim s,s1,sIdent,i,i1,i2
s=VVar("UE:ANR","ANR_BEARBZ")
If s<>" " Then
    i=vbsIntDef(s,"0")
    i1=i \ 86400 ' full days
    i2=i \ 3600 ' full hours
    i2=i2 mod 24 ' remaining hours
    If i2<10 Then s1="0"+CStr(i2) Else s1=CStr(i2)
    s=CStr(i1)+":"+s1
    sIdent=VVar("UE:PAR","*ANR_BEARBZEI")
    UE_RET=Item(sIdent,s)
    UE_RET = Item("#CTRL:SHOW","_MNR;_SGR_KLKLZ")
    UE_RET=Item("*_SGR_KLKLZ",";"+scrTranslate("Plandauer [Tage:Std]",""))
End If
...

```

Bem. 2	
Plandauer [Tage:Std]	12:10

11.2.1.21 Example 4: Operation name in field "Bem 1"

```

'-----
Sub UserExitAGInfoGetCaption
    UE_RET=""
    s=VVar("UE:PAR","DecimalPlaces")
    ' ***** BEM1 -> AGBEZ *****
    ' Change caption
    ' Note: IDs from ctwin\etc\controls.lst
    UE_RET=Item("*_BEM1","clBtnFace;clBlack;AG-Bezeich.")
    UE_RET=Item("#CTRL:SHOW","_Bem1;AG-Bezeichnung")
    ' Enter value
    sIdent=VVar("UE:PAR","*BEM1")

```

```

sEntry=VVar("UE:PAR",sIdent)
If sEntry<>" " Then
    s=VVar("UE:ANR","AGBEZ")
    If s<>" " Then
        sNewValue=s
        UE_RET=Item(sIdent,sNewValue)
    End If
End If
End Sub
'-----

```

11.2.2 CTWIN + AIP

This section describes user exits that you can use in the CTWIN and in the AIP.

11.2.2.1 UserExitInitLosnummer

You use this user exit to configure the length of the standard batch number.

```

'*****
Sub UserExitInitLosnummer
'-----
'-- if the value <LEN:CNR> is not set the
'-- DB-Setup-batch number length remains unchanged
'-----
    UE_RET = Item("LEN:CNR", "10")
End Sub
'*****

```

User exit background:

- The customer-specific batch number should have 20 digits and is used for order postings.
- For goods receipt batches, pallets, etc., a "Hydra standard batch number" with 10 digits is to be generated.
- To correctly show both batch numbers in all console evaluations, you must configure the batch number length in the DB setup entry with <20>. Configure the "Hydra standard batch number" length in the user exit <UserExitInitLosnummer> on the terminal and set the required length.

11.2.2.2 UserExitLosnummer

This user exit implements a customer-specific batch number. Each time the terminal automatically generates a batch number, this user exit is run.

<u>Available functions</u>	<u>Description</u>
VTnr("XYZ")	Info from the list TKENN.LST
VVar("UE:PAR","XYZ")	Transfer parameters
VVar("UE:MNR","XYZ")	Info from list MNR.LST for the currently selected machine
VVar("UE:ANR","XYZ")	Info from list ANR.LST for the currently selected order
rsIni(ini,Sektion,Key,Default)	Read INI file (if the entry in the INI file is not available, it is created using the specified <default>).
wsIni(ini,Sektion,Key,Value)	Write INI file


```

*****
Sub UserExitLosnummer
*****
    dim lfd, cnr
    -----
    Select Case VVar("UE:PAR","MODE")
    -----
        Case "GENERATE"
            -----
            '-- Modi = "CNR.FIX.TYP" = "cmWarenEingang", "cmFertigung"
            '-- if no "CNR=NNNNNNNNN" in <UE_RET> is defined
            '-- the standard batch number generation is used
            -----
            If VVar("UE:PAR","CNR.FIX.TYP") = "cmWarenEingang" Then
                UE_RET = Item("RET", "DEFAULT")
            Else
                lfd = rsIni( "60704.ini", "Losnummer", VTnr("TNR")+>LFD", "0" )
                lfd = IncStrDec( lfd )
                lfd = wsIni( "60704.ini", "Losnummer", VTnr("TNR")+>LFD", lfd )
                lfd = wsIni( "60704.ini", "Losnummer", "CNR->UNDO->TNR->LFD", lfd )
                cnr = "2"+VTnr("TNR") + StrFmtRight( lfd, 5, "0" )
                cnr = wsIni( "60704.ini", "Losnummer", "CNR->UNDO->TNR->CNR", cnr )
                UE_RET = Item("CNR", cnr )
                UE_RET = Item("RET", "0" )
            End If
        Case "UNDO"
            -----
            '-- if a "RET=0" in <UE_RET> is defined
            '-- the standard batch number undo function is skipped
            -----
            UE_RET = Item("RET", "0")
            -----
    End Select
*****
End Sub
*****

```

User exit background:

- Here, you can implement a customer-specific batch number generation.

11.2.2.3 UserExitScriptInfos

You use this user exit to create customer-specific evaluations in the <OP info> on the terminal.

Available functions	Description
VTnr("XYZ")	Info from the list TKENN.LST
VVar("UE:PAR","XYZ")	Transfer parameters
VVar("UE:MNR","XYZ")	MNR info of the machine displayed in the OP info
VVar("UE:ANR","XYZ")	ANR info of the order/operation displayed in the OP info
scrUECmd(...)	Execution of a DB command with a file as result

```

*****
Sub UserExitScriptInfos
*****
    If VVar("UE:PAR","MODE") = "SCRIPT-INFO->INIT" Then
        -----
        '-- Begin -> ScrInfo - Initialization --- NOTE: !!! can only be changed on start of program
        '!!!
        -----
        '-- ScrInfo[01..10] = ScriptID | TS-Header [ | TS-Variante | TS-Icon | TS-IniSection ]
        '-----
    End If

```

```

' --- ScriptID      = ID für UserExit <UserExitScriptInfos>
' --- TS-Header     = Tabsheet-Header      = Title of Tabsheet in OP info
' --- TS-Variante   = Tabsheet variant    = scrtsREPORT[DEFAULT], scrtsGRID,
' ---              =                      scrtsTEXT, scrtsLISTE
' --- TS-Icon       = Tabsheet-Icon       = rbREPORT, rbGRID, rbLISTE, rbTEXT
' --- TS-IniSection = Tabsheet-IniSection = IniSection for TS variant(scrtsGRID)
' -----
UE_RET = Item("RET", "0")
UE_RET = Item("ScrInfo01", "AGI-Rprt (01); " )
UE_RET = Item("ScrInfo02", "AGI-Grid (02);scrtsGRID;rbGRID;Auftrags-Fortschritt " )
' -----
' --- End -> ScrInfo - Initialization ---
' -----
Else
' -----
' --- Set file name -> <ScrInfo[01..10].lst> -----
UE_RET = Item("FILE", VVar("UE:PAR", "SCRIPT.ID") + ".2"+VTnr("TNR") )
' -----
Select Case VVar("UE:PAR", "SCRIPT.ID")
' -----
Case "ScrInfo01"
' --- Initialization of the dialog to be called -----
UE_RET = Item("MNR", VVar("UE:MNR", "MNR") )
UE_RET = Item("AUNR", VVar("UE:ANR", "AUNR") )
' -----
' --- If no dialog is required, you must set RET for further processing
' --- UE RET = Item("RET", "0") ' without dialog
' -----
' --- Parameter 1 : "u_tnr002" = Dynamic dialog
' --- Parameter 2 : "0" or "*" = Return code if dynamic dialog is not available
' ---              -> "0" processing also continues without dialog
' ---              -> "*" Error message no dialog available
' --- Parameter 3 : <UE RET> = Dialog initialization "FILE=...|MNR=...|AUNR=.."
' -----
UE_RET = scrExecDynDlg( "u_tnr002" , "0", UE_RET ) ' with dialog "0"
' -----
If VVar("UE:RET", "RET") = "0" Then
UE_RET = Item("DLG", "SYSTEM.CALL" )
UE_RET = Item("PROG", "hytnrrpv.scr" )
UE_RET = Item("HYDSCR", "u_tnr002" )
UE_RET = Item("DATEI", ".\spool/"+VVar("UE:RET", "FILE") )
UE_RET = Item("CMD:LST", "DELETE" )
UE_RET = Item("ANR.AUNR", VVar("UE:RET", "AUNR") )
UE_RET = Item("CMD:CPY", "BINARY" )
scrUECmd( UE_RET )
End If
' -----
Case "ScrInfo02"
' --- Sample implementation of the standard OP info <OP progress>-----
UE_SND = Item("DLG", "LIST;11" )
UE_SND = Item("MOD", "F" )
UE_SND = Item("AUNR", VVar("UE:ANR", "AUNR") )
UE_SND = Item("FILE", VVar("UE:RET", "FILE") )
UE_SND = Item("CMD:LST", "CLEAR" )
scrUECmd( UE_SND )
' -----
End Select
' -----
End If
' -----
'
' SYS SCRIPT DEBUG
' *****
End Sub
' *****

```

User exit background:

- Display of customer-specific Quick Reports that have been implemented in the console using the license "HYD-RPVTNR".

11.2.2.4 UserExitDynDlgBarcode

This user exit is run if the terminal program includes a bar code.

How the user exit is used:

- Manipulation of a bar code
- Implementation of an alternative bar code processing

An alternative bar code processing usually depends on the bar code identified and the dialog that is currently opened.

Available functions

VVar("UE:Bar","XYZ")

Description

Transfer parameter from bar code dispatcher

VVar("UE:RET","XYZ")

Return: Processed bar code

Available parameters

VVar("UE:Bar","DLG")

Dialog ID

VVar("UE:Bar "LESERTYP")

Reader type (LESERTYP=BAR,.....)

VVar("UE:Bar "COM")

Comport of reader

VVar("UE:Bar"," BAR.DLGID")

Input field ID (?? if cannot be assigned from dispatcher)

VVar("UE:Bar "BAR.VALUE")

Bar code read

VVar("UE:Bar "DLG.ALL.FLD")

All dialog input field IDs separated by semicolon

VVar("UE:Bar "DLG.FLD")

Focused field (ID)

VVar("UE:Bar "BAR")

Complete string that would be sent to the dialog

Ex. CNR=xxxxxxxx or ??=xxxxxxxx

VVar("UE:BAR","BAR.RAWDATA")

Original bar code string as of ctwin 7.2.3.28

```

Function UserExitDynDlgBarcode
dim did, sts, barval, islen, dlg
-----
did = VVar("UE:BAR","BAR.DLGID")
barval = VVar("UE:BAR","BAR.VALUE")
dlg = VVar("UE:BAR","DLG")
Barlen = Len(barval)
if dlg = "A_P_AN_MPL" or dlg = "A_AN_MPL" or dlg = "CE_WL_MPL" then
    If did = "DLL" or did = "CNR" or did = "???" or Barlen >= 18 Then
        sts = VVar("UE:BAR","BAR.VALUE")
        If Barlen >= 18 Then
            If Left(sts,2) = "00" Then
                If did = "???" Then

```

Sample return: 'result from script':

```

UE_RET = ""
UE_RET = Item( did , Right(sts,Len(sts)-2) )

```

Example:

```

did = VVar("UE:BAR","BAR.DLGID")
barval = VVar("UE:BAR","BAR.VALUE")

```

```

dlg = VVar("UE:BAR","DLG")
comport = VVar("UE:BAR","COM")
' Field where the cursor is
SelectFld= VVar("UE:BAR","DLG.FLD")
If SelectFld = "CNR:VO" Then
    UE_RET = ""
    UE_RET = Item( "CNR:VO", barval )
End If

```

Note: The original bar code is available with the following parameters

```
barcode = VVar("UE:BAR","BAR.RAWDATA")
```

11.2.2.5 UserExitLocalMnrAnrUpdate

You use this user exit to update the local machine list (MNR.LST) and order list (ANR.LST) on the terminal after a successfully performed posting on the HYDRA server.

This script is also run, if automatically recorded quantities are passed that are then displayed (→ local quantity even of the machine data collection MDE). If this mode is used, the variable sMode has the value "MENGE". See example below.

It is not recommended to use this user exit under MW2.0 if you use the parallel partitioning of the order.

→ To call this user exit, the send string must include a machine (MNR) and an order [ANR] that are included in the local order list (anr.lst).

Available functions

VVar("UE:SND","<ID>")

Description

You use this function to access the values of the DB event sent.

→ Event „DLG=M_MST|MST=2|..|“

→ VVar("UE:SND","MST") ergibt "2"

scrStoreUpdate(sMode,sID,sValue)

Function to read, write, add up values in DD lists. See section "Script functions".

```

' *****
Sub UserExitLocalMnrAnrUpdate
    dim sDlgID,sMode,rc
' -----
    sMode    = VVar("UE:PAR","MODE")
    sDlgID   = VVar("UE:SND","DLG.DLGCFG")
    If sDlgID = "" Then
        sDlgID = VVar("UE:SND","DLG")
    End If
' ----- P R O C E S S I N G - N O T E S -----
' ### Case "MNR->UPDATE" ###
' --- - If the event includes an "MNR", the <case> is performed for the <sDlgID>
' ### Case "ANR->UPDATE->RUNNING" ###
' --- - If the event includes an "ANR" and an "MNR", the <Case> for the <sDlgID>
' --- - is performed; this <Case> is only performed if the "ANR" is running at the "MNR"
' --- - <AST_OPT_PKENN=L>
' --- - Example: Update of e.g. "Batch number" that is in relation to the "MNR"
' ### Case "ANR->UPDATE" ###
' --- - If the event includes an "ANR", the <case> is performed for the <sDlgID>
' --- - this <Case> is run after "ANR->UPDATE->LAUFEND"
' --- - Example: Update of e.g. "order yield" that is globally increased
' ----- P R O C E S S I N G - N O T E S -----

```

```

Select Case sDlgID
'-----
Case "CA WL RF"
    Select Case sMode
        Case "MNR->UPDATE"
            scrLog( " UserExitLocalMnrAnrUpdate ( "+sMode+" )" )
        Case "ANR->UPDATE->LAUFEND"
            scrLog( " UserExitLocalMnrAnrUpdate ( "+sMode+" )" )
            If VVar("UE:SND","CNR") <> "" Then
                rc = scrStoreUpdate( "UPDATE","CNR",VVar("UE:SND","CNR") )
                scrLog( " UserExitLocalMnrAnrUpdate ( CNR = "+rc+" )" )
            End If
        Case "ANR->UPDATE"
            scrLog( " UserExitLocalMnrAnrUpdate ( "+sMode+" )" )
            If VVar("UE:SND","KDCNR") <> "" Then
                rc = scrStoreUpdate( "ADD","AGR_FU_11","1" )
                scrLog( " UserExitLocalMnrAnrUpdate ( AGR_FU_11 "+rc+" (+1)" ) )
            End If
        End Select
Case "A UN RF"
    Select Case sMode
        Case "MNR->UPDATE"
            scrLog( " UserExitLocalMnrAnrUpdate ( "+sMode+" )" )
        Case "ANR->UPDATE->LAUFEND"
            scrLog( " UserExitLocalMnrAnrUpdate ( "+sMode+" )" )
            rc = scrStoreUpdate( "UPDATE","AST_OPT_PKENN","U" )
        Case "ANR->UPDATE"
            scrLog( " UserExitLocalMnrAnrUpdate ( "+sMode+" )" )
            If VVar("UE:SND","KDCNR") <> "" Then
                rc = scrStoreUpdate( "ADD","AGR_FU_11","1" )
            End If
        End Select
Case "A AB RF"
    Select Case sMode
        Case "MNR->UPDATE"
            scrLog( " UserExitLocalMnrAnrUpdate ( "+sMode+" )" )
        Case "ANR->UPDATE->LAUFEND"
            scrLog( " UserExitLocalMnrAnrUpdate ( "+sMode+" )" )
            rc = scrStoreUpdate( "UPDATE","AST_OPT_PKENN","E" )
        Case "ANR->UPDATE"
            scrLog( " UserExitLocalMnrAnrUpdate ( "+sMode+" )" )
            If VVar("UE:SND","KDCNR") <> "" Then
                rc = scrStoreUpdate( "ADD","AGR_FU_11","1" )
            End If
        End Select
    End Select
    scrLog( " UserExitLocalMnrAnrUpdate [" +sDlgID+"/" +sMode+"] == " +
VVar("UE:SND", "#GET#ALL#VALUES#") )
'-----
'      SYS SCRIPT DEBUG
End Sub
'*****

```

11.2.2.6 UserExitDynDlgBeforeSend (internal UE)

You use this user exit to manipulate or suppress postings for the HYDRA server. If you open a dialog that includes a script with an own event handler <DynDlgBeforeSend_XYZ>, the general <UserExitDynDlgBeforeSend> is not called.

Available functions

VDlg("<ID>")

Description

Sent data of dialog

```

'*****
Sub UserExitDynDlgBeforeSend
    dim sDlgID, sScrID

```

```

' -----
sDlgID = VDlg("DLG")
sScrID = VDlg("SCRIPT.ID")
' scrMsgBox ( "Functions -> DynDlgBeforeSend -> [ "+ sDlgID +" / "+ sScrID + " ]")
' -----
Select Case sDlgID
' -----
Case "M_T_Z"
    DLGSND = Item("EVENT", "EVENT_ONLINE_OHNE_AUTO_MENGEN" )
    DLGSND = Item("MNRTNR.TNR", VDlg("TNR") )
    DLGSND = Item("MNRTNR.MNR", VDlg("MNR") )
    DLGSND = Item("MNRTNR.OPT:TMP", "J" )
    If VDlg("MODUS") = "Z" Then
        ' -- create new dynamic terminal machine assignment -----
        ' -- DLG=MNRTNR.INSERT|MNR=xxxx|TNR=xxx|KNR=xxxx|OPT:TMP=J -----
        DLGSND = Item("DLG", "MNRTNR.INSERT" )
    Else
        ' -- delete dynamic terminal machine assignment -----
        ' -- DLG=MNRTNR.DELETE|MNR=xxxx|TNR=xxx|KNR=xxxx -----
        DLGSND = Item("DLG", "MNRTNR.DELETE" )
    End If
' -----
Case "XYZ"
    ' *** Implementation of customer-specific actions
' -----
End Select
' -----
SYS SCRIPT DEBUG
End Sub
' *****

```

Sending a posting can be prevented with the following line in the script:

```
DLGSND=AddIt("EVENT","EVENT_DIALOG_OHNE_SENDEN","")
```

In UserExitDynDlgBeforeSend, it is possible to prevent sending and to go back to the dialog:

```
DLGSND=AddIt("EVENT","EVENT_ERROR","")
```

Via the assignment with DLGSND, you can change existing acronyms or add new acronyms. To delete existing acronyms from the send string, you must delete the string and reassign it:

asSend=VDlg("#GET#ALL#VALUES#")	' save dialog data
DLGSND="#DELETE#ALL#VALUES#"	' delete dialog data
asSend=scrEraseDDItem("CNR:AUS",asSend)	' remove acronym
DLGSND=asSend	' reassign send string

If you want to send another dialog before sending the actual dialog, the sending buffer must be deleted afterwards.

Example:

```

Sub UserExitDynDlgBeforeSend
Dim sOut,sSnd,sRcv,rc
Select Case VDlg("DLG.DLGCFG")
Case "A_UN","A_AB","A_TR"
    ' back up dialog data
    SendDat=VDlg("#GET#ALL#VALUES#")
    ' send other dialog
    sOut="DLG=M_AST|MNR="+VDlg("MNR")+ "|ANR="+VDlg("ANR")
    sOut=sOut+"|EVENT=EVENT_ONLINE_MIT_AUTO_MENGEN|"
    rc=scrDDSnDRcv(sOut,sSnd,sRcv)
    ' delete dialog data

```

```

        DLGSND="#DELETE#ALL#VALUES#"
        ' restore original dialog
        DLGSND=SendDat
    End Select
End Sub

```

11.2.2.7 UserExitDynDlgAfterSend (internal UE)

This user exit is called, once a posting has been transferred successfully to the HYDRA server.

Typical use cases:

- Local updates after a posting
- Request reloading of lists
- Perform follow-up actions for complex processes

(see also section "DynDlgAfterSend_XYZ")

```

'*****
Sub UserExitDynDlgAfterSend
'*****
dim sDlgID
'-----
    sDlgID = VSnd("DLG")
'-----
'*** for DD-LIST-Reload [ ANR,MNR,PNR,MAT,MST,RES ""=no DD-Lst-Updates] *****
'*** <RES> for resource list (as of WRM/MDE > 7.2)
'-----
    Select Case sDlgID
        '----- Ex. "Dynamic machine terminal assignment -----
        Case "M_T_Z"
            DD_RCV = Item( "LOAD", "MNR,MST,ANR,PNR,"+ VRcv("LOAD") )
        '-----
        Case "XYZ"
            ' *** Implementation of customer-specific actions
    End Select
'-----
    scrLog( "UserExitDynDlgAfterSend = "+VSnd("#GET#ALL#VALUES#") )
    SYS SCRIPT DEBUG
'*****
End Sub
'*****

```

For the local data update, the terminal functions can be used. To this end, an appropriate "send string" is created and passed as follows:

```

asUpdate="TYP=ANR,MNR|DLG=A_TR|MNR="+VSnd("MNR")+"|ANR="+VSnd("ANR")+"|EGR:GUT="+CStr(sGut)+
EGR:AUS="+CStr(sAus)
rc=scrSetData("LocalUpdate",asUpdate)

```

If you want to change the return string supplied by the server before it is further processed in the terminal, you must use the value "#UPDATE_RCV#=1" to identify this string.

Example:

```

DD_RCV=Item("RET.SGE_P","ST")
DD_RCV=Item("#UPDATE_RCV#","1")

```

Note: If a posting is refused by the HYDRA server, this user exit does not work. If you want to react, then you must use →UserExitAfterSendError.

11.2.2.8 UserExitButtonClick

This user exit is called if one of the keys below the lists in the main view of the terminal is pressed.

Typical use cases:

- Preventing execution of a dialog (e.g. specific plausibility checks)
- Replacing programmed behavior with own functions
- Implementing new functions that are not related to a dynamic dialog.

The current machine and order line are transferred to the user exit.

<u>Available functions</u>	<u>Description</u>
VVar("UE:MNR", "XYZ")	Info from list MNR.LST for the currently selected machine
VVar("UE:ANR", "XYZ")	Info from list ANR.LST for the currently selected order

Notes:

The entry "UE_RET = Item("BTN.FKT", "#FKT#->#EXIT#")" prevents the standard processing of the key.

The entry „UE_RET = Item("BTN.FKT", "A_UN")“ overwrites the configured function (in the following example = <A_AB> with <A_UN>)

The entry „UE_RET = Item("ANR+MNR", "RELOADED")“ has the effect that the data of the selected machine and the selected operation are reloaded before the dialog is called.

Using the entry „UE_RET = Item("BTN.FKT", "SCRIPT->A_UN")“, the standard implementation is avoided and the function is performed via script "mpdv-A_UN.scr" or "(customer number)-A_UN.scr". This way, you can use one dialog script for the dialogs A_UN und A_AB:

```
If VVar("UE:PAR","BTN.FKT")="A_AB" Then UE_RET=Item("BTN.FKT","SCRIPT->A_UN")
```

The entry "UE_RET=Item("BTN.RET", "1")“ prevents the message "Hydra-Fct. ... not implemented"

```
*****
Sub UserExitButtonClick
*****
dim sfkt,smnr,sanr
'-----
sfkt    = VVar("UE:PAR","BTN.FKT")
smnr    = VVar("UE:MNR","MNR")
sanr    = VVar("UE:ANR","ANR")
' scrMsgBox( "UserExitButtonClick == [ "+sfkt+" ] \n Machine = [ "+smnr+" ]\n order  = [
"+sanr+" ]\n -> "+VVar("UE:PAR","#GET#ALL#VALUES#") )
'-----
Select Case sfkt
'-----
Case "A UN"
    If VVar("UE:ANR","AGNR") = "0051" Then
        scrMsgBox( " (A_UN) bei AGNR = 0051 über Script !\n Script: [ A_UN ] ausführen." )
        UE_RET    = Item("BTN.FKT", "SCRIPT->A_UN")
        UE_RET    = Item("ANR+MNR", "RELOADED" )
    End If
'-----
Case "A AB"
    If VVar("UE:ANR","AGNR") = "0052" Then
        scrMsgBox( " (A_AB) bei AGNR = 0052 nicht möglich !\n Funktion: [ A_UN ] ausführen." )
        UE_RET    = Item("BTN.FKT", "A_UN")
    End If
'-----
Case "A TR"
    If VVar("UE:ANR","AGNR") = "0053" Then
        scrMsgBox( " (A_TR) bei AGNR = 0053 nicht möglich !\n Funktion wird abgebrochen." )
        UE_RET    = Item("BTN.FKT", "#FKT#->#EXIT#")
    End If
End Select
```



```

' -----
scrLog( "UserExitButtonClick == [ "+sFkt+" ] -> "+VVar("UE:PAR", "#GET#ALL#VALUES#") )
'
  SYS_SCRIPT_DEBUG
' *****
End Sub
' *****

```

11.2.2.9 UserExitPccDllToTerminal

You use this user exit to process PCCDLL events. Here, all data that comes from the PCC interface is run through. In combination with the output function "scrPCCValues()", you can implement complex control tasks in the terminal script.

Available functions

Description

VVar("UE:DAT", "<ID>")

Access to specific fields of the PCCDLL event

VVar("UE:DAT", "#GET#ALL#VALUES#")

Reading the complete string

```

' *****
Sub UserExitPccDllToTerminal
' -----
dim xData
' -----
  xData = VVar("UE:DAT", "#GET#ALL#VALUES#")
' scrMsgBox( "UserExitPccDllToTerminal:[ "+xData+" ]" )
  If scrPosStr("|I:I301=1|", xData) <> "" Then
    , Do something
  End If
' -----
' scrLog( "UserExitPccDllToTerminal == [ "+xData+" ] " )
'
  SYS_SCRIPT_DEBUG
End Sub
' *****

```

Example: Change values of the driver and pass them on to the terminal

UE_RET = Item("#PCCDATA-MODE#", "#NEW#") **must be set**

```

' *****
Sub UserExitPccDllToTerminal
  Dim sWaageNr, sWaageValue
' -----
' From the balance, it is always V:BRUTTO=xxxx
' only the balance number changes if the balance changes V:WAAGENR=1 or V:WAAGENR=2
' Value of balance 1 (Waage 1) writes
' reset value of balance 2 (Waage 2) to the correct input field in the dialog
' -----
  sWaageNr = VVar("UE:DAT", "V:WAAGENR")
  sWaageValue = VVar("UE:DAT", "V:BRUTTO")
  If (sWaageNr = "2") Then
    UE_RET = ""
    UE_RET = Item("DLG", "GETVAL")
    ' To take over changed data from the script
    UE_RET = Item("#PCCDATA-MODE#", "#NEW#")
    ' Balance value, if balance number=2 is set to dialog field of balance 2
    UE_RET = Item("V:WAAGENR", sWaageNr)
    UE_RET = Item("V:EGR:GUT", sWaageValue)
    ' so that the field of balance 1 is not also filled
  End If
' -----
  SYS_SCRIPT_DEBUG
End Sub
' *****

```

Example: In the system script <UserExitPccDllToTerminal> you want to reset the machine status. This status is to be additionally transferred to the MDE.
To take over the status in the MDE in case of a dialog string DLG=M_MST from the system script, you must add this parameter to the send data

```
DD_SND = Item("PROCESSDLGEVENT", "TRUE")
```

Additional functions:

```
UE_RET = Item("#PCCDATA-MODE#", "#CLEAR#")
```

→ deletes the data

```
UE_RET = Item("#PCCDATA-MODE#", "#EXIT#")
```

→ exits the distribution dunction (the data record is not passed)

11.2.2.10 UserExitUpdateEVCOM

You use this user exit to update MDE data (partitioning, target cycle, machine status and quantities) after a script dialog.

11.2.2.11 UserExitPzeCfgLoad

You use this user exit to load customer-specific files as part of the PZE configuration

Parameter: UE: PAR=TNR=826|TYP=830|CFG:1=1|HWADR=10.10.62.163|TZ=|..|PORT=|'

Result: UE:RET=RET=*>|..|CNT=< number of loaded files >|..|

```

'*****
Sub UserExitPzeCfgLoad
  dim cnt
  '-----
  cnt = "0"

  '*** load wage types
  UE_SND = ""
  UE_SND = Item("DLG", "SYSTEM.CALL" )
  UE_SND = Item("PROG", "mf_hoppe.scr" )
  UE_SND = Item("AKTION", "lohnart" )
  UE_SND = Item("DATEI", ".\spool\lohnart."+SYS_USR )
  UE_SND = Item("FILE", "lohnart.lst" )
  '----- UE_SND = Item("CMD:CPY", "BINARY" ) ' load binary if required
  scrUECmd( UE_SND )

  If VVar("UE:RCV", "RET") = "0" Then
    cnt = IncStrDec(cnt)
  End If

  '*** load cost centers
  UE_SND = ""
  UE_SND = Item("DLG", "SYSTEM.CALL" )
  UE_SND = Item("PROG", "mf_hoppe.scr" )
  UE_SND = Item("AKTION", "kostenst" )
  UE_SND = Item("DATEI", ".\spool\kostenst."+SYS_USR )
  UE_SND = Item("FILE", "kostenst.lst" )
  '----- UE_SND = Item("CMD:CPY", "BINARY" ) ' load binary if required
  scrUECmd( UE_SND )

  If VVar("UE:RCV", "RET") = "0" Then
    cnt = IncStrDec(cnt)
  End If

  UE_RET = Item("CNT", cnt)
  '-----
  '      SYS_SCRIPT_DEBUG
End Sub
'*****

```

11.2.2.12 UserExitBarcodeToMain

If a bar code is scanned when the terminal program is in the main view (no dialog is opened), the bar code is processed using UserExitBarcodeToMain.

Via the identified bar code type, a specific dialog can be opened, for example.

Bar code event / parameter:

- BAR = raw data (bar code as scanned)
- BAR.DLGID = field ID identified via bar code length (e.g. KNR)
- BAR.VALUE = bar code (can be without check digit with KNR) – value that is transferred to the field

Example:

```
UE:BAR=BAR=PR3X58G112|LESERTYP=BAR|COM=1|DLG=MAIN->FORM|
BAR.DLGID=CNR|BAR.VALUE=PR3X58G112|MNR=RW10|ANR=080006830290|
```

Return values: (possible via <FKT.ID> or <#F[1..12]>)

- (1) UE:RET=RET=#FKT#->#EXIT#|..“
 ➔ prevents standard processing in the terminal program (processing of bar code only using the script)
 ➔ using function <scrAddAction()> you can open a dialog
- (2) no return value
 ➔ the standard processing in the terminal program opens the dialog "Change status" (M_MST)

```

' *****
Sub UserExitBarcodeToMain
' -----
    did      = VVar("UE:BAR", "BAR.DLGID")
    barstr    = VVar("UE:BAR", "BAR")
    barval    = VVar("UE:BAR", "BAR.VALUE")
    ' MsgBox "UserExitBarcodeToMain = "+VVar("UE:BAR", "#GET#ALL#VALUES#")
    If did = "CNR" Or len(barstr) >= 10 Then
        rc = scrAddAction("mtaDIALOG", "DLG=C_MR_PACK|" , Item("CNR", barstr) )
        UE_RET = Item("RET", "#FKT#->#EXIT#")
    End If
' -----
' SYS_SCRIPT_DEBUG
End Sub
' *****

```

11.2.2.13 UserExitAutomaticQuantities

This UserExit is called from different places of the terminal program. Each time a specific mode is passed:

```
sMode=VVar("UE:PAR","MOD")
```

"LOCAL" → All events of the internal MDE processing are passed to the visualization layer of the terminal program (EventToLocalData).

Note: At this point, the meter quantities have not yet been evaluated with regard to "Posting during production lock". It is therefore possible that a yield recorded here, is later identified as scrap or is deleted.

"ADD" → If a data record is posted to the HYDRA server, the automatically recorded quantities of the machine that have not yet been posted are normally added to the string. This data can be modified here.

Note: The data is changed from MDE 7.1 to MDE 7.2. If this user exit is used, it must be adapted in case of a release change.

The evaluation "Posting during prod. lock" has already been made here. Yield counters can also be changed to scrap or be deleted.

„UpdateSheet“ → Is called after the current data of the MDE processing has been read to update the machine info.

Users of this user exit must know the internal processes of the terminal program because you manipulate the recorded quantities here.

```

'*****
Function UserExitAutomaticQuantities
    sMode=VVar("UE:PAR","MOD")
    Select Case sMode
        Case "ADD"      ' *** getMDEactivity ***
            sGut=VVar("UE:SND","AGR:GUT")
            If sGut<>"" Then
                If CInt(sGut)<>0 Then
                    UE_SND=AddIt("AGR:GUT","0","")      ' delete yield
                    UE_SND=AddIt("AGR:PRB",sGut,"")      ' yield as problem quantity
                End If
            End If
        Case "UpdateSheet"  ' *** UpdatefromMDE ***
            UE_SND=AddIt("MDES_gut","0","")      ' delete yield
    End Select
End Function
'*****

```

11.2.2.14 UserExitDynDlgBeforeInitialize

This user exit is called before a dynamic dialog is initialized.

With standard dialogs, the dialog script "DynDlgInit" is not called. Here, you can assign default values via DLGVAR using UserExitDynDlgBeforeInitialize.

You can also assign a terminal script for a programmed dialog to integrate customer-specific extensions. Or you can change the script ID, so that several dialogs are controlled using the same script.

Examples are the extensions "machine-related preassignment of badge numbers" and "recording of material consumption".

Here, the user exit implemented in this project is used to

- (1) control the standard dialogs <A_TR> , <A_UN> , <A_AB> to be able to correctly control the script dialog <A_VERB_WZB> called via the <OK> button.
- (2) perform the machine-related preassignment of badge numbers based on customer-specific dialog events.

```

' *****
Sub UserExitDynDlgBeforeInitialize
' *****
dim sDlgID
' -----
sDlgID = VOut("DLG")
If VTnr("TGRP") = "901" Then
    Select Case sDlgID
        ' ----- Recording of material consumption in toolmaking -----
        Case "A_UN", "A_AB", "A_TR"
            DLGVAR = Item("SCRIPT.ID", "A_MENGE_WZB")
        End Select
    End If
' -----
' ----- KASK_EINF_HYDRA_MW20 / 200706251 / 10 /
' ----- Preassignment of badge number in...
' -----
DialogEventBasedPresetKnr "GET", VOut("#GET#ALL#VALUES#")
' *****
End Sub
' *****

```

In this user exit, you can prevent the dialog from being opened if you set the following return value:

```
DLGVAR=Item("RET", "#CANCEL#")
```

On the AIP, you can also hide text fields.

11.2.2.15 UserExitMainInitLoopStop

This user exit is not only performed after the terminal initialization, but it is cyclically performed during operation and when you shut down the terminal

The call is performed in the following modes: (query with < VVar ("UE:PAR", "MODE") >)

- „INIT“ → on restart of terminal before the proper processing starts after initialization and loading of programs
- „LOOP“ → cyclic call if < LOOPTIME=X > if X > 0 has been set. Note: The terminal program waits until this user exit is processed. If you directly open a dialog here, the whole MDE processing is stopped. In this case, also the time on the bottom right stops running.
- „STOP“ → is called when the terminal program is closed (manually or remote via terminal status)

Explanation of variables < UE:PAR > :

MODE=INIT	mode of call
-----------	--------------

LOOPTIME=0 current LOOPTIME (default = 0)
 MINSTEP=5 minimum cycle (default=5)
 ONETIME=FALSE unique <LOOP> call if in DebugScreen "Reload-Status"
 <Ctrl+Alt+T> < UserExitMainInitLoopStop > has been activated.

If you use the mode "LOOP", you can run background operations in the terminal script. A typical use case is the integration of timeouts.

```

'*****
Sub UserExitMainInitLoopStop
'*****
' --- set LOOPTIME GLOBAL -> with "STOP" -> no meaning
' UE_RET = Item("LOOPTIME", "60")
Select Case VVar("UE:PAR","MODE")
Case "INIT"
  scrLog(" UserExitMainInitLoopStop = PAR ( "+VVar("UE:PAR","#GET#ALL#VALUES#")+ " )")
  ' --- after program start -> call LOOP after 5 seconds
  UE_RET = Item("LOOPTIME", "5")
  ' --- NOTE: if <LOOPTIME> is not set in "INIT"
  scrLog(" UserExitMainInitLoopStop = RET ( "+VVar("UE:RET","#GET#ALL#VALUES#")+ " )")
Case "LOOP"
  ' --- the next row should not be active in the RT script at the customer's
  scrLog(" UserExitMainInitLoopStop = RET ( "+VVar("UE:RET","#GET#ALL#VALUES#")+ " )")
  ' --- then -> call LOOP after 10 seconds
  UE_RET = Item("LOOPTIME", "10")
Case "STOP"
  scrLog(" UserExitMainInitLoopStop = PAR ( "+VVar("UE:PAR","#GET#ALL#VALUES#")+ " )")
End Select
'*****
'   SYS SCRIPT DEBUG
End Sub
'*****
  
```

11.2.2.16 UserExitExternalReaderEvent

You use this user exit to process the external ID/bar code readers integrated via < HYREADER.DLL >.

```

'*****
Sub UserExitExternalReaderEvent
'*****
Dim sMode,sEvent,sData
'-----
sMode = VVar("UE:PAR","MODE")
Select Case sMode
'-----
Case "CALLBACKEVENT"
'-----
'--- Here, the ID/bar code events are processed
'-----
sEvent = VVar("UE:BAR","#GET#ALL#VALUES#")
sData = VVar("UE:BAR","RAWDATA")

'-----
' !!! Implementation !!!
' see NOTES
'-----
' *** !!! bar code processed
' *** -> do not pass to application any more !!!
UE_RET = Item("RESULT","-1")

Case "CALLBACKSTATE"
  
```

```

'-----
'--- Here, the messages INFO/WARNING/ERROR are processed
'-----

'-----
' !!! Implementation !!!
'-----

Case Else
    UE_RET = Item("ACTION", "### "+sMode+" ###")
End Select
'-----
End Sub
'*****

```

Notes on processing

Terminal script	Description
UE_RET = Item("RESULT", "-1")	→ value < 0 Bar code has been processed, do not pass to application any more
UE_RET = Item("RESULT", "1")	→ value > 0 (DEFAULT with special case) Bar code has been processed, do not pass to application any more Using <HYREADER.DLL> function < ComportEventResult() >, data is written on external reader → value of <RESULT> is copied to <RET> Special case: If the ID <KNR> is included in ID/bar code event and no <IDCODE> is included. If the ID <FIR> does not match the configured <company number> (<SYSNR> from TKENN.LST), then <IDCODE> is internally set using ID/bar code event IDs <FIR>+<KNR> +<PZ>.
UE_RET = Item("RESULT", "0")	→ value = 0 (DEFAULT) Bar code is passed to application Special case: If the ID <KNR> is included in ID/bar code event and no <IDCODE> is included. If the ID <FIR> matches the configured <company number> (<SYSNR> from TKENN.LST), then <IDCODE> is internally set using ID/bar code event IDs <FIR>+<KNR> +<PZ>.
UE_RET=Item("SEND-AS-BARCODE", "FALSE")	→ value = FALSE (DEFAULT) Standard transfer of the data → value is transferred in a dialog into an active field, for example.
UE_RET=Item("SEND-AS-BARCODE", "TRUE")	→ value = TRUE Transfer of data as STD-BARCODE (identification of length, acronym,...)

Terminal-Script-Callback-Functions - scrComportDataWrite(string):string - scrComportEventResult(string):string	<HYREADER.DLL> functions for external ID/bar code readers → to write data to external readers → to write processing result to external readers (e.g. ".. RET=1 RET.TXT=Error->Firmennr ..")

11.2.2.17 UserExitOnExternOrderListChange

This user exit is called if operations have been added or removed when the order list is reloaded from the server. This is the case if the orders have been logged on or off by another terminal, by a console/MOC or by a server automatism.

For these orders, the system can perform customer-specific actions that are normally performed if the order postings are directly performed on the terminal. (Examples: setting an output signal, sending order data to a machine connection,...).

```
Function UserExitOnExternOrderListChange
  If IsCustom_ErfassungOFF Then Exit Function
  ' UE:DAT: OPs added
  ' UE:PAR: OPs removed
  ' Format: <MNR1>=<ANR1>|<MNR1>=<ANR2>|<MNR2>=<ANR3>|<MNR3>=<ANR4>|
  asLostAGData=VVar("UE:PAR", "#GET#ALL#VALUES#")
  If asLostAGDat<>"" Then
    '*****
    res=LoadNewAGPictures(asLostAGDat)
    '*****
  End If
End Function 'UserExitOnExternOrderListChange
```

Reading the data:

```
iPos=1
Do
  sEntry=scrGetPart(asLostAGDat, "|", iPos)
  If sEntry="" Then Exit Do
  sMaschine=scrGetPart(sEntry, "=", 1)
  sAuftrag=scrGetPart(sEntry, "=", 2)
  '*** edit sMachine, sOrder
End If
iPos=iPos+1
Loop
```


11.2.2.18 UserExitGetCellData

This user exit is used for the free programming of a field content in a grid.

To this end, you must define a field name in the grid configuration in ctwinlay.ini/ctaisplay.ini that starts with "@".

```
[Auftragsliste]
```

```
...
```

```
@PAL.CNT=N10.0,60,R,A.S.a.P. ; Anzahl Schläge auf Palette
```

The following data is passed to the user exit:

UE:RET	@GRD.ITMFLD	Acronym of the field (→@PAL.CNT)
	@GRD.ITMVAL	Previous value
	@GRD.ROWNUM	Row in the grid
	@GRD.COLNUM	Column in the grid
	@GRD.ACTROW	Active (selected) row of the grid
	@GRD.TABLENAME	Section in ctwinlay.ini/ctaisplay.ini → 'order list'
	@GRD.FILENAME	List file with path
	@GRD.EXTFILENAME	List file → ,anr.lst'
	@GRD.INIFILE	Actual name of layout file (e.g. ctwinlay.ini)
	@GRD.FILTER	Filter of grid (e.g. "MNR=4711")
	@GRD.ORDER	Sorting of grid
UE:GRD		The complete data row of the grid that is to be drawn

The value that is identified is returned in UE_RET using the ID "@GRD.ITMVAL".

Example:

```
Function UserExitGetCellData
  sFile=VVar("UE:RET","@GRD.EXTFILENAME")
  If sFile="anr.lst" Then
    sAuftrag=VVar("UE:GRD","ANR")
    If IsNumeric(sAuftrag) Then
```

```
sAkro=VVar("UE:RET", "@GRD.ITMFLD")
If sAkro="@PAL.CNT" Then
    sMaschine=VVar("UE:GRD", "MNR")
    sValue=CStr(iSchlagPalette("Read", sMaschine, 0))
    UE_RET=Item("@GRD.ITMVAL", sValue)
End If
End If
End If
End Function 'UserExitGetCellData
```

11.2.2.19 UserExitOnGatewayData

This user exit is called for each gateway event. There are two types of gateway events:

1. Notify GateWay-Events.

In the GateWay server routine, these events are immediately identified as received and processed for the calling program. The processing is then performed asynchronously in the main timer of the application program. → You must never write results here because a reception confirmation has already been sent.

2. Standard Gateway-Events

These events are processed as soon as the Windows queue of the application program has been processed. → Here, a result must be issued if the command is intended for an active module by default. (e.g. customer-specific events)

The following data is passed to the user exit:

UE:DAT	komplettes Gateway-Event	e.g. COM.ID=2@ DLG=KFS_MST MELDZEI=43200 MELDDAT=03/05/2008 BEARB=KFS MNR=M000002 MST=1 CLI.SND.T=10:03:59.983
UE:RET	Copy of the komplettes Gateway-Event	e.g. COM.ID=2@ DLG=KFS_MST MELDZEI=43200 MELDDAT=03/05/2008 BEARB=KFS MNR=M000002 MST=1 CLI.SND.T=10:03:59.983 RET=* → with annexed .. RET=* for internal processing

The PDM dialog can be changed. Make the following entry.

```
UE_RET = Item("#DATA#UPDATE#", "TRUE")
```

```
' -----
Sub UserExitOnGatewayData
    dim sDlgID, xDATA, xTCMS, rc
' -----

    sDlgID = VVar("UE:RET", "DLG")
    Select Case sDlgID

        ' ----- customer-specific KFS connection -----
        Case "KFS_MST", " KFS_STK", " KFS_HUB"
```

```

rc = scrStatusBarMsg(scrTranslate("# KFS-Event-Verarbeitung für Dialog
[ <DLG> ] läuft! Bitte warten ... #",Item("DLG",sDlgID)), "EVMsg", "-1")
xTCMS = scrDateTime("TCMS")
xDATA = VVar("UE:RET", "#GET#ALL#VALUES#")

If vbsExecuteKFSEvent( xDATA ) Then
    scrLog(" vbsExecuteKFSEvent ( TRUE ) "+StrFmtRight( CStr(
scrDateTime("TCMS")-xTCMS ), 8, "0" )+" msec < "+sDlgID+" > "+xDATA+" < " )
    rc = scrStatusBarMsg(scrTranslate("# KFS-Event-Verarbeitung für
Dialog [ <DLG> ] beendet! #",Item("DLG",sDlgID)), "EVMsg", "1")
Else
    scrLog(" vbsExecuteKFSEvent ( FALSE ) "+StrFmtRight( CStr(
scrDateTime("TCMS")-xTCMS ), 8, "0" )+" msec < "+sDlgID+" > "+xDATA+" < " )
    rc = scrStatusBarMsg(scrTranslate("# Abbruch der KFS-Event-
Verarbeitung für Dialog [ <DLG> ] ! #",Item("DLG",sDlgID)), "EVMsg", "1")
End If

End Select
'-----
' SYS_SCRIPT_DEBUG
End Sub
'-----

```

➔ For further information, refer to the section (1.4.3.1 Using < scrGWCUpdateResult > / < UserExitOnGatewayData >)

11.2.2.20 UserExitEventFinished

You use this user exit to execute customer-specific requirements after a successful database posting (= event).

The script processing of a DB event is structured as follows.

(e.g. for dialog „DLG=A_XYZ|MNr=M100|ANr=1A007..|DLG.DLGCFG=XYZ|...“)

The item <DLG.DLGCFG> is preferred. If <DLG.DLGCFG> is not included, <DLG> is used.

The identified event may only include the following characters „_“, „A“.. „Z“, „0“ ..“9“ !

Other characters are replaced with „_“. (e.g. DLG=ADEPRO.ADD ➔ „ADEPRO_ADD“)

1. DynDlgBeforeSend_XYZ *1
UserExitDynDlgBeforeSend ➔ Case „XYZ“
(if *1 does not exist or with background event)
2. DynDlgAfterSend_XYZ *2
UserExitDynDlgAfterSend ➔ Case „XYZ“
(if *2 does not exist or with background event)
3. Here, the label printing is performed
4. UserExitLocalMnrAnrUpdate ➔ Case „XYZ“
(if available and <MNR>/<ANR> included in DB event)
5. UserExitEventFinished_XYZ ➔ Customer-specific implementation /
(if available) / **system_<project>.scr**
6. UserExitEventFinished__XYZ__ ➔ Standard processing
(if available) / **mpdv-system.scr**

This user exit has been realized for the standard processing of coil cutting processes. Here, you can delete the cutting plans once the coil cutting OPs have been logged off or interrupted.

Example of a standard processing.

➔ Deleting cutting plans of a coil cutting OP after OP interruption (A_UN_S).

```

'-----
Sub UserExitEventFinished__A_UN_S__
'-----
    vbsDeleteSchnittplaene VSnd("MNR"),VSnd("ANR")
'-----
End Sub
'-----

```

11.2.2.21 UserExitMde72Quantities

Bisher hier realisiert Anwendung:Kundenverarbeitung system_PUEHL.scr

Here, you get the quantities calculated for the machine under MDE72 in ctwin

AGR:GUTP, AGR:AUSP, AGR:PRBP,

These quantities are calculated using the partitioning and the pulse factor

You cannot change the quantities, you can only read

You can save quantities in dialogs for a later processing.

VVar("UE:PAR","MOD")	➔	LOCAL72MNR_QUANTITIES
VVar("UE:SND","DLG")	➔	IST_ZAEHLMENGE

```

Function UserExitMde72Quantities
    Dim sMode, sDLG, sMnr, sPRB
    '    SYS_SCRIPT_DEBUG
    sMode=VVar("UE:PAR","MOD")
    Select Case sMode
        Case "LOCAL72MNR_QUANTITIES" ' *** nur lesen möglich ***
            sDLG=VVar("UE:SND","DLG")
            If sDLG = "IST_ZAEHLMENGE" Then
                sPRB=VVar("UE:SND","AGR:PRBP")
                If scrStr2Real(sPRB)<>0.0 Then
                    sMnr=VVar("UE:SND","MNR")
                    If sMnr <> "" Then
                        ' ◊ Bsp. Aufruf einer Funktion
                        Add_PRBMenge2IniFile sMnr , sPRB
                    End If
                End If
            End If
        End Select
    End Function

```

Example: Check if target quantity is reached is only possible with MDE72 posting of quantities. For each order, the quantities of the orders are available after a counter update of autom. quantities. The complete order data record is available under UE:SND if

VVar("UE:PAR","MOD") = ORDER_DATA

```

Function UserExitMde72Quantities
    Dim sMode, sAnr, sMnr, sMenge, sSoll, rSoll, r, rc
    '    SYS_SCRIPT_DEBUG
    sMode=VVar("UE:PAR","MOD")
    Select Case sMode
        Case "ORDER_DATA"                ' *** read only ***
            sMnr = VVar("UE:SND","MNR")
            sAnr = VVar("UE:SND","ANR")
            sSoll = VVar("UE:SND","SGR:GUTP")
            rSoll = scrStr2Real(sSoll)
            sMenge = VVar("UE:SND","EGR:GUTP")
            r = scrStr2Real(sMenge)
            sMenge= VVar("UE:SND","EGR:AUSP")
            r = r + scrStr2Real(sMenge)
            If (r >= rSoll) and (rSoll > 0) Then
                '-- an action can now performed here
                rc = Auftrag_unterbrechen(sMnr,sAnr)
            End If
        End Select
    UserExitMde72Quantities = TRUE
End Function

```

11.2.2.22 UserExitSysReadFile

This user exit is called if a file has been loaded using the kernel function < sys_read_file() >.

Example: "Redrawing of button bar" with CTWIN after reload of machine list (mnr.list).

```

'-----
Sub UserExitSysReadFile
'-----
    If VVar("UE:PAR","mnr.lst") <> "" Then
        UE_RET = Item("BTN.REPAINT","#TRUE#")
    End If
'-----
'    SYS_SCRIPT_DEBUG
End Sub
'-----

```

Notes:

- The loaded files are always inserted in the list using lower case letters (LowerCase).
- The data in "UE:PAR" is structured as follows

```
FILE:COUNT=1|anr.lst=c:\ctwin\spool\anr.lst;8132;2011-05-27;09:24:45.036;1;|...
```

<FILE:COUNT>	=	<number of loaded files>
<file name>	=	<path+file name> ;
		<file size in bytes> ;
		<date> (Format YYYY-MM-DD) ;
		<time> (Format HH:MM:SS.ZZZ) ;
		<transfer format> (0=Binary,1=Text)

11.2.2.23 UserExitAfterSendError

This user exit is called if the HYDRA server refuses a posting. The call is made before the error message is displayed on the terminal.

The function has been developed and tested for CTWIN. The function is generally also available on the AIP, but it has not been tested (status 21-FEB-2011).

Available functions

Description

VVar("UE:PAR","XYZ")

Send string that the terminal has sent to the server

VVar("UE:RET","XYZ")

Return string of the server (error number, error text...)

The error number is requested with VVar("UE:RET","RCV.RET") !

Here, you can store data in global variables (→GLOBALVAR) that are used when the dialog is reopened.

```
Sub UserExitAfterSendError
    Dim sDlg,sU_RET,sRET,sVal
    'msgbox("RET="+VVar("UE:RET","#GET#ALL#VALUES#"))
    'msgbox("PAR="+VVar("UE:PAR","#GET#ALL#VALUES#"))
    scrLog("UserExitAfterSendError|"+VVar("UE:PAR","#GET#ALL#VALUES#"))
    GLOBALVARS = "#RCV#AFTER#SEND#ERROR#="
    sDlg = VVar("UE:PAR","DLG")
    If sDlg = "CA_WL_PA" Then
        sU_RET = VVar("UE:RET","U_RET")
        sVal = VVar("UE:PAR","*JA_NEIN_CHECK")
        sRET = VVar("UE:RET","RCV.RET")
        If (sU_RET = "7013") Then
            ' Can be used to stop error message
            UE_RET = Item("VIEWERROR","FALSE")
        End If
        GLOBALVARS="#RCV#AFTER#SEND#ERROR#="+
VVar("UE:RET","#GET#ALL#VALUES#")+Item("U_RET",sU_RET)+I
        tem("*JA_NEIN_CHECK",sVal)
    End If
End Sub
```

Using the data when a dialog is reopened:

```
Sub DynDlgInit_CA_WL_PA
    If VOut("REOPEN")="J" Then
        s=GVars("#RCV#AFTER#SEND#ERROR#","RCV.RET")
        ' ...
```

2. example: remove a field and immediately re-send data

```
Sub UserExitAfterSendError
    Dim aRet,sData,sMATCHCHECK,sDialogText
    Select Case VVar("UE:PAR","DLG")
        Case "A_P_AN","A_AN"
            sData = VVar("UE:PAR","#GET#ALL#VALUES#")
            sMATCHCHECK = VVar("UE:RET","MATCHCHECK")
            If VVar("UE:RET","RCV.RET")="424" and (sMATCHCHECK = "FALSE") Then
```

```
sDialogText=rsCfg("DIALOG->TEXT","MATCHCHECK_TEXT","Nummer speichern?")
aRet=DlgJaNein(scrTranslate("Auswahl",""),scrTranslate(sDialogText,""))
If aRet = "#JA#" Then ' JA: re-send data
    ' here, the items that are to be deleted (U_MATCHCHECK,HALLO,CHECK)
    UE_RCV=Item("#DELETE_ITEM#","U_MATCHCHECK,HALLO,CHECK")
    'UE_RCV=Item("BZW","J")
    UE_RET=Item("VIEWERROR","FALSE") + Item("#REPEAT_SND#","J")
Else
    UE_RET = Item("VIEWERROR","FALSE")
End If
End If
End Select
End Sub
```

11.2.2.24 UserExitModifyListCmd

This user exit is called if the terminal requests lists from the server. Here, you can modify the load command.

Available functions	Description
VVar("UE:PAR","XYZ")	Load command

In UE_RET the changed send data can be returned. If you want to change the send string, the string must be completely passed here!

```
*****
Sub UserExitModifyListCmd
*****
Dim sDlg
Dim sDat
'-----
' SYS_SCRIPT_DEBUG
UE_RET = ""
sDlg = VVar("UE:PAR","DLG")
'msgbox = VVar("UE:PAR","#GET#ALL#VALUES#")
If sDlg = "LIST;74" Then
    ' All data is read
    sDat = VVar("UE:PAR","#GET#ALL#VALUES#")
    ' Ex.: If the list 74 is requested, the ID TEST=XYZ is added
    ' all data and the additional items must be returned
    UE_RET = sDat + Item("TEST","XYZ") + Item("XXX","XYZ")
End If
*****
End Sub
*****
```

11.2.3 AIP

This section describes all user exits that are only available in the AIP.

11.2.3.1 UserExitAGInfoGetCaption

You use this user exit to customize the AIP dialogs MINFO(MMINFO) and AINFO(MAINFO).

You can make outputs/changes for a field.

You can also describe an added field in the `dyn.Dialogkonfiguration`

The ID "MINFO" or "AINFO" specifies the dialog that is used to run the function.

```

'*****
Sub UserExitAGInfoGetCaption
  Dim sShowMode, s, s1, sHub, sMnr, sSzy, rSzy, r
' SYS_SCRIPT_DEBUG
  On Error Resume Next
  s = VVar("UE:PAR", "#GET#ALL#VALUES#")
  scrLog(s)
  sMnr = VVar("UE:PAR", "MNR.MNR")
  sShowMode=VVar("UE:PAR", "MODE")      ' MINFO / AINFO
  'Ex.: Output CLOCK/MIN in MINFO
  'Query: MINFO=Machine info data
  'Query: AINFO=Order info data
  If sShowMode = "MINFO" Then '// or respectively AINFO
    sMnr = VVar("UE:PAR", "MNR.MNR")
    mData = scrGetInfo("GetMachineData", "MNR="+sMnr)
    sSzy = scrDDitem("SZY",mData)
    rSzy = scrStr2Real(sSzy) / 1000
    If rSzy <> 0 Then
      r = 60 / rSzy
      'msgbox("SZY="+sSzy)
      s = RealToStrNK(r,2)
      s1 = RealToStrNK(rSzy,2)
      UE_RET=Item("HUB",s) ' + Item("MNR.SZY",s1)      End If
    End If

    If Err.Number <> 0 Then
      scrLog("Error:UserExitAGInfoGetCaption|ERR.Number:"&CStr(Err.Number) &
"|Quelle:"&Err.Source &"|Beschreibung:"&Err.Description&"|")
    End If
    On Error Goto 0
  End Sub
'*****

```

11.2.3.2 UserExitAfterListLoaded

Using this user exit, the developer can perform standard and custom extensions in the user exit after request of lists.

This user exit is therefore run twice to develop the custom and the standard extensions each.

1. `UserExitAfterListLoaded_LIST_13` → Custom implementation /
(if available) / `aip_system_<project>.scr`
2. `UserExitAfterListLoaded__LIST_13__` → Standard processing
(if available) / `aip_mpdv-system.scr`

' Example: Standard extension <DLG=LIST;13|MOD=P|..>

```

Sub UserExitAfterListLoaded__LIST_13__

```



```

dim uD,xD,rc,xM,xF,ix,sM,sC,xC,xG,wr,cx
'-----
uD = VVar("UE:LST.PAR","DLG")
If( VVar("UE:LST.SND","MOD") = "P" )Then
  xM = ""
  rc = GSrce( "LOAD" , Item("FILE",DIR_SPOOL+"mnr.lst") )
  rc = GSrce( "FIRST" , "" )
  If rc <> "#EOF#STORE#" Then
    Do
      If( scrPosStr(";A;",";"+VSrce("VISLIST3")+";" ) <> "" )Then
        xM = xM + VSrce("MNR")+ "|"
      End If
      rc = GSrce( "NEXT" , "" )
    Loop Until rc = "#EOF#STORE#"
    rc = GSrce( "CLOSE" , "" )
  End If
  If( xM <> "" )Then
    xF = VVar("UE:LST.PAR","FILE")
    '-----
    ' ... further processing see <aip_mpdv_system.scr>
    '-----
  End If
End If
'-----
' SYS_SCRIPT_DEBUG
End Sub
'-----

```

11.3 DIALOG – Script Processing

The script dialog processing has been implemented for the initialization and the dialog control of new dynamic dialogs (that are not implemented in the source code). Such dialogs are configured or called via an entry in the file "ctwinbut.ini" or "ctaipbut.ini".

For notes on the storage and naming of the system script, refer to the section "1.2 Storage". For notes on the processing, refer to the section "1.3 Processing".

11.3.1 CTWIN + AIP

This section describes dialog user exits that you can use in the CTWIN and in the AIP.

Currently, the following dialog user exits are implemented or defined:

Dialog "user exits"	Script description
DynDlgInit_[DIALOG-ID]	Dyn. dialog (initialization)
DynDlgGridInit_[DIALOG-ID]	Dyn. dialog (grid initialization)

DynDlgFieldChange_[DIALOG-ID]	Dyn. dialog (input field - change/bar code)
DynDlgFieldExit_[DIALOG-ID]	Dyn. dialog (input field - exit)
DynDlgFieldListe_[DIALOG-ID]	Dyn. dialog (input field - attached list)
DynDlgFunctions_[DIALOG-ID]	Dyn. dialog (button - function)
DynDlgBeforeSend_[DIALOG-ID]	Dyn. dialog (before DB posting)
DynDlgAfterSend_[DIALOG-ID]	Dyn. dialog (after DB posting with <RET=0 ..>)
DynDlgTimer_[DIALOG-ID]	Dyn. dialog (timer for cyclic processings)
DynDlgFormValidationBeforeFunction_[DIALOG-ID]	Dyn. dialog (entry of validation before execution of function)
DynDlgKeyDown_[DIALOG-ID]	Dyn. dialog (keyboard - events)
DynDlgPluginCreate_[DIALOG-ID]	Dyn. dialog (plug-in - initialization)
DynDlgWFTabEnter_[DIALOG-ID]	Dyn. dialog (display before workflow)
DynDlgWFTabExit_[DIALOG-ID]	Dyn. dialog (exit before workflow)

11.3.1.1 DynDlgInit_XYZDynDlgInit_XYZ

This user exit is called when a script dialog is initialized. For standard dialogs that are processed using a terminal program, this user exit is not run.

Available functions

Description

VVar("DLG.DLG", "XYZ")	Basic initialization of the dyn. dialog (e.g. DLG=M_MST ...)
VPar("XYZ")	Parameter of the dialog call (or the dialog data of the calling dialog)
VMnr("XYZ")	Current machine info from MNR.LST for the machine selected in the main view. (Values in < VPar("XYZ") > take priority)
VAnr("XYZ")	current order info from ANR.LST for the order selected in the main view. (Values in < VPar("XYZ") > take priority)
VVar("DLG.CGD", "XYZ")	includes (if available) the current row of the third grid of the main view or the selected row if the call has been made using a button in a dynamic dialog with grid.

The functions: VMnr(), VAnr() und VVar("DLG.CGD") include the following additional information:

- <#FILE#LIST#> → includes the file name without path
- <#FILE#NAME#> → includes file name with path

The values of these fields are in lower case letters

For the active grid of the main view, the following information is additionally passed:

→ "..|#GRD#STATE#=FOCUS|.."

From a script dialog with grid, the following value is passed

→ "..|#GRD#STATE#=DIALOG|.."

→ Important :

(1) Also in case of dialog input, the information is not updated, i.e. if you change the machine or the order in the dialog, these values are not changed.

(2) Also if you access the < DynDlg..._ > user exit that follows, the variable content might not be available or correct.

→ The values required for the processing should be included in STATUS or in hidden dialog fields. Using the < DLGVAR = Item("<field ID>", VMnr("MNR")) > you can create a hidden field in the dialog in each < DynDlg... > user exit. You can also save values using < GLOBALVARS = "#XXX #PAR#=WAAGENTERMINAL=1" >. The developer is responsible for editing the contents and deleting after use.

```

'-----
Sub DynDlgInit_XYZ
'-----
    If VOut("REOPEN") = "J" Then
' *****
' ----- repeated opening of the dialog, e.g. after DB plausibility error <RET=..|KT=..|LT=..|>
' *****
' scrMsgBox(" RET = "+VOut("RET")+" -> Reopen [ "+VOut("DLG")+" / "+VOut("ScriptID")+" ]")
    Else
' *****
' ----- Plausibility checks if authorization for opening dialog exists
' *****
        If "X" <> "X" Then
            ' scrMsgBox(" Dialog -> Plaus. error [ "+VOut("DLG")+" / "+VOut("ScriptID")+" ]")
            DLGVAR = AddIt("RET", "$">"+VOut("DLG")+"<" , "" )
            DLGVAR = AddIt("KT", "(Kurzttext)" , "" )
            DLGVAR = AddIt("LT", "(Langtext)" , "" )
        Else
' *****
' ----- opening the dialog e.g. via <ButtonClick()> or via <Remote-Dialog-Call()>
' *****
            ' scrMsgBox(" Dialog -> Init [ "+VOut("DLG")+" / "+VOut("ScriptID")+" ]")
            DLGVAR = AddIt("DT", SYS_DT, "" )
            DLGVAR = AddIt("MNR", VMnr("MNR"), "" )
            DLGVAR = AddIt("ANR", VAnr("ANR"), "" )
        End If
    End If
'-----
    SYS_SCRIPT_DEBUG
End Sub
'-----

```

If the check "X"<>"X" is replaced by a reasonable plausibility check, the opening of a dialog can be stopped. An error message is shown that is specified via the return values set (RET, KT, LT). If you do not want to show an error message, you can specify a specific return value:

```
....DLGVAR=AddIt("RET", "#INVISIBLE#MSG#", "") ' do not show dialog standard message
```

11.3.1.2 DynDlgGridInit_XYZ

If the dialog includes a grid, you can initialize it using this user exit. Condition: The grid must be configured with the "field attribute" SCRIPT_GRID.

```

-----
Sub DynDlgGridInit XYZ
-----
    SCRVARs = "GRD.CMD="      + "DLG=LIST;104|MOD=U|MNR=<MNR>|ANR=<ANR>|"
    SCRVARs = "GRD.FILE="    + "c_unklar.lst"
    SCRVARs = "GRD.INI="     + "hytnrcfg.ini"
    SCRVARs = "GRD.SECTION=" + "Layout->C_UNKLAR"
    SCRVARs = "GRD.FILTER="  + ""
    SCRVARs = "GRD.ORDER="   + "CNR"
-----
End Sub
-----

```

Parameter	Description
GRD.CMD	Command to load list from HYDRA server. If value is set, the list is loaded on opening of dialog.
GRD.FILE	The data of this file (in the subdirectory "spool") is displayed.
GRD.INI	Configuration file including the layout configuration (default: ctwinlay.ini or ctaipplay.ini).
GRD.SECTION	Section in the configuration file that includes the layout.
GRD.FILTER	Filter criterion to show only part of the data records of the list.
GRD.ORDER	Sorting criterion – you can specify several field IDs separated by " ". The first criterion has the highest priority. Example for descending sorting: GRID_ORDER=MSDAUER=-

Notes:

1. With the following entry, you can also use the < GRID_ORDER > entry of a < SCRIPT_GRID > that is included in the configured section of the INI file.
SCRVARs = "GRD.ORDER=" + "#USE#INI#ITEM#"
2. If you want to reload the grid after having changed the file, you can set the following value in a dialog script (e.g. DynDlgFieldListe_xx):
DLGVAR = AddIt ("DLG.GRID", "RELOAD", "")

11.3.1.3 DynDlgFieldChange_XYZ

This user exit is called to change a field. If you manually enter "123" in a field, the user exit is run 3 times. In most cases, it is better to use the user exit DynDlgFieldExit_XYZ because the system should only react to the value entered as a whole.

Also if another row is selected in the grid, this user exit is called.

➔ After execution of the function < scrFieldChange >, the result configured with "LST.RET" is passed to <[n#]DLG.OUT>. If no entry is found, the specified fields are deleted and the input field is colored in **cyan**.

NOTE! In the user exit, you can **not** set IDs including * placeholders (DLGVAR = AddIt("*ABC", ... is ignored)

```

'-----
Sub DynDlgFieldChange_XYZ
'-----
    Select Case VDlg("DLG.FLD")
        Case "MST:2"
            LSTVARS = "LST.FILE=" + "mstat.lst"
            LSTVARS = "LST.FILTER=" + "MNR="+VDlg("MNR")+ " & "+"MST="+VDlg("MST:2")
            LSTVARS = "LST.RET=" + "MSTTXT:2=MSTTXT"
            scrFieldChange
        End Select
    End Sub
'-----

```

Dialog with grid: If you select a row, data of this row should be passed to the dialog fields.

```

'-----
Sub DynDlgFieldChange_RES_AB
    Select Case VDlg("DLG.FLD")
        Case "DLG.GRD", "DLG.GRD.DBLCLK"
            If VStore("RES") <> "" Then
                DLGVAR=Item("RES",VStore("RES"))
                DLGVAR=Item("RESTYP", VStore("RESTYP"))
            End If
        End Select
    End Sub
'-----

```

11.3.1.4 DynDlgFieldExit_XYZ

This user exit is called if an input field in the dialog is exited or if a field has obtained a bar code. You can identify that it is a bar code event using the query << If VDlg("FLD.MOD")="BARCODE" Then..>>.

The dialog data is available in VDlg („XYZ“) . Use the function DLGVAR to pass values to the dialog.

Example: After having exited a field with the ID "MST:1", the system reads the status text from the list that is entered for the specified status and enters it in the dialog:

```

'-----
Sub DynDlgFieldExit_XYZ
'-----
    Select Case VDlg("DLG.FLD")
        Case "MST:1"
            LSTVARS = "LST.FILE=" + "mstat.lst"
            LSTVARS = "LST.FILTER=" + "MNR="+VDlg("MNR")+ " & MST="+VDlg("MST:1")
            LSTVARS = "LST.RET=" + "MST:1=MST"+ " & "+"MSTTXT:1=MSTTXT"
            scrFktList
        End Select
    End Sub
'-----

```

Using the extension `< VDlg("FLD.MOD") = "MOUSEDOWN" >` in CTAIP (as of V# 2.0.3.6), you can implement a "grid recording" in a configured `<IMAGE>` using the field attribute `<MOUSEDOWN>`. With this "grid recording", you can for example place a grid on an article image to record the position of a failure.

```

'-----
Sub DynDlgFieldExit_XYZ
'-----
  If VDlg("FLD.MOD") = "MOUSEDOWN" Then
    Select Case VDlg("DLG.FLD")
      Case "ATKIMG"
        x = Int( CInt(VDlg("ATKIMG@XPOS")) / Int(CInt(VDlg("ATKIMG@WIDTH")) / 10))+1
        y = Int( CInt(VDlg("ATKIMG@YPOS")) / Int(CInt(VDlg("ATKIMG@HEIGHT")) / 5))+1
        If x > 10 Then x = 10
        If y > 5 Then x = 5
        DLGVAR = Item("RASTER:X",CStr(x))+ Item("RASTER:Y",CStr(y))
      Case Else
    End Select
  Else
    ...
  End Select
'-----
End Sub
'-----

```

11.3.1.5 DynDlgFieldListe_XYZ

You can use this user exit to implement a list selection for any field.



`VDlg („DLG.FLD“)` includes the ID of the field whose list button has been pressed.

The function `LSTVARS` is filled with the parameters for the list:

LST.CMD	Command to request the list from the HYDRA server (optional).
LST.FILE	File name (the local directory "spool" is always put in front).
LST.CAPTION	Window caption of the selection dialog
LST.FILTER	Filter for the list to be displayed (e.g. "MNR=100 & ZUMAN=J N")
LST.SORT	List sorting
LST.INI	INI file where the <code><Section></code> is read (""=ctwinlay.ini,ctwinlay.<TGRP>)
LST.SECTION	INI section including the layout definition of the list to be displayed
LST.RET	Configuration of the values from the list that are transferred into the calling dialog e.g. <code>< MST:1=MST"> + " & "+"MSTTXT:1=MSTTXT" ></code> copies the values of columns <code><MST></code> and <code><MSTTXT></code> of the selected entry of the list in the dynamic dialog fields <code><MST:1></code> und <code><MSTTXT:1></code> .
LST.MODE	Additional processing modes (configurations separated by " ")

	"COLNUMSORT=TRUE" (or in INI section GRID_COLNUMSORT) "DYNAMICFILTER=MGRP,MNR,MST" (or in INI section <GRID_DYNAMICFILTER) "PAGESCROLLING=TRUE" (or in INI section GRID_PAGESCROLLING) "WILDCARD=+" "FILTERSENSITIVE=TRUE" (or in INI section GRID_FILTERSENSITIVE) "DISABLE-BUTTONS=6;7;8" (disabling of standard buttons if you use list function scrFieldVAGList)
--	---

The result string configured in LST.RET is transferred into the global variable <[n#]DLG.OUT> (is equal to <DLGVAR>). Additionally, the complete row selected is stored in <[n#]LST.VALUES>. If the list has been read by the HYDRA server (→LST.CMD), the result of the server request is saved in <[n#]LST.CMD:RET>.

Note: → as < LST.FILE > you may not specify a static list

Example: Selection of a local list

→ Example: Entry of an additional MST in a dialog (this selection is not possible by default, i.e. currently, you can only realize this selection using a script).

→ Usage of a general selection list < scrFieldList > in (f_liste.pas)

```

'-----
Sub DynDlgFieldListe XYZ
'-----
  Select Case VDlg("DLG.FLD")
  ' ---- Input field with dialog list button
  Case "MST:1"
  ' ---- Initialization of the <LSTVARS>
    LSTVARS = ""
  ' ---- File name (local spool directory is always put in front)
    LSTVARS = "LST.FILE=" + "mstat.lst"
  ' ---- Window caption of the selection dialog
    LSTVARS = "LST.CAPTION=" + "Maschinenstatusliste [ <DLG> ]"
  ' ---- Filter for the list to be displayed (e.g. "MNR=100 & ZUMAN=J|N" )
    LSTVARS = "LST.FILTER=" + "MNR="+VDlg("MNR")
  ' ---- List sorting
    LSTVARS = "LST.SORT=" + "MNR|MST"
  ' ---- INI file where the <Section> is read ("=ctwinlay.ini,ctwinlay.<TGRP>)
    LSTVARS = "LST.INI=" + ""
  ' ---- INI section including the layout definition of the list to be displayed
    LSTVARS = "LST.SECTION=" + "Maschinenstatusliste"
  ' ---- Configuration of the values from the list that are transferred into the calling
  dialog
  ' ---- - e.g. < MST:1=MST"> & ">MSTTXT:1=MSTTXT" >
  ' ---- - copies the values of columns <MST> and <MSTTXT> of the entry selected
  ' ---- - of the list in the dynamic dialog fields <MST:1> und <MSTTXT:1>
  ' ---- - with < MST "> & ">MSTTXT" > the DlgID is not transferred
    LSTVARS = "LST.RET=" + "MST:1=MST"> & ">MSTTXT:1=MSTTXT"
  ' ---- Additional processing modes (configurations separated by "|")
  ' ---- - "COLNUMSORT=TRUE" (or in INI section <GRID COLNUMSORT)
  ' ---- - "DYNAMICFILTER=MGRP,MNR,MST" (or in INI section <GRID DYNAMICFILTER)
  ' ---- - "PAGESCROLLING=TRUE" (or in INI section <GRID PAGESCROLLING)
  ' ---- - "WILDCARD=+"
  ' ---- - "FILTERSENSITIVE=TRUE" (or in INI section <GRID_FILTERSENSITIVE)
    LSTVARS = "LST.MODE=" + ""

```

```

' ---- further INI section configurations / internal note
' ---- - GRID MAXIMIZE LIST=TRUE          (shows the selection list in maximized size)
' ---- - the field contents of the calling dialogs are also transferred
scrFieldList
End Select
'-----
End Sub
'-----

```

Example: Selection of an ONLINE/server list

→ Example: Implementation of a module (RS) or a customer-specific sequencing list (user exit to server)

→ Use of order sequencing list < scrFieldVAGList > in (f_vorlst.pas)

→ Section < sequencing list > of customer-specific global INI file < hytnrcfg.ini >

```

'-----
Sub DynDlgFieldListe_A_AN_RS
'-----
  Select Case VDlg("DLG.FLD")
    case "ANR"
      ' ---- Initialization of the <LSTVARS>
      LSTVARS = ""
      ' ---- File name (local spool directory is always put in front)
      LSTVARS = "LST.FILE=" + "vrslst.lst"
      ' ---- Server command to request file
      LSTVARS = "LST.CMD=" + "DLG=LIST;11|MOD=V|MNR="+VDlg("MNR")+ "|MOD3=M| "
      ' ---- Window caption of the selection dialog
      LSTVARS = "LST.CAPTION=" + "Vorgabeliste [ <MNR> -> <DLG> ]"
      ' ---- INI file where the <Section> is read ("=ctwinlay.ini,ctwinlay.<TGRP>")
      LSTVARS = "LST.INI=" + "hytnrcfg.ini"
      ' ---- INI file where the <Section> is read ("=ctwinlay.ini,ctwinlay.<TGRP>")
      LSTVARS = "LST.SECTION=" + "Vorgabeliste"
      ' ---- Configuration of the values from the list that are transferred into the calling
      dialog
      LSTVARS = "LST.RET=" + "ANR" & "+" & "ATK" & "+" & "ABEZ=AGBEZ"
      ' ---- Additional processing modes (configurations separated by "|")
      ' ---- - "COLNUMSORT=TRUE"          (or in INI section <GRID COLNUMSORT)
      ' ---- - "DISABLE-BUTTONS=6;7;8"    (disabling of standard buttons)
      LSTVARS = "LST.MODE=" + ""
      scrFieldVAGList
    Case Else
      ' scrMsgBox ( "FLD.LISTE = "+VDlg("DLG.FLD") )
  End Select
'-----
End Sub
'-----

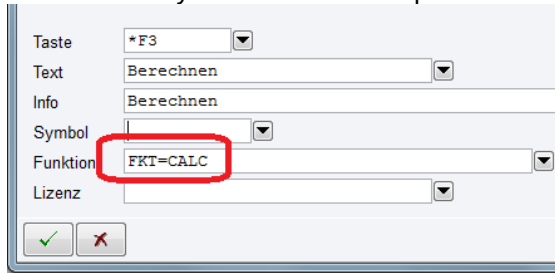
```

11.3.1.6 DynDlgFunctions_XYZ

You use this user exit to implement function keys of the dialog. It is also called if the dialog is exited via OK or CANCEL and if the processing returns from a dialog called.

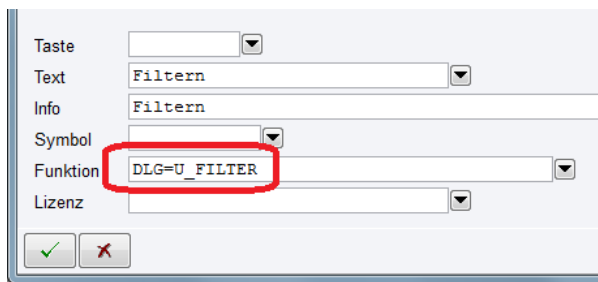
Note the following for function keys:

- To assign the function to the function key, the dialog configuration must start with "FKT=" to redirect the key to the terminal script.



- For the example above, the request would be "CALC" in the select-instruction for VDIg(„DLG.FKT“).
- A value must be returned to the function. The terminal program then knows that the function key has been processed in the terminal script. Otherwise an error message is displayed.
Example: DLGVAR="RET=0"

The system returns from a dialog called (e.g. CASE „DLG=U_FILTER“) if the call was performed using the function key configuration:



The data of the dialog called (and now closed) is then available in VVar("DLG.RET","XYZ"). You can access the dialog called as usual via VDLG(„XYZ“).

If you leave the dialog, the case "DLG.CLOSE=TRUE" is run. The query <<VDIg("DLG.RESTYP")="1">> specifies if the request was started by clicking OK or CANCEL.

```

-----
Sub DynDlgFunctions_XYZ
-----
Select Case VDIg("DLG.FKT")
-----
Case "MST:BTN:1"
' *****
' *** Example for a selection list
' *** unusual see < DynDlgFieldListe ... >
' *****
DLGVAR = "RET=0"
LSTVAR = "LST.FILE=" + "mstat.lst"
LSTVAR = "LST.CAPTION=" + "Maschinenstatusliste [ <DLG> ]"
LSTVAR = "LST.FILTER=" + "MNR="+VDIg("MNR")
LSTVAR = "LST.SORT=" + "MNR|MST"
LSTVAR = "LST.SECTION=" + "Multi->Maschinenliste"

```

```

LSTVARS = "LST.RET="      + "MST:1=MST" & "+"MSTTXT:1=MSTTXT"
scrFieldList
'-----
Case "DLG=U XYZ"
' *****
' *** Example of a return to the calling dialog/script
' *** after execution of a script dialog with the
' *** Condition: button with function „DLG=U_XYZ“ must have RCode (7,8,9)
' *** Purpose: take over values
' *** VDlg("<>") -> data of calling dialog
' *** VVar("DLG.RET","<>") -> data of dialog called
' *****
If VVar("DLG.RET", "DLG.RESTYP") = "7" Then
    DLGVAR = Item("FAKTOR", VVar("DLG.RET", "FAKTOR") )
End If
'-----

Case "DLG=U ABC"
' *****
' *** Example of a return to the calling dialog/script
' *** after execution of a script dialog with the
' *** Condition: button with function „DLG=U_XYZ“ must have RCode (7,8,9)
' *** Purpose: process control
' *****
If VVar("DLG.RET", "DLG.RESTYP") = "1" Then
    DLGVAR = Item("DLG.RESTYP", "9")      ' dialog remains open
Else
    DLGVAR = Item("DLG.RESTYP", "0")      ' dialog is closed and sent
End If
'-----

Case "DLG.CLOSE=TRUE"
' *****
' *** To prevent that dialog can be closed if a condition
' *** is fulfilled
' *** Ex.: If the dialog field has the value <> „0“ the dialog must not be
' *** closed via ESC / virtual key "Cancel" or button with RCode (1)
' *****
If VDlg("DLG.RESTYP") = "1" Then
    If VDlg("NUM") <> "0" Then
        DLGVAR = Item("DLG.CLOSE", "FALSE")
    End If
End If
'-----

Case "OUT:1#2"
' *****
' *** Script function <FKT=OUT:1#2> to set an output with PCCDLL connection
' *****
' set output 1 (=channel 301) and remove output 3 (=channel 303)
scrPCCValues("DLG=SETVAL|O:0301=1|O:0303=0|")
End Select
'-----
End Sub
'-----

```

11.3.1.7 DynDlgBeforeSend_XYZ

Just like the „UserExitDynDlgBeforeSend“, the dialog-specific user exit „DynDlgBeforeSend“ is called before a posting is sent to the HYDRA server. The call is only performed if the dialog script is loaded - i.e. if the dialog is open or has just been closed with this posting. If a posting with identical dialog ID (DLG=XYZ) is sent in the background when the dialog is closed, the dialog script does not work.

If „DynDlgBeforeSend“ is loaded, then „UserExitDynDlgBeforeSend“ is not run!

Especially with complex project, it is therefore recommended to use only "UserExitDynDlgBeforeSend" in the system script.

The definition in the dialog script is enough with dialog-oriented postings, i.e. if the posting is not triggered by a PCCDLL event or similar.

You can use the function DLGSND to change the send string as shown in the example below. The processing of the posting is controlled via the ID "EVENT=EVENT_...":

EVENT_DIALOG_OHNE_SENDEN:

If you press OK, the dialog is not sent. This is used if you only use the dialog to show data of if the actual posting is explicitly sent with open dialog.

EVENT_OHNE_AUTO_MENGEN:

No automatic quantities of the machine are added to the posting. You should use this setting for customer-specific postings because the server does not process automatic quantities by default. This way, quantities can be lost.

EVENT_MIT_AUTO_MENGEN:

This is the default behavior.

EVENT_QUEUE_OHNE_AUTO_MENGEN, EVENT_QUEUE_MIT_AUTO_MENGEN:

This posting is first set in the queue of the terminal and is then issued with delay. The same behavior is used by default for the PZE postings CLOCK IN/OUT.

EVENT_ONLINE_OHNE_AUTO_MENGEN, EVENT_ONLINE_MIT_AUTO_MENGEN:

The posting may only be sent online. If an immediate posting cannot be sent to the server, the data record is not added to the queue. Instead the message is rejected with an error code. This variant is used if it is important for further processing that the posting has been booked on the server. This way, the server can perform processing steps that are not known to the terminal. After confirmation of the booking, the terminal can load lists that include the result of the processing.

```

'-----
Sub DynDlgBeforeSend_XYZ
'-----
' *****
' *** EVENT -> Additional parameters to control dialogs that are not implemented
' *****
' *** EVENT_DIALOG_OHNE_SENDEN
' *** EVENT_OHNE_AUTO_MENGEN          ,EVENT_MIT_AUTO_MENGEN
' *** EVENT_QUEUE_OHNE_AUTO_MENGEN    ,EVENT_QUEUE_MIT_AUTO_MENGEN
' *** EVENT_ONLINE_OHNE_AUTO_MENGEN    ,EVENT_ONLINE_MIT_AUTO_MENGEN
' *****
DLGSND = AddIt("DT", SYS_DT, "")
DLGSND = AddIt("EVENT", "EVENT_ONLINE_OHNE_AUTO_MENGEN", "")
' *****
' *** Here, you can change / correct the dialog send ID / field ID if required
' *****
DLGSND = AddIt("DLG", "A_TR", "")
DLGSND = AddIt("MST", VDlg("MST:1"), "")
'-----
End Sub
'-----

```

11.3.1.8 DynDlgAfterSend_XYZ

This user exit is called, once a posting has been transferred successfully to the HYDRA server. It is an alternative option to the UserExitDynDlgAfterSend in the system script. The same rules apply with respect to the processing at the same time as for DynDlgBeforeSend and UserExitDynDlgBeforeSend.

With script dialogs, the main lists (MNR,ANR,PNR,TNRMAT) are loaded by default after a posting. If you set the item <LOAD>, the standard update is not performed. In the example below, only the MNR.LST and MSTAT.LST are reloaded.

If you add <RES> for the resource list (as of MDE/WRM >= 7.2.1), the system ignores this, if the product version <WRM> is smaller than <7.2.1> or if no active resource list display is configured for this terminal (<MNR.VISLIST3> does not include „R“).

```

'-----
Sub DynDlgAfterSend_XYZ
'-----
' *****
' *** for DD-List-Reload [ ANR,MNR,PNR,MAT,MST,RES "=no DD-Lst-Updates] *****
' *** <RES> for resource list (as of WRM/MDE > 7.2)
' *****
' *** => this row updates the MNR.LST + MSTAT.LST on the terminal
' *** => with „VRcv("LOAD")“ the DD-List-Reloads are added by the "Server"
' *****
' DD_RCV = Item( "LOAD", "MNR,MST," +VRcv("LOAD") )
'-----
End Sub
'-----

```

Trick: reopen dialog after posting until "Cancel" is pressed:

```

Sub DynDlgAfterSend_RES_AN
' *****
rc=scrSetData("DelayedButtonClick","RES_AN")
' *****
End Sub

```

Trick 2: reopen dialog after posting until sending is successful (prevent "Cancel"):

```
rc=scrSetData("DelayedButtonClick","CA_WL|FORCEDIALOG=ON")
```

11.3.1.9 DynDlgTimer_XYZ

To realize a cyclic call, you can not only use the timer function available in the system script via UserExitMainLoopStop, but you can also realize it via "DynDlgTimer" in the dialog.

The timer is activated, if you set the interval in ms in the UserExit DynDlgInit_XYZ:

```
DLGVAR=Item("DYNDLG.TIMER","100")
```

The timer event is only triggered, if the terminal is running in the foreground.

You can change the interval in the timer. You deactivate the timer if you transfer "0".

```

'-----
Sub DynDlgTimer_XYZ
'-----
' *** DLG.RESTYP is not processed in the result <DLGVAR>
'-----
' *** the following row displays the date and the current time in the dialog field <DT>
'-----
' DLGVAR = Item("DT",Cstr(now))
'-----
End Sub
'-----

```

11.3.1.10 DynDlgFormValidationBeforeFunction_XYZ

You use this user exit to check the dialog entries before the user exit "DynDlgFunctions" is executed.

```

'-----
' *** Function used for FormValidation before <Button> function is executed
' *** -: - DLG.RESTYP is not processed in the result <DLGVAR>
' ***      - Result <DLGVAR> is not processed
' *** - Notes / processing:
' *** - DLG.PROCESS.RESULT=TRUE in the result <DLGVAR> takes over the results of <DLGVAR>
before
' ***      an eventual FormValidation
' ***      -> if <DLG.PROCESS.RESULT> <> <TRUE>, no <DLGVAR> is taken over
' **** - DLG.FORM.VALIDATION=TRUE in the result <DLGVAR> executes the FormValidation before the
' ***      configured <Button> function
' ***      -> If an error occurs, the <Button> function is not performed
' *** - DLG.SET.FORM.VALIDATION.ERROR=XXX sets an FormaValidationError and focuses
' ***      /colors the input field passed <XXX> red
'-----
Sub DynDlgFormValidationBeforeFunction_XYZ
'-----
If VDlg("DLG.FKT") <> "" Then
' --- <FormValidationBeforeFunction> activate because of defined <Button> function
Select Case VDlg("DLG.FKT")
Case "DLG=V_BLZ"
' --- Transfer of <DLGVAR> before <FORMVALIDATION> ---
DLGVAR = Item("DLG.PROCESS.RESULT", "TRUE")
DLGVAR = Item("DLG.RESTYP", "9")
' --- perform / activate <FORMVALIDATION> before <Button> function ---
DLGVAR = Item("DLG.FORM.VALIDATION", "TRUE")
If VDlg("XXX") = "" Then
' --- Setting a <FORM.VALIDATION.ERROR> independent of DynDlg configuration ---
DLGVAR = Item("DLG.SET.FORM.VALIDATION.ERROR", "XXX")
' MsgPopUp scrTranslate("Wert für Feld <XXX> erforderlich", "") , "3"
End If
Case Else ' DEFAULT [ VTST, .. ]
DLGVAR = Item("DLG.FORM.VALIDATION", "FALSE")
End Select
' SYS_SCRIPT_DEBUG
Else
' --- <FormValidationBeforeFunction> activate because of <Button>-<RCODE>
Select Case VDlg("DLG.RESTYP")
Case "0", "7"
' DLGVAR = Item("DLG.FORM.VALIDATION", "TRUE")
End Select
' SYS_SCRIPT_DEBUG
End If
'-----
'      SYS_SCRIPT_DEBUG
End Sub
'-----

```

11.3.1.11 DynDlgKeyDown_XYZ

You use this DynDlg user exit for the dialog-specific processing of keyboard events. In general, you can use this user exit to react in the script to each single key pressed. But because the focus changes each time, this can have the result that you must reprogram basic editing functions in the script depending on the required processing.

11.3.3 AIP

This section describes all user exits that are only available in the AIP.

Dialog "user exits"	Script description
DynDlgPluginCreate_[DIALOG-ID]	Dyn. dialog (plug-in - initialization)
DynDlgWFTabEnter_[DIALOG-ID]	Dyn. dialog (display before workflow)
DynDlgWFTabExit_[DIALOG-ID]	Dyn. dialog (exit before workflow)

11.3.3.1 DynDlgPluginCreate_XYZ

You use this [CTAIP](#) DynDlg user exit to initialize a dialog / workflow plug-in.

The processing is equal to the processing of the DynDlg user exit <DynDlgInit_XYZ>.

11.3.3.2 DynDlgWFTabEnter_XYZ

This [CTAIP](#) DynDlg user exit is executed before the display of a dialog or a workflow tab. Using this user exit, you can perform an initialization without changing the dynamic dialog configuration. The initialization is similar to the configured dialog function (1) (e.g. FKT=DLGSHOW) in the user exit <DynDlgFunctions_XYZ>.

11.3.3.3 DynDlgWFTabExit_XYZ

This [CTAIP](#) DynDlg user exit is executed before the display of a dialog or a workflow tab. Using this user exit, you can perform an initialization without changing the dynamic dialog configuration. The initialization is similar to the configured dialog function (2) (e.g. FKT=DLGEXIT) in the user exit <DynDlgFunctions_XYZ>.

If you set the item <RESULT> to the value <FALSE>, the dialog cannot be closed or the workflow tab cannot be exited.

```

'-----
Sub DynDlgWFTabExit_XYZ
'-----
    If VDlg("XYZ") <> "" Then
        DLGVAR = Item("RESULT","FALSE")
    End If
'-----
End Sub
'-----

```

Notes:

- You can force a tab change using the following command:
rc=scrSetData("DelayedDialogFunction","DLG=@ACTIVE|WFTAB=WF_AAN_CHK")
- Use the following commands to change to a previous/subsequent tab:
rc=scrSetData("DelayedDialogFunction","DLG=@ACTIVE|WFTAB=#PREV#")
rc=scrSetData("DelayedDialogFunction","DLG=@ACTIVE|WFTAB=#NEXT#")

11.3.4 Tips and tricks with the dialog control

This section describes tips and tricks to control dialogs.

- Starting a dialog timer for monitoring
(Example: Cycle 500 msec
→ Processing in <DynDlgTimer_XYZ >)
→ `DLGVAR = Item("DYNDLG.TIMER","500")`
- Starting a dialog autoclose timer
(Example: run time <7> seconds + return code <1> = CANCEL
→ Processing in < DynDlgFunctions_ _XYZ >)
→ `DLGVAR = Item("DLG.TIMER","7^1")`
(Example: run time <10> seconds + return code <0> = OK
→ Processing in < DynDlgFunctions_ _XYZ >)
→ `DLGVAR = Item("DLG.TIMER","10^0")`
(Example: Timer remains active also without dialog focussing
Default is „`..^..^1`“ / Timer stops if dialog is not active/focused
Run time <10> seconds + return code <1> = CANCEL
→ Processing in < DynDlgFunctions_ _XYZ >)
→ `DLGVAR = Item("DLG.TIMER","10^1^0")`
- Creating temporary dynamic dialog variables
(Example: Variable <*XXX> with value <1>
→ Processing in all < DynDlg.._XYZ > functions)
→ `DLGVAR = Item("*XXX","1")`
- Controlling of dialog buttons with ID <> ""
(Example: Button (BTN.<CANCEL>) with text <ESC> and font color <clRed>
→ Processing in all < DynDlgFunctions_XYZ >)
→ `DLGVAR = AddIt("BTN.CANCEL","ESC,clRed",cFFEnable)`
- Workaround: Change list in UNIX format into Windows format on the terminal
If the server is on a UNIX system, the lists requested on the terminal are stored in UNIX format on the terminal. But the terminal can only read files in Windows format. Using the following commands, the file on the terminal is saved in Windows format.
`rc = GSrce("LOAD", Item("FILE", DIR_SPOOL + ***file name***)`
`rc = GSrce("CLOSE", Item("SAVE", "TRUE"))`

- Color text field in the dialog red → UserExitDynDlgBeforeInitialize
→ does not work if the field has the attribute "STATUS"
→ If VDlg("DLG")="..." Then
 DLGVAR = AddIt("INFO","",cFFVisible+"#COL-clRed")
- Prevent opening the dialog → UserExitDynDlgBeforeInitialize
→ If VDlg("DLG")="..." Then
 DLGVAR=Item("RET","#CANCEL#")
- Perform actions on exiting the (non dynamic) dialog "Log on/off persons":
In „UserExitButtonClick“, the function „@@CANCEL.A_P_AN_AB“ can be caught. With the return value "UE_RET=Item("BTN.RET",-1)" one can prevent that the dialog is closed.
→ Select Case VVar("UE:PAR","BTN.FKT")
 Case "@@CANCEL.A_P_AN_AB"
 If GVars("SYSTEM","KEEP_OPEN")="J" Then
 UE_RET=Item("BTN.RET",-1)
- In the dialog script DynDlgFunctions_XYZ, one can react to escaping the selection list (e.g. scrap reason) via "ESC" :
→ Case "@@LIST_CANCEL"
 OnListCancel

11.4 Script Functions/Variables

You use the user exits previously presented to jump out when a terminal program is running (Delphi→VB). If a user exit is called, you interfere in the terminal program.

The script functions and variables are functions that the terminal program provides for the scripts to read or change information or to execute processes (VB→Delphi).

11.4.1 Notes on script variables

<u>Variable</u>	<u>Read access - Description - Application / Example</u>	<u>Write access - Description - Application / Example</u>
UE_RET	When used in read access, the complete content of the script variable <[n#]UE:RET> is returned. [n#] → When functions are called recursively, a reference index is added in front of them. → read access with < VVar("UE:RET","<DlgID>") > Or all values with < VVar("UE:RET","#GET#ALL#VALUES#") >	When used in write access, the complete content of the script variable <[n#]UE:RET> is deleted if an empty string is assigned. If you assign a DlgID with value (e.g. "TEST=1"), an already existing entry is replaced. If the entry does not exist, it is added to the existing values.
UE_SND	When used in read access, the complete content of the script variable <[n#]UE:SND> is returned. [n#] → When functions are called recursively, a reference index is added in front of them. → read access with < VVar("UE:SND","<DlgID>") > Or all values with < VVar("UE:SND","#GET#ALL#VALUES#") >	When used in write access, the complete content of the script variable <[n#]UE:SND> is deleted if an empty string is assigned. If you assign a DlgID with value (e.g. "TEST=1"), an already existing entry is replaced. If the entry does not exist, it is added to the existing values.
UE_RCV	When used in read access, the complete content of the script variable <[n#]UE:RCV> is returned. [n#] → When functions are called recursively, a reference index is added in front of them. → read access with < VVar("UE:RCV","<DlgID>") > Or all values with < VVar("UE:RCV","#GET#ALL#VALUES#") >	When used in write access, the complete content of the script variable <[n#]UE:RCV> is deleted if an empty string is assigned. If you assign a DlgID with value (e.g. "TEST=1"), an already existing entry is replaced. If the entry does not exist, it is added to the existing values.
DLGVAR	When used in read access, the complete content of the script variable <[n#]DLG.DLG> is returned. [n#] → When functions are called recursively, a reference index is added in front of them. → equals read access to all values with	See <DLGOUT>

	< VDlg („#GET#ALL#VALUES#“) > or < VVar("DLG.DLG", "#GET#ALL#VALUES#") >	
DLGSND	Here, a direct read access is not possible (e.g. MsgBox " DLGSND " + DLGSND) → read access to a value with < VOut(„<DlgID>“) > oder < VVar("DLG.OUT", "<DlgID>") > → read access to all values with < VOut(„#GET#ALL#VALUES#“) > bzw. < VVar("DLG.OUT", "#GET#ALL#VALUES#") > → ### otherwise identical to < DLGOUT >	Special feature delete buffer with < '#DELETE#ALL#VALUES#' > <pre>DLGSND=" #DELETE#ALL#VALUES#" sDlg=scrDeleteItems (sDlg, "EGT:GUT EGT:AUS EGT:GES") DLGSND=sDlg</pre> See <DLGOUT>
DLGOUT	When used in read access, the complete content of the script variable <[n#]DLG.OUT> is returned. [n#] → When functions are called recursively, a reference index is added in front of them. → read access to a value with < VOut(„<DlgID>“) > oder < VVar("DLG.OUT", "<DlgID>") > → read access to all values with < VOut(„#GET#ALL#VALUES#“) > bzw. < VVar("DLG.OUT", "#GET#ALL#VALUES#") >	When used in write access, the complete content of the script variable <[n#]DLG.OUT> is not deleted if an empty string is assigned. If you assign a DlgID with value (e.g. "TEST=1"), an already existing entry is replaced. If the entry does not exist, it is added to the existing values. → Deleting of <DlgID>s using the functions < EraseDlgOut("<DlgID>") > → Deleting the buffer or all <DlgID>s using the call < EraseDlgOut("#ERASE#ALL#DLG.OUT#") >
LSTVARS	Here, a direct read access is not possible (e.g. MsgBox " LSTVARS " + LSTVARS) [n#] → When functions are called recursively, a reference index is added in front of them. For example: LSTVARS = "LST.FILTER=" + "MNR=100 & ZUMAN=J" LSTVARS = "LST.MODE=" + "COLNUMSORT=TRUE DYNAMICFILTER= MNR,MST" → read access to a value with < VVar("LST.MODE", "< COLNUMSORT >") > == "TRUE" → read access to all values with < VVar("LST.FILTER", "#GET#ALL#VALUES#") > == "MNR=100 & ZUMAN=J"	When used in write access, the complete content of the script variable <[n#]LST...> is deleted if an empty string is assigned. During assignment (e.g. LSTVARS = "LST.MODE=" + "COLNUMSORT=TRUE") the previously set value is completely replaced, i.e. there is no DlgID update. → To delete a "single entry", you use the assignment LSTVARS = "LST.MODE=" → Deleting the buffer or all <LST.xyz>s using the call LSTVARS = "" or EraseDlgVars("LST.") Use with - scrFktList - scrFieldChange (DynDlgFieldChange_XYZ) - scrFieldVAGList (DynDlgFieldListe_XYZ) - scrFieldList (DynDlgFieldListe_XYZ)
	→ ##### NOT FOR USE ##### → sollte nicht benutzt werden (bestehende sind Script/Quellcode-Leichen) → ---- TestCallback < scrFktCall > gibt Hinweismeldung aus	→ ##### NOT FOR USE ##### → sollte nicht benutzt werden (bestehende sind Script/Quellcode-Leichen) → ---- TestCallback < scrFktCall > gibt Hinweismeldung aus
DD_SND	When used in read access, the complete content of the script variable <[n#]DD.SND> is returned. [n#] → When functions are called recursively, a reference index is added in front of them.	When used in write access, the complete content of the script variable <[n#]DD.SND> is deleted if an empty string is assigned. If you assign a DlgID with value (e.g. "TEST=1"), an already existing entry is replaced. If the entry

		does not exist, it is added to the existing values.
	→ read access to a value with <code>< VSnd("<DlglD>") > or < VVar("DD.SND","<DlglD>") ></code> → read access to all values with <code>< VSnd("#GET#ALL#VALUES#") > or < VVar("DD.SND","#GET#ALL#VALUES#") ></code>	
DD_RCV	When used in read access, the complete content of the script variable <[n#]UE:RCV> is returned. [n#] → When functions are called recursively, a reference index is added in front of them. → read access to a value with <code>< VRcv("<DlglD>") > or < VVar("DD.RCV","<DlglD>") ></code> → read access to all values with <code>< VRcv("#GET#ALL#VALUES#") > or < VVar("DD.RCV","#GET#ALL#VALUES#") ></code>	When used in write access, the complete content of the script variable <[n#]DD.RCV> is deleted if an empty string is assigned. If you assign a DlglD with value (e.g. "TEST=1"), an already existing entry is replaced. If the entry does not exist, it is added to the existing values.
SCRVARS	→ ##### equals LSTVARS implementation ### Here, a direct read access is not possible (e.g. MsgBox " SCRVARs" + SCRVARs) [n#] → When functions are called recursively, a reference index is added in front of them. For example: SCRVARs = "XXX.FILTER=" + "MNR=100 & ZUMAN=J" SCRVARs = "XXX.MODE=" + "COLNUMSORT=TRUE DYNAMICFILTER= MNR,MST" → read access to a value with <code>< VVar("XXX.MODE","< COLNUMSORT >") > == "TRUE"</code> → read access to all values with <code>< VVar("XXX.FILTER","#GET#ALL#VALUES#") > == "MNR=100 & ZUMAN=J"</code>	When used in write access, the no script variable <[n#]xyz...> is deleted if an empty string is assigned. During assignment (e.g. SCRVARs = "XXX.MODE=" + "COLNUMSORT=TRUE) the previously set value is completely replaced, i.e. there is no DlglD update. → To delete a "single entry", you use the assignment SCRVARs = "XXX.MODE=" → Deleting the buffer or all <XXX.xyz>s with EraseDlglVars(" XXX.")
GLOBALVARs	When used in read access, the complete content of the global variable is returned. (if necessary several rows) For example: GLOBALVARs = "#X#=" + Item("1","1")+ Item("2","2") GLOBALVARs = "#Z#=" + Item("A","A")+ Item("B","B") → read access to a value from a row <code>< GVars("#X#","1") > == "1"</code> → read access to all values with <code>< GVars ("#Z#" ,"") > == "A=A B=B "</code>	When used in write access, the no script variable <[n#]xyz...> is deleted if an empty string is assigned. Setting /savint DD items GLOBALVARs = "#XXX#=" + Item("1","1") GLOBALVARs = "#XXX#=" + Item("2","2") equals GLOBALVARs = "#XXX#=" + Item("1","1")+ Item("2","2") Update GLOBALVARs = "#XXX#=" + Item("2","333") → Deleting a "DDitem" in a "row" with the assignment GLOBALVARs = "#XXX#=" + Item("2","") → Deleting a "row" with the assignment GLOBALVARs = "#XXX#=" + ""

→ **!!! IMPORTANT !!!** Deleting the complete memory with
GLOBALVARS = "#DELETE#ALL#GLOBALVARS#"

Reading system variables / functions

---- Write access is not possible

SYS_IP	IP address of terminal (according to TNR status = variable otherwise via API function)
SYS_DDHEADER	Dialog data header („DAT=09/17/2007 ZEI=48637 USR=2706 SWZ=S USR=2706 ID=4 “)
SYS_USR	Hydra user = Terminal number(TNR) + 2000 → 2001 .. 2999
SYS_TNR	Terminal number(TNR) → 1 .. 999
SYS_DAT	Terminal system date in format („MM/DD/YYYY“) → „09/17/2007“
SYS_ZEI	Terminal system time in format („NNNNN“ = seconds per day) → „43200“
SYS_DT	Terminal system date/time string (with current WIN setting) → „17.09.2007 12:00:00“
SYS_SCRIPT_DEBUG	Terminal script debug window (see section "4.1.2 Script - Debug – Dialog")
SYS_NEW_CNR_FR	Standard - production batch (not applicable for customer batches with reference to MNR or ANR)
SYS_NEW_CNR_WE	Standard - goods receipt batch (not applicable for customer batches with reference to MNR or ANR)
SYS_QUEUE_ITEMS	Number of QUEUE entries (→ ddqueue.dta)
SYS_OFFLINE	Check OFFLINE / ONLINE using < hypdm32.dll > function
SYS_DEMO	Terminal demo mode active
SYS_SCRFCT	Issues current terminal script function
SYS_TNRGRP	Issues terminal group of terminal (0 = „“ otherwise „xxx“)
SYS_PING	ONLINE String-cmd „DLG=SCMD;47 “ for the check if HYMW/HYDDI are running

For the dialog control of dynamic dialog fields

---- Write access is not possible

DLGVAR = AddIt(„ANR“, „“, cFFEnable)

Note: If there are several field attributes, they should be added as follows

cFFReadOnly	DLGVAR = AddIt(„ANR“, „“, cFFEnable+„#F“)
cFFEnable	< ;#RO > = readonly (Field → set attribute READONLY)
cFFDisable	< ;#E > = Enable (enable field)
	< ;#D > = Diasable (disable field)

cFFHide	< ;#H >	= Hide (hide field)	
cFFVisible	< ;#V >	= Visible (show field)	
cFFRequired	< ;#R >	= Required (mandatory field)	
cFFFocus	< ;#F >	= Focused (focus field)	
cFFBarcode	< ;#B >	= Bar code (Field → set attribute BARCODE)	
cFFHideListBtn	< ;#HL >	= Hide-List-Btn (hide list button of an input field)	
other	< ;#N >	= Nullable (Field → set attribute NULL = optional field = without entry)	
	< ;#C >	= Field caption/change text	
DIR_APP	- Application – directory	(e.g. C:\CTWIN\)	---- Write access is not possible
DIR_SPOOL	- Spool - directory	(z.B. C:\CTWIN\SPool\)	
DIR_ETC	- Etc - directory	(e.g. C:\CTWIN\ETC\)	
SystemNecessaryProgramVersion	Is used to check if the minimum version of the terminal software is available (for the standard) The definition is made in the system script file "mpdv-system.scr". e.g. <pre>Public const SystemNecessaryProgramVersion = "7.2.5.77"</pre>		
CustomNecessaryProgramVersion	Is used to check if the minimum version of the terminal software is available (for customers) The definition is made in the customer-specific system script file "system_<project>.scr". e.g. <pre>Public const CustomNecessaryProgramVersion = "7.2.5.77"</pre>		

→→Script functions

Module	Available functions	Description
(UE) = User exit (DLG) = Dialog		→ Registered Delphi function
(UE) + (DLG)	VTnr ("XYZ")	Info from list TKENN.LST
(UE) + (DLG)	VVar (item,id)	Function to read from script VARS – Items
(UE)	VVar ("UE:PAR", "XYZ")	Transfer parameters
(UE)	VVar ("UE:MNR", "XYZ")	Info from list MNR.LST for the current machine

(UE)	VVar("UE:ANR","XYZ")	Info from list ANR.LST for the current order
(UE) + (DLG)	rsIni(ini,Sektion,Key,Default)	Read INI file (with automatic writing of <default> if entry is not available)
(UE) + (DLG)	wsIni(ini,Sektion,Key,Value)	Write INI file
(UE)	scrUECmd(...)	Executing a server command that creates a file and loading of file into terminal. For examples, refer to the descriptions of the user exits.
(UE) + (DLG)	SYS_SCRIPT_DEBUG	Script debug window message (see section "4.2.2 Script - Debug – Dialog")
(UE) + (DLG)	SYS_DT	Date/time stamp
(DLG)	SYS_NEW_CNR_FR	New production batch number the UserExitLosnummer() is not run!
(DLG)	SYS_NEW_CNR_WE	New goods receipt batch number the UserExitLosnummer() is not run!
(DLG)	SYS_NEW_CNR_HU	New packing (handling unit) batch number the UserExitLosnummer() is not run!
(UE) + (DLG)	AddIt(id,value,attribut)	Script function (implemented in mpdv-system.scr) Is used if a field should include not only a value, but also an attribute when a dialog is initialized. (Format: ID=VALUE;ATTR) Attribute: cFFReadOnly, cFFEnable, cFFDisable, cFFHide, cFFRequired, cFFFocus, cFFBarcode, cFFHideListBtn
(UE) + (DLG)	Item(id,value)	Script function (mpdv-system.scr) To generate dialog item in format "ID=VALUE"
(UE) + (DLG)	IncStrDec (int)	Script function (mpdv-system.scr) Decimal incrementing of an integer string (note: up to 15 digits) e.g. IncStrDec("100") -> "101"
(UE) + (DLG)	StrFmtRight(Value,Len,char)	Script function (mpdv-system.scr) Right-aligned formatting of a string with leaders e.g. StrFmtRight("101", 5, "0") -> "00101" StrFmtRight("101", 2, "0") -> "01"
(UE) + (DLG)	MsgPopUp(msg,sec)	Script function (mpdv-system.scr) MsgPopUp "Ticket [XYZ] is printed." , "3"

		If sec = „" dialog must be closed with OK
(UE)	VPz (wert,pzmodus)	Function to identify check digit → up to now only <pzmodus> „EAN-13“ impl.
(DLG)	VPar(id)	Function to read 'DLG.PAR' – Items VPar(id) can only be used in user exit DynDlgInit → Here, on opening a dialog with script initialization < REOPEN = TRUE/FALSE > is set in < DLG.PAR > FALSE = Firstcall TRUE = repeated opening after e.g. DB error
(DLG)	VMnr(id)	Function to read 'DLG.MNR' - Items ('DLG.PAR' takes priority) VMnr(id) is only used in the user exit DynDlgInit
(DLG)	VAnr(id)	Function to read 'DLG.ANR' - Items ('DLG.PAR' takes priority) VAnr(id) is only used in the user exit DynDlgInit
(DLG)	VVAR („*ANR“, id) VVAR („*MNR“, id)	Direct access to “DLG.ANR” or „DLG.MNR“ → „DLG.PAR“ is bypassed!!
(DLG)	VDlg(id)	All configured fields of a dialog can be read. For example: Badge number=VDlg(„KNR“)
(UE) + (DLG)	VDat(offset)	Function to read the current date in format „MM/DD/YYYY“ with <offset> = „0“ = today with <offset> = „-1“ = yesterday
(UE) + (DLG)	VZei(offset)	Function to read the current time in format „NNNNN“ = seconds since midnight with <offset> = „0“ = now with <offset> = „-30“ = now – 30 seconds
(DLG)	GStore(func,filter)	These functions and the three subsequent functions enable an easy access to data in BAPI files. Note on the selection of grid rows (as of V# 7.2.4.76) Using the instruction DLGVAR = Item("GRD.ROW", "<value>") in a “DynDlg” user exit, you can make a selection in the current dialog grid. Possible values are: - “FIRST” first grid row is selected - “LAST” last grid row is selected

		- "PREV" Current display position – 1 (if display position > 1) - "NEXT" Current display position + 1 (if display position < "LAST") with the return value of function GStore(..) you can select as follows: - "0" .. "X" Position index – without taking into account any sorting - "#" + "0" .. "X" Display position – taking into account a possible sorting
(DLG)	VStore(id)	Function for Store (-> GStore(..)) to read in 'DLG.GRD'
(DLG)	SStore(id, value)	Function for Store (-> GStore(..)) to write in 'DLG.GRD'
(DLG)	AStore(id)	Function for Store (-> GStore(..)) to add up (write) in 'DLG.GRD'
(DLG)	VOut(id)	Function to read the dialog data transferred in user exit DynDlgInit
(DLG)	VSnd(id)	Function to read the dialog data sent in the user exits DynDlgAfterSend and UserExitDynDlgAfterSend
(DLG)	VRcv(id)	Function to read the reply returned from the server in the user exits DynDlgAfterSend and UserExitDynDlgAfterSend
(DLG)	EraseDlgOut(id)	Function to delete individual IDs from the dialog string ('DLG.OUT'). Usually in the user exits DynDlgBeforeSend and UserExitDynDlgBeforeSend
(DLG)	EraseDlgVars(id)	Function to delete SCR-VARS -> "LST.", "FKT.", "DD."
(DLG)	scrMsgBox(msg) scrMsgBox("3^Halle") scrMsgBox("3 vModal Caption^Halle")	Display of an info window 3^ → Display for 3 seconds; message closes automatically Caption: text displayed in the title bar of the message vModal: the dialog is displayed as a modal dialog window The simple message box (without parameters) is displayed in a modal window. This means that the script processing stops and waits until OK is pressed. In case of a message box that is automatically closed after x seconds, the script processing is immediately continued. If the parameter "vModal" is additionally set, the script processing waits also in case of automatically closed messages until the message is confirmed or is automatically closed.
(DLG)	DlgJaNein(caption,msg)	Display of a query with the options Yes/No Example: <pre>Res=DlgJaNein("delete advance login","really delete selected batch?") If Res="#JA#" Then DeleteVLos(sMNR) End If</pre>
(DLG)	DlgJaNeinAbbruch(caption,msg)	Display of a query with the options Yes/No/Cancel (Check with "#JA#", "#NEIN#", "#CANCEL#")

(DLG)	scrFieldChange	You use this function to link data and text fields. For example: If you enter a status, the status text can immediately be updated. For an example, refer to the description of the user exit DynDlgFieldChange.
(DLG)	scrFieldList	You use this function to implement a list selection in the user exit DynDlgFieldListe. „LST.MODE=.. FORCEAREOPEN=TRUE ..“ (only with function < scrFieldList >) <i>Effect: Data is loaded from the hard disk. This way, it is possible to display a file that is already in memory after a server comparison.</i> For examples, refer to the user exit description.
(DLG)	scrFieldVAGList	[f_vorlst.pas] function for DynDlg -> FieldVAGListe ➔ LSTVARS <LST.xxx> are used ➔ Display of the order list selection (optional with loading from server)
(DLG)	scrFktList	[f_liste.pas] function for DynDlg -> FktList ➔ LSTVARS <LST.xxx> are used ➔ Function like list selection without dialog (optional with loading from server) ➔ Transfer of data in <[n#]DLG.OUT> if <LST.FILTER=xyz> + <LST.RET=xyz> is set.
(DLG)	scrDDSnd	Function for DynDlg -> scrDDSnd ➔ DD_SND is used Note: with <EraseDlgVars("DD.")> the old data buffer <DD_SND> can be deleted ➔ new send parameter <PROCESSDLGEVENT=TRUE> execution via < ProcessDlgEventSend > with ScriptlokalUpdate ➔ new send parameter <\$TNR.KEEP_DATETIME=ON> transferred time stamp (DAT/ZEI) is preserved ➔ new send parameter <*LST_RELOAD=OFF> Reload request of server is ignored Example: DD_SND="DLG=A_AN MNR=1234 ANR=123450020"

		scrDDsNd
(DLG)	scrPCCValues (value)	Function for DynDlg -> PCC-Values ➔ transfer of the <value> to „pccdll.dll“ Return of requested values in (1) customer system script < UserExitPccDllToTerminal > (2) In addition, it is tried to send all requested "V:..." values in an opened dynamic dialog as bar code. The values transferred can be processed in the DynDlgUserExit < DynDlgFieldExit_ XYZ > (XYZ=DynDlgKennung). (➔ If VDlg("FLD.MOD") = "BARCODE" Then)
(DLG)	scrBatchPrint	Function for DynDlg -> PRN-DD-List ➔ LSTVARS <PRN.xxx> are used No converting of print file: LSTVARS = "PRN.DLG=" + "FILECONVERT=FALSE "
(DLG)	scrUECmd(string)	Function to create a file in the user exit
(DLG)	scrExecDynDlg (dlg, ret, values)	Function for DynDlg -> calls (without script) <ret> = RETURN if DLG does not exist
(UE) + (DLG)	rsCfg (Sektion, Key, Value)	Function to read (string) from HyTnrCfg.Ini
(UE) + (DLG)	<pre>// General file functions scrFileExists(file) scrFileDelete(file) scrFileCopy(file,newfile) scrFileRename(file,newfile)</pre>	Function file exists (OK ➔ „0“) Function delete file (OK ➔ „0“) Function copy file (OK ➔ „0“) Function rename file (OK ➔ „0“)
(DLG)	<pre>// Allgemeine DD-Listen GSrce (sFct,sParam) VSrce (sID) SSrce (sID,sValue) ASrce (sID,sValue)</pre>	Function to access DD list -> see tips in section 4.2 Function for read access Function for write (update) access Function for write (add) access
(DLG)	// General Windows functions	

	scrAppRuns (sCaption) scrAppShow (sParam) scrAppExec (sExeCall, sParams)	Check of running applications Display of running applications Function to start applications sParams = "WINSTYLE=SW_NORMAL[SW_HIDE,SW_MINIMIZE,SW_SHOW]"
(UE+DLG)	scrStatusBarMsg (sMsg, sMode, sSec)	Output of status bar messages
(DLG)	scrLog (sLine)	Logging function
(DLG)	scrReadRemoteFile (remote, local, params) scrReadCfgFile (ssLstCmd, ssLstFile)	Function to read a file from the server Function to create (DLG=LIST;..) and read a file from the server If LOAD_EMPTY=Y is added as parameter in ssLstCmd, then also an empty list is loaded that only includes a header.
(UE+DLG)	// General DD list functions scrDeleteItemsInDlgLstFileWithFilter (sFileName, sFilter, sParam) scrMergeDlgLstFileIntoFile (NewItemFile, SourceFile) scrQuickSearch (sFilename, sFilter) scrClearDlgLstFile (sFilename) scrCreateEmptyFile (sFilename) scrMergeDataIntoDlgLstFile (asData, asDlgLstFile, "TRUE") scrGetDlgLstLine (sFilename, sLine)	Deletes entries in a DD list file that match the filtering Merges a DD list file with a target file Searches in a DD list file for the first entry that matches the filtering Deletes all rows in a DD list file except the header Creates an empty file -> size 0 byte Creates a DD list file or adds a data row with all IDs included in the DD list header → TRUE forces a file reload (e.g. ANR.LST) otherwise only file operation Reads any row in a DD list file (sLine="1"/ „2“.. or „FIRST“/„LAST“)
(DLG)	// General conversion functions scrStr2Real (value):real scrReal2Str (value:real):string scrDDItem (sID, Values):string scrStrReplace (Value, OldPattern, NewPattern):string scrEraseDDItem (sID, Values):string scrReplaceDDItem (sID, sItem, Values):string scrReplaceAllDDKennung (sVor, sNach, sValues, sNoCnvIDs)	Converts a string 123.256 into a real value Converts a real 123.256 into a string Identifies an item from a DD string Replaces <old strings> by <new strings> in a string Deletes a DD item from a DD string Replaces a DD item <sID> by <sItem> in a DD string <values> Replaces all DD items of <sValues> with prefix <sVor> and suffix <sNach> and leaves out <sNoCnvIDs> e.g. scrReplaceAllDDKennung ("V.", ".N", "DLG=XX MNR=1 X=3 ID=5 ", "DLG ID")

	scrGetPart (sString,sSeparator,sIndex) scrPosStr (ssSubString,ssString)	result string ➔=> "DLG=XX V.MNR.N=1 V.X.N=3 ID=5 " Returns a substring of <sString> with separator <sSeparator> with index <sIndex> scrGetPart ("R W Q" , " " , " 1") ➔ "R" scrGetPart ("R W Q" , " " , " 3") ➔ "Q" scrGetPart ("R W=T Q=Z" , "=" , " 2") ➔ " T Q" Function if substring <ssSubString> in String <ssString> is included. scrPosStr ("DLG=" , "XXX=100 DLG=12 ..") ➔ "DLG=12 .."
(DLG)	scrDDSnToIPatPort (Data,IP,Port,TimeOut)	Sends data <Data> to a host <IP> on port <Port> with timeout
(DLG)	scrDoLosProperties (sAnr)	Requests the batch attributes for the order and returns the entered values as DD string. ➔ If "DLG->RET=ESC" has been returned, the input dialog has been cancelled
(DLG)	scrLosnummer (sParam,sMnr,sAnr)	Function for the customer-specific generation of batch numbers
(DLG)	scrStoreUpdate (sMode,sID,sValue)	Function for local update <ANR.LST> + <MNR.LST> in user exit <UserExitLocalMnrAnrUpdate> Explanation <sMode> = "READ" <sID> = "XYZ" ➔ reads value from DD list <sMode> = "ADD" <sID> = "XYZ" <sValue> = "10" ➔ adds <10> to value in DD list <sMode> = "UPDATE" <sID> = "XYZ" <sValue> = "ABC" ➔ updates value in DD list to <ABC> For more details, refer to user exit <UserExitLocalMnrAnrUpdate>
(DLG)	scrTranslate (Text,Data)	Function to translate texts ➔ Text "The password of the person [<PNR>]<n>expires in <PWD:VALIDTG>. day(s)." ➔ Data "RET=0 KT= LT= INFO=3645 PNR=44444444 PWD:VALIDTG=1 ID=152 " Notes: The placeholders <XYZ> are replaced from "Data". <n> = line feed + <t> = tabulator
(DLG)	scrWriteRemoteFile (local,remote)	Function to write a local file for the server
(DLG)	scrProcessQuickReportPrinterForDialog (dlg,data)	Function to print a configured label
(DLG)	scrProcessQuickReportPrinterForDialogEx (dlg,data,ret)	➔ (ab CTWIN V# 7.2.5.77 / CTAIP V# 2.0.2.20) Function to print a configured label with transfer Ret
(DLG)	scrExecuteQuickReportPrinter (params,file)	Function to print a file (RPB format) using < qr_print.exe >

(UE) + (DLG)	CDateCompare(date1,date2,mode)	Function to compare date strings in internal format "MM/DD/YYYY" ➔ Example see <mpdv-system.scr>
(UE) + (DLG)	// General file/directory functions vbsFolderExists(sFolder) vbsFolderCreate(sFolder) vbsFileExists(sFile) vbsCreateFolderTree(sFolder) vbsValidateFolder(sFolder) scrWriteDataIntoFile(asData,asFile)	➔ see <mpdv-system.scr> - checks if directory exists -> OK = „0“ - creates directory -> OK = „0“ (exists) or „1“ (has been created) - checks if file exists -> OK = "0" - creates a directory tree - identify directory string with closing "\" and placeholders <DIR_APP> + <DIR_SPOOL> . Writes (added) data string in a file or creates the file if it does not exist.
(UE) + (DLG)	scrAddAction(sAction,Param,Data)	Saves an action that is processed in the main loop of the application (CTWIN). e.g. scrAddAction("mtaDIALOG","DLG=AUTO:CA_WL_RS ..","MNR=100 ..") ➔ saves an action to automatically open a dialog Further notes: - with Data = Item("ANR+MNR","RELOADED.WITH.VALUES") the start dates ANR/MNR row are identified using the parameters In the example above, MNR=100 is used irrespective of the currently selected machine row to start the script dialog (!!! This function is only possible with script dialogs !!!) - scrAddAction("#STATE#","#BASIC#","") returns the number of actions and the execution status - „0 0“ = no action / no action / dialog open - „1 1“ = 1 action / Action/Dialog open IMPORTANT: You can only transfer data to the dialog if USEREXITButtonClick has been implemented in the script. If necessary, as empty body.
(UE) + (DLG)	GVars(id,item) Save data in script with GLOBALVARS = „ABC=XYZ=1 ...“	Global buffer of variables // each time a dialog is opened via script // the call parameters „#{DLG-ID}#PAR#“ are saved. // also „#{DLG-ID}#ANRR#“ , „#{DLG-ID}#MNR#“, .. e.g. xRowID = GVars("#CE_WL_RF#PAR#","ID#ROW")
(DLG)	scrELosAbmeldVorschlag(ssDlgData,ssEMatFile)	MPL function for script-controlled call of the input batches marked "Log off from server" in the input material list
(DLG)	scrELOSEChecking(ssDlgData, ssEMatFile)	MPL function to check if the input material list is complete

(DLG)	scrIndexOfExpr(iIndex, sFileName, sExpression)	returns an index for sExpression. See function IndexOfExpr of StoreSource. sFilename must be a complete path + file name. If sExpression is empty, -1 is returned. (the function has no effect on an existing grid, if available)
(DLG)	scrFieldByNameIdx(iIndex, sFileName, sFieldName)	returns a string with the field content of sFieldName. See function FieldByNameIdx of StoreSource. sFilename must be a complete path + file name. (the function has no effect on an existing grid, if available)
(DLG)	scrDDSnWOErr	sends the BAPI string from DD_SND like scrDDSn, but does not automatically issue an error that might be returned. ➔ new send parameter <PROCESSDLGEVENT=TRUE> execution via <ProcessDlgEventSend> with ScriptokalUpdate
(UE)	scrEvaluateDuration(sDatB, sZeiB)	Creation of a continuous string in the configured format to be displayed in the OP info. The parameters sDatB and sZeiB are passed in MPDV format (MM/DD/YYYY, Sec. since midnight). The duration since this point in time is calculated.
(UE)	scrFormatDuration(sSeconds)	Formats a duration in seconds and uses the specified format (industrial time unit, if required) for the display in the OP info
(UE)	scrFormatTimeStamp(sDat, sZei)	Formats a time stamp in MPDV format for the display in the OP info (hh:mm dd.mm.yyyy)
(UE) + (DLG)	scrUrlDownload(scheme,user,password,host,port,url_path,loc_path,prot_path:AnsiString):integer; iRes=scrUrlDownload("hydra","hydadm","hydadm","win2003-3","10403", "\\hydra724\dncfiles\H10007410750.pdf", ".\spool\Temp.pdf", ".\spool\prot_ev.txt")	Load files via URLDownload
	Ret=scrUrlDownload2(Path,FileName)	Simplified call of URLDownload: Path: using this string, the download parameters are read from „Paths.lst“
(UE) + (DLG)	scrUrlUpload(scheme,user,password,host,port,url_path,loc_path,prot_path:AnsiString):integer;	Copy files via URLUpload (same syntax as scrURLDownload)
(UE) + (DLG)	scrStarteProg(ExeFileName)	Starting a program (file names with complete path as parameter)
(UE) + (DLG)	scrDynDlgConfig(asDLG:AnsiString)	Checking if a dynamic dialog is configured Ex.: If scrDynDlgConfig(„A AB“)=1 Then ScrMsgBox(„A AB is configured“)
(UE) + (DLG)	scrSplitOrder(sAuftrag)	Returns all individual IDs for an order number (ANR, AUNR, AFOLG, AGNR, UAGNR, SPLNR) scrSplitOrder(„TK00000000010“) ➔ ANR=TK00000000010 AUNR=TK0000000 AFOLG= AGNR=0010 UAGNR= SPLNR=
(UE) + (DLG)	scrDateTime(Mode:Ansistring):double	Function to read the TickCount/Now ➔ Result = DOUBLE

		- "TC" provides the current time in seconds since the program has been started. - "TCMS" = provides the current time in milliseconds since the program has been started. - "TCSYS" =provides the current time in seconds since the computer has been started. - "TCSYSMS" =provides the current time in milliseconds since the computer has been started. - "DTMS" =Time in milliseconds since 30 December 1899 - "", "DT" =Time in seconds since 30 December 1899
(UE) + (DLG)	scrMSSPort(channel:integer;Action:string):integer	Direct access to MSS channels: Ret=scrMSSPort(1,"SET") \ set channel 1 result:1 Ret=scrMSSPort(1,"RESET") \ reset channel 1 result:0 Ret=scrMSSPort(1,"GET") \ read channel 1 result:1/0 ;set channel 1 for 2sec: Ret=scrMSSPort(1,"SET 2000")
(UE) + (DLG)	scrGetInfo(Fkt,Param:string):string;	extensible query function
	scrGetInfo("HasAutomaticRecording","MASCH100")= "J"/"N"	Machine with automatic recording?
	scrGetInfo("GetPLock","MASCH100")= "J"/"N"	Prod. lock active?
	scrGetInfo("HasShift","MASCH100")= "J"/"N"	Machine has shift?
	sAnrLst=sAnrLst=scrGetInfo("GetAllOrdersOfMachine",<machine>)	Returns orders logged on to machine separated by commas.
	sAnrLst=scrGetInfo("GetParallelOrders",<order>)	Returns all orders logged on in parallel to the same machine including the order transferred (separated by comma).
	scrGetInfo("InputDialog","<Caption> <Promt> <Default> [<Format>]")	Opens a dialog to enter a string.
	scrGetInfo("CheckPParam","MASCH100")="Y"/"N"	Call of the production parameter check (this must be activated!)
	scrGetInfo("GetDefaultPerson","MNR=4711")	Specified person to get dialogs (only if HoldPersonInfo=on is set)

	scrGetInfo("GetAGFile","TYP=<TYP> ANR=<ANR>[MOD=<MOD> REF=<REF>][LISTFILE=<ListFileName>]")	Reads OP records from server scrGetInfo("GetAGFile","TYP=AI ANR=G8TC1610HN0010 MOD=M REF=TYP LISTFILE=qsinfo.lst") MOD=M -> Several files are generated. If the field <REF> in the list read is changed, a new file is started. The files are named File<TYP>_<n>.txt (->FileAI_1.txt, FileAI_2.txt...) Return value: F1=<FileName1>,<REF1> F2=<FileName2>,<REF2> Example: F1=FileAI_1.txt,Q F2=FileAI_2.txt,R F3=FileAI_3.txt,V ListFileName: Name of list file that is read by the server (Default:agscrinfo.lst)
	scrGetInfo("CheckPPParam","MASCH100")="Y"/"N"	Checking the production parameters of a machine
	scrGetInfo("GetDefaultPerson","MNR=4711")	Read person according to HoldPersonInfo
	scrGetInfo("GetDlgBufferValue","DLG=@ACTIVE AKRO=ANR")	Read value from dialog (→set values with scrSetData("SetField"..) Note: If Windows is locked ([WIN]-L), the data cannot be read any more
	scrGetInfo("GetLayoutFile","")	Layout file currently used by ctwin (Default: ctwinlay.ini)
	scrGetInfo("GetPathData","PATH="+sPath+" FILE="sLoadDateiname+" EXT="")	Query of path data → SCHEME=FTP USR=TK PWD=123...
	scrGetInfo("GetSelected","")	Returns selected machine and order → MNR=.. ANR=..
	scrGetInfo("GetProductionLock","4711")	Query production lock of a machine → „J"/„N"
	scrGetInfo("mdeGetMid2MaschinenNr","MIDX=1")	identify the machine number for the index from the MDE Result = machine number
	scrGetInfo("mdeGetData","MNR=xxxx")	Read MDE info Result=Dialog string (anistring) SKSTA=SCH_NORM oder SCH_BEG,SCH_END,NO_SCH Schichtzähler , MST, IZY, TLG, IMPFAKT, SZY, PSPERRE, PKENN
	scrGetInfo("GetCounterBookingType","4711")	Read posting type of a machine → J/G/A/N
(DLG)	scrGetInfo("GetGridLineWithFilter","ATK=12345")	Reading a row of the dialog grid that matches the filter function
(DLG)	scrGetInfo("GetGridData","-1") asGrid=scrGetInfo("GetGridData","DLG=@FIL=DLG=A_AN LINE=-1")	Read active row of the dialog grid After dialog, the filters are added
(UE) + (DLG)	scrGetInfo("GetBatchMode","MNR=4711")	Read batch mode of a machine → lmNormal,lmLos,lmDurchlauflos,lmChargenVerw,lmUser1,lmUnknown

(DLG)	scrGetInfo("IsDlgFieldVisible", "DLG=@ACTIVE AKRO=MNR")	Query if a field is visible in the dialog → Y/N
	scrGetInfo("GetDDlgEntries", "TNR=100 DLG=GEB_DRU TGRP=42")	Read all configured dialog fields of a dialog
	scrGetInfo("HydraPath", "")	Variable for access to server files
	scrGetInfo("GetPnrFileExtension", "")	Cyclic generation of a file ending (000...999) that starts on restart
	scrGetInfo("mdeGetMid2MaschinenNr", "MIDX=01")	Identify the machine number for the index from the MDE Result = machine number
	scrGetInfo("mdeGetData", "MNR=xxxx")	scrGetInfo("mdeGetData", "MNR=xxxx")
	scrGetInfo("GetOrderData", "ANR=TK0000000010")	Reads order row from anr.lst, vlist.lst or from server (nanr.lst)
	scrGetInfo("GetMessageMode", "")	Query of specific settings (TAKA) →PopupToStatusBar=ON/OFF ShortPopupMode= ON/OFF PopupTime= StatusBarTime=
	scrGetInfo("GetDlgValue", "DLG=@ACTIVE AKRO=ANR") scrGetInfo("GetDlgValue", "DLG=@ACTIVE AKRO=[AUTOFILTER]")	Query value from active dialog Read content of the AUTOFILTER field Further info: [DLG], [DLGCFG], [BUFFER], [DLGCAPTION] (only AIP)
	scrGetInfo("GetDlgBuffer", "DLG=@FIL=DLG=M_MST")	Read all values from a dialog
	scrGetInfo("GetButtonCaption", "DLG=@ACTIVE FKT=FKT=WG1")	Read button text of a dynamic dialog (reference via FKT)
	scrGetInfo("GetTimeDiff", "D1=11/04/2008 T1=48744 D2=11/04/2008 T2=52000")	Specification of the number of seconds between two time stamps
	scrGetInfo("GetCodePageItem", "")	Specify active code page Result: CLIENT.CP=windows-1252
	scrGetInfo("GetAGFile", "TYP=AI ANR="+sAuftrag+" MOD=T LISTFILE="+sFileName)	Read record type "AI" for an order from server
	scrGetInfo("GetTimeStamp", "")	ReadTime in format yyyyymmddhhnnss
	scrGetInfo("GetDNCInterface", "")	→ „PCC“ or „MWP“
	scrGetInfo("GetFocusedDomain", "")	Only ctaip: when a script dialog is initialized, it can be used to query the area that made the call (return value: MNR, ANR, LIST3)

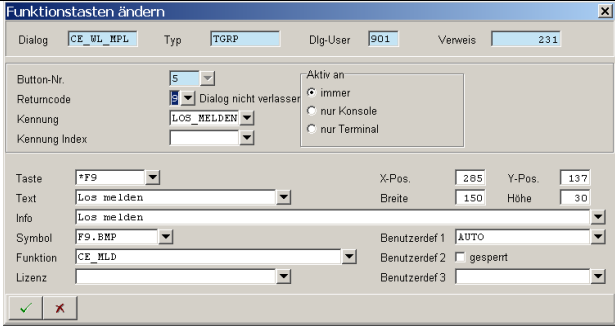
	asProp=scrGetInfo("GetFieldProperties","DLG=@ACTIVE AKRO=EGR:NETTO")	Reads the properties of a field from an active dynamic dialog: ENABLED=TRUE/FALSE COLOR=.. [CAPTION=..] bei TLabel TEXT=..
	scrGetInfo("GetMachineData","MNR=4711")	Reads the complete row from the machine list → MNR=4711 MGRP=122 MBEZK=..
	scrGetInfo("GetMachineData","MNR=4711 AKRO=MST")	Reads a specific value from the machine list
	scrGetInfo("GetSelectedGridData","LIST3")	Reads the complete data row of the selected row from the local lists. MNR - selected machine ANR - selected order LIST3 - Third list (material, person, resource..)
	scrGetInfo("GetMWPFile","")	Returns path and file name of the file written to the MWP2 interface.
	scrGetInfo("GetDialogHandle","")	Returns handle of the opened dynamic dialog
	scrGetInfo("GetIniButtonFunctionIDs","")	Returns all FunctionIDs of the current/of all button bar(s)
	scrGetInfo("IniSectionExists","INIFILE=ctwinlay.123 SECTION=CE-Scan-Liste")	Check if a section in the INI file is included ("0"→Yes / "-1"→No)
	scrGetInfo("GetGridFileName","DLG=@ACTIVE PATH=FALSE")	Query of file name of dialog grid (with/without path)
	asDat=scrGetInfo("ValidateYield",asDat)	Using the current status of the production lock, the currently configured logic for "Update during prod. lock" is applied to the transferred data string. In the data string returned, the yield is changed to scrap or the yield is deleted.
	scrGetInfo("ScreenSize","")	Returns the screen size in pixel: Ex.: WIDTH=1920 HEIGHT=1080
	scrGetInfo("GetForegroundWindow","")	Returns the window handle of the active application
	scrGetInfo("GetWindowInfo","HANDLE=12345")	Returns information on the window including the handle transferred. Ex.: CLASS=Tmain TEXT=Hydra CTAIP V2.0.2.15 Server:10.10.60.91:4 TNR:1 [order/machine list] PARENT.HANDLE=660336 LEFT=448 TOP=252 WIDTH=1024 HEIGHT=576
	aDat =scrGetInfo("GetSelectedAGInfoGridRow","")	Returns the currently selected GRID row from the active dialog in the OP info.
	scrGetInfo("GetNextWindow",sHandle)	Using the Windows function of the same name, returns the handles of some preceding and succeeding windows: NEXT=267338 NEXT2=332820 NEXT3=332648 NEXT4=594798 NEXT5=267206 PREV=2167356 PREV2=463374 PREV3=398006 PREV4=267156 PREV5=988846 Using this function, also a search through all active windows in the

		system is possible.
	scrGetInfo("GetSelectedGridData","PNR")	CTWIN: read selected row from the list of persons in the input dialog
	scrGetInfo("GetButtonFile","") scrGetInfo("GetButtonFile","ExtractFileName")	Query currently used button configuration file (e.g. ctwinbut.ini, ctwinbut.902,...,ctaipbut.ini)
	scrGetInfo("GetHttpLink","PATH=HTTP1 FILE=produkte.htm")	Builds an HTTP link using the transferred file name and the path configuration from paths.lst
	scrGetInfo("GetOPInfoData","")	AIP: Read data of the OP displayed in the OP info
	scrGetInfo("GetNetRuntime","MNR=4711 ANR=123456780010 GETYIELD=1")	Read net run time of the operation. This is the time since logon of the OP when the production lock was green. The value is only available if the terminal has not been restarted since logon. This time is also used to calculate the target quantity since logon. Return string: DATETIME: decimal value in days SEC: time in seconds RESTART=1: Since logon, the terminal has been restarted YIELD: target quantity since logon (if GETYIELD=1)
	scrGetInfo("OperationProgress","ANR=12345678 MNR=0815")	Query of order progress Return: SGR:GUTP=10500.000000 EGR:GUTP=200.000000 EGS:GUT=200.000000 COMPLETION=1.9 DEVIATION=-92 SGS:GUT=2561 STATUS=VALID MONITORED=Y SGR:GUTP→target quantity EGR:GUTP→yield EGS:GUT→yield since logon COMPLETION→completion in % (target quantity - yield) DEVIATION→deviation in % (target quantity since logon - yield since logon) SGS:GUT→target quantity since logon STATUS=VALID→plausibility check MONITORED=Y→order has been logged on after restart of terminal
	scrGetInfo("GetForms","") scrGetInfo("GetForms","TOP")	Output of all windows of the terminal program TOP: only the active window
	scrGetInfo("GetScriptStack","")	Returns stack of user exits - to analyze recursive calls

(UE) + (DLG)	scrSetData (Fkt, Param:string):string	Extensible function to set data
	scrSetData ("SetFocusToField", "DLG=@ACTIVE AKRO=EGR:PRB RED=1")	Set focus in the active dialog to the field "EGR:PRB" and color the field red
	scrSetData ("QuantityToMDE", "MNR=Maschl00 GUT=10 AUS=5")	Transfer relative yield and scrap to MDE - is required with uploads of part quantities so that the correct quantities are displayed for the machine in the machine overview.
	scrSetData ("UpdateView", "")	Update machine overview (e.g. after quantity update)
	scrSetData ("QuantityToLocalLists", "DLG=MENGE MNR=TKAG G11 AGR:GUT=1 AGR:AUS=-1 AGR:LEN=0 AGR:HUB=0")	Locally set quantities in the order and machine list (relative)
	scrSetData ("PressButton", "DLG=@ACTIVE RCODE=0")	Press keys of dialog from script ' RCODE=0->OK 1->Cancel
	scrSetData ("PressButton", "DLG=@ACTIVE AKRO=NEXT")	Press keys of dialog from script → Selection of button via ID
	scrSetData ("SetField", "DLG=@ACTIVE AKRO=CNR VALUE=100000000")	Fill fields in dialog with values. Using "VALUE=", can also be used to empty the field
	rc=scrSetData ("SetField", "DLG=@ACTIVE AKRO=CNR COLOR=clLime ENABLE=1")	Highlights an input field (here light green)
	rc=scrSetData ("SetField", "DLG=@ACTIVE AKRO=CNR FONT.COLOR=R=clLime CAPTION.FONT.COLOR=clLime")	FONT.COLOR: Font color of the field content CAPTION.FONT.COLOR: Color of description (Caption) Note: This does not work with all field types!
	scrSetData ("SetFocusToButton", "DLG=@ACTIVE RCODE=0")	Set focus on a key ' RCODE=0->OK 1->Cancel
	scrSetData ("SetButtonVisible", "DLG=@ACTIVE FKT=OK ACTION=HIDE")	Show/hide key , ACTION=SHOW/HIDE/READ/TGL → ACTION=DISABLE/ENABLE
	scrSetData ("ButtonClick", "CA_WL")	Trigger pressing a key in ctwin (the acronym transferred matches the entry in ctwinbut.ini)
	rc=scrSetData ("DelayedButtonClick", "CA_WL")	Only the main timer triggers pressing the key (advantage: the script processing does not stop at this point)
	rc=scrSetData ("ProcessMsg", "MSG="+sMsg+" INIT=80 TIME="+CStr(iWaitTime))	Messages with splash screen MSG: Text of message INIT: Window is reopened with specified height TIME>0: Window automatically closes after [ms] TIME<0: Close window (then without MSG) TIME=0: Window remains open
	rc=scrSetData ("ProtIntoFile", "PROTFILE="+sProtDatei+" MSG="+sMessage)	Logging of messages in any file in the directory spool (time stamp is automatically put in front).
	rc=scrSetData ("SelectData", "MNR=4711 ANR=12345")	Select machine and/or order on the terminal
	rc=scrSetData ("SetProductionLock", "MNR=4711 ACTIVE=1 ")	Set/release production lock of a machine

rc=scrSetData("DelayedButtonClick","CA_WL")	Delayed triggering of a button click. The button is clicked when the timer event is released in the main timer. Advantage: If the button click opens a dialog, the processing continues in the background.
rc=scrSetData("DelayedButtonClick","CA_WL FORCEDIALOG=ON")	The dialog is repeated until "OK" is pressed.
rc=scrSetData("UpdateGrid","DLG=@ACTIVE GRD.ACCESS=TRUE DLG.GRID=REOPEN")	Read grid of a dynamic dialog
rc=scrSetData("SetMaterial","DLG=@ACTIVE")	Function to support MPL demo mode: The file lnr.lst is set to the material selected as input material in the dialog.
rc=scrSetData("AddListFileLine","FILE=mat.lst CNR=123 ZLO=...")	Entry of a new row in list file
rc=scrSetData("DelayedButtonClick","DLG=@ACTIVE RCODE=0")	Press key with RCODE=0 in the dialog with delay
rc=scrSetData("DelayedButtonClick","DLG=@ACTIVE FKT=FKT=SEND")	Trigger specified function of the dialog with delay
rc=scrSetData("DelayedButtonClick","CA_WL CLOSE_ALL_DLG=ON ")	Before triggering the dialog, all opened dynamic dialogs are closed
rc=scrSetData("DelayedButtonClick","DLG=@AINFO BTN.FKT=A I_CLOSE")	Press key in the OP info (also for DLG=@MINFO)
rc=scrSetData("DelayedDialogFunction","DLG=@ACTIVE FLD.COL=CNR,clRed")	Color field with delay. Advantage: If the field is directly colored, it could be overwritten during event processing. → does not work if field has the attribute "STATUS"
rc=scrSetData("DelayedDialogFunction","DLG=@ACTIVE FLD.VAL=CNR,100")	Set field value with delay.
rc=scrSetData("DelayedDialogFunction","DLG=@FIL=DLG=CA_WL_MPL DLG.FOCUSED.FLD=ATTR:10")	Focus field with delay.
rc=scrSetData("DelayedDialogFunction","DLG=@ACTIVE SCFKT=DNC_REOPEN_GRID")	Delayed call of a script function (→scrSetData("ExecFunction",...))
rc=scrSetData("LocalUpdate","TYP=ANR,MNR DLG=A_TR MNR=4711 ANR=TK111110010 EGR:GUT=5 EGR:AUS=1")	The tables (ANR, MNR) specified under TYPE are locally updated using the event transferred.
rc=scrSetData("LocalUpdate","TYP=MDE DLG=M_TLG MNR=4711 TLG=4")	Locally pass target data (M_SZY, M_TLG) to the MDE
rc=scrSetData("LocalUpdate","TYP=MDE DLG=M_MST MNR=4711 MST=477 DAT=11/30/2010 ZEI=46800")	Pass status to the MDE
rc=scrSetData("LocalUpdate","TYP=FAST DLG=TLGMENGE MNR=4711 ANR#1=TK0000010 AGR:GUT#1=2 AGR:AUS#1=1 ANR#2=..")	Add data to the lists using the fast update function
rc=scrSetData("MessageMode","PopupToStatusBar=ON PopupTi	Configuration of different modes for the display of errors PopupToStatusBar: redirects error messages to the status bar

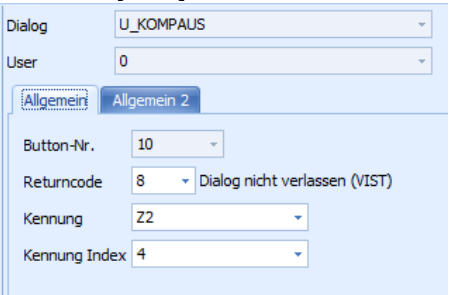
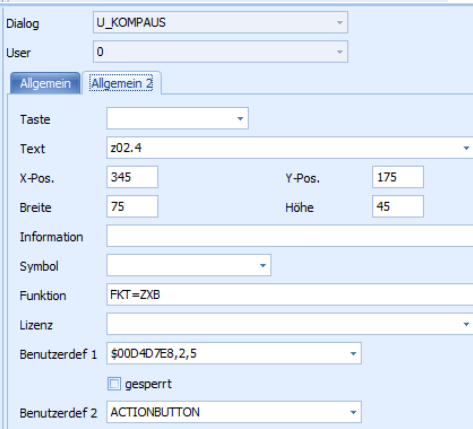
me=2000 StatusBarTime=2000")	PopupTime: display time of pop-up window in shorter pop-up mode StatusBarTime: display time of messages redirected to the status bar
rc=scrSetData("MessageMode","RESET")	Resetting all configurations of the MessageMode
rc=scrSetData("MessageMode","ShortPopupMode=ON")	Activation/deactivation of the shorter pop-up mode
rc=scrSetData("SetFieldAttribut","DLG=@ACTIVE AKRO=ATK ATTR=READONLY VALUE=FALSE")	Setting attributes of dialog fields
rc=scrSetData("SetFieldAttribut","DLG=@ACTIVE AKRO=CNR ATTR=ENABLE VALUE=TRUE")	Access to the property "enabled" of a dialog field
rc=scrSetData("BusChannelsToHYDRA","DLG=NC_AB MNR=4711 ANR=12345")	Transfers a record from the script to ctwin that is processed in the same way as if the record had come from the pccdll. Also this record then runs through the "UserExitPccDllToTerminal".
rc=scrSetData("TriggerLoopStop","MODE=ONETIME")	Once execute UserExitMainInitLoopStop
rc=scrSetData("LocalUpdate","TYP=SYM DLG=A_UN MNR=4711")	Direct update of symbol display (ctwin V7.2.3.73)
rc=scrSetData("SetButton","DLG=@ACTIVE FKT=FKT=WG1 FONT.STYLE=fsBold,fsUnderline")	Change properties of the button labeling in the dynamic dialog and FONT.STYLE=-fsUnderline
rc=scrSetData("SetButton","DLG=@ACTIVE FKT=FKT=WG1 CAPTION=Hauptgerüst")	Change button labeling in the dynamic dialog
rc=scrSetData("LocalUpdate","TYP=ANR DLG=A_TR MNR=4711 ANR=AG100000010 ##ANR_FU_7=2")	Quantity update for a user field
rc=scrSetData("LocalUpdate","TYP=ANR DLG=A_TR MNR=4711 ANR=AG100000010 ##ANR_FU_7=0")	Set absolute value
rc=scrSetData("UpdateGrid","DLG=@ACTIVE GRD.ACCESS=TRUE GRD.FILTER=LEVEL=1")	Dynamic dialog with grid: reset filter - GRID.FILTER=<ALL> (no filtering) - GRD.ORDER=ART (specify sorting)
rc=scrSetData("UpdateGrid","DLG=@FIL=DLG=CE_ASW_RF GRD.ACCESS=TRUE GRD.FILTER=ABKZ=N & ATK="+VDlg("ATK")+" & SLP=0002;00001")	SLP=0002;00001 → Semicolon is used for an OR conjunction
rc=scrSetData("DelayedButtonClick","DLG=@ACTIVE KENN=LOS_MELDE")	Press the key with KENNUNG=LOS_MELDEN in the dialog with delay

		
	rc=scrSetData("DisableReload","PNR,ANR")	Deactivate cyclic loading of lists →MNR, ANR, PNR, MSTAT, HZTYP, AGRD, LPKZ, BPOS, NCOM, LICENSE, ZLO, TPE, PATHS, DNC FAM, IOP RQ, AART, MAT, RES, TNRRDATA
	rc=scrSetData("DialogStartTime","Elapsed")	prevents message 'Dialog is open for more than 5 min'
	rc=scrSetData("DelayedDialogFunction","DLG=@ACTIVE DLG.LEFT=0 DLG.TOP=0")	Call in DynDlgInit → Sets of dialog position in relation to the main window
	rc=scrSetData("ActivateSetupFunction", "ButtonPaintOnOrderChange")	General: Setting of buttons in ctwin code ButtonPaintOnOrderChange: If the order row is changed in the main view, the script function "UserExitOnBtnIniPaint" is called to change buttons.
	rc=scrSetData("ActivateSetupFunction", "IgnoreTicketPrintOnBatchChange")	Deactivate ticket printing function (it is taken over by the script)
	rc=scrSetData("ActivateSetupFunction", "NoTargetCycleToAggregates") rc=scrSetData("ActivateSetupFunction", "NoPartitioningToAggregates")	Function to deactivate the transfer of target cycle and partitioning to the aggregates.
	rc=scrSetData("ActivateSetupFunction", "IgnoreMDEAutoStatus") rc=scrSetData("DeActivateSetupFunction", "IgnoreMDEAutoStatus")	Stop /reactivate cyclic M_AST messages
	rc=scrSetData("ActivateSetupFunction","DisableAllOperationFilters")	Deactivate filter of order list in the main view (all OPs of all machines are then displayed that are included in the list anr.lst)
	rc=scrSetData("DeleteLine","FILE=List.lst LINE=5")	Deletes a row in a list. Only the file operation is performed. The refreshes are not triggered.
	rc=scrSetData("ResetBatch","CNR=PR..")	Resetting of the last batch number that has been generated automatically using the function SYS NEW CNR FR
	rc=scrSetData("SetMaxParallelOrders","20")	Increasing the maximum number of OPs that are permitted to be run at the same time at a machine with parallel make-to-order production (OPs with

		different partitioning). If you use this function, errors might occur because of data strings that are too long!!
	rc = scrSetData("BusDataToMDE",a) Bsp: a = DLG=EVENT C:Cxxx=xx I:Ixxx=xx	If the Busdll is active, it can be used to transfer data to the MDE for processing. Data as it comes from the PCCDLL DLG=EVENT C:Cxxx=xx I:Ixxx=xx
	rc=scrSetData("AddListFileColumn","FILE=vlist.lst AKRO=SEL VALUE=")	Add column to a list file
	rc=scrSetData("ExecFunction","DLG=@ACTIVE FKT=REFRESH")	Call a script function of a dialog (→in DynDlgFunctions_.. the function should be implemented)
	rc=scrSetData("UpdateTextView","DLG=@ACTIVE AKRO=LOC:NOTE ACTION=CLEAR")	Ctaip: Access to a TextView of a dynamic dialog via its ID. Possible calls: ACTION=REOPEN/ RELOAD/ CLEAR
	rc=scrSetData("EnableDistReason","TRUE") rc=scrSetData("HideErrors","ERR=1240,1250,1242,1248,1243,1249 DLG=A_UN") rc=scrSetData("BypassBZW","")	Functions to replace the logic of the release of field deviation reason in case of over/underdelivery.
	rc=scrSetData("SetFieldVisible","DLG=@ACTIVE AKRO=EGI:GUT FKT=HIDE")	Hide, show, enable, etc. field of a dialog. Values for FKT: SHOW → Field is visible HIDE → Field is not visible TOGGLE → Toggle visible<->invisible ENABLE → Field allows input DISABLE → Field becomes ReadOnly
	rc=scrSetData("SetMaschProperty","MNR="+sMaschine+" P=mpAutoMengenSperre ACTIVE=1")	Suspend posting of automatic quantities to the server → should be reset using ACTIVE=0!
	rc=scrSetData("DeleteListFileLine","FILE=mat.lst FILTER=ATK=1234")	Deletes all rows in a LST file that match the filter criterion. Return values: 0:OK / -1:file not found / -2:no data rows available / -3:no data matching the filter
	rc=scrSetData("DelayedDlgSelectLine","DLG=@ACTIVE AKRO=ATK VALUE="+sATK) rc=scrSetData("DelayedDlgSelectLine","DLG=@ACTIVE AKRO=ENTNR VALUE=00044 SWITCH_ALWAYS=TRUE")	Selects the first row in the dialog grid that matches the filter criterion. The function is performed with the next timer run, also if the current script function has been completed. SWITCH_ALWAYS=TRUE → if the row that is already active is found during selection, then the active column is at least changed to trigger a CellChange event. NOTFOUND=SELECTFIRST / NOTFOUND=SELECTLAST Selects the first or the last row if the filter has no result. If the first or the last row of the grid should generally be selected, then the function is faster if AKRO is empty.

	<code>rc=scrSetData("DeleteGridLine", "DLG=@ACTIVE")</code>	delete current row in the dialog grid
	<code>rc=scrSetData("WriteToQueue", "QNAME=CE_SCAN DATA="+sComp ort+";"+sPref+";"+sBar) sItem=scrSetData("GetFromQueue", "QNAME=CE_SCAN") asText=scrSetData("ReadQueueText", "QNAME=CE_SCAN")</code>	Queue function provided by the terminal program: WriteToQueue → write a record GetFromQueue → read a record (and delete) ReadQueueText → read complete text (no delete)
	<code>rc=scrSetData("ReopenDialogGrid", "DLG=CA_WL_MPL MNR=...")</code>	Dialog grid is locally read from file a new time
	<code>rc=scrSetData("SetButtonFunction", "DLG=P_AN_AB BTN=F6 FK T=@PAR_MLD CAPTION=paralleles Melden")</code>	Overwrites a key of the dialog "log on/off person" by a function that must be programmed in "UserExitButtonClick". The function currently only works for the keys F1..F7 of the dialog P_AN_AB, which is not a dynamic dialog.
	<code>rc=scrSetData("DelayedButtonClick", "DLG=@ACTIVE GRD_FKT= CE_MLD")</code>	Calling an internal function assigned to the dialog grid (knowledge of the terminal program is required (only AIP)).
	<code>rc=scrSetData("LocalUpdate", "TYP=MDE,MNR DLG=M_SZY MNR=4 711 SZY=12000 FORCE=SZY")</code>	Setting the target cycle transferred - as of MDE 7.2, the maximum value of the target cycles of the logged on OPs is otherwise set.
	<code>rc=scrSetData("BarcodeFieldConfig", "FLD=KBN.RES:NR CMD=A DD")</code>	Taking over any bar codes if the field is focussed - after activation is valid for fields with the specified ID in all dynamic dialogs. Deactivation with CMD=REMOVE
	<code>rc=scrSetData("PositionForegroundWindow", "LEFT=960 Top=1 0 WIDTH=960 HEIGHT=1080")</code>	Set position and size of active Windows application
	<code>rc=scrSetData("MoveAIP", "LEFT=10 TOP=800")</code>	Move terminal program AIP to specific position
	<code>rc=scrSetData("DialogOnTop", "DLG=@ACTIVE")</code>	Moves the currently open dynamic dialog to the top edge of the terminal program AIP
	<code>rc=scrSetData("MoveDialogButtonBar", "DLG=@ACTIVE TOP=80")</code>	Moves the button bar of the current dynamic dialog vertically to the specified position
	<code>rc=scrSetData("MoveGrid", "DLG=@ACTIVE TOP=1 HEIGHT="+Cst r(iGrid))</code>	Moves the grid of the dialog and resets its height.
	<code>rc=scrSetData("SetWindowPos", "HANDLE=12345 LEFT=0 Top=0 WIDTH=1920 HEIGHT=864")</code>	Sets position and size of the window whose handle has been transferred. A window handle can be specified e.g. with <code>scrGetInfo("GetForegroundWindow", "")</code> .
	<code>rc=scrSetData("ReopenMainGrid", "MNR,ANR,LIST3")</code>	Locally read the specified grids in the main view a new time (no reload from server)
	<code>asRet=scrSetData("CyclicProcesses", "RELOAD=0 T_STATUS=0 MDE=0")</code>	Only AIP: For performance-critical processes, specific cyclical actions of the terminal can be suppressed. They should be reactivated promptly afterwards (RELOAD=1 T_STATUS=1 MDE=1) RELOAD: cyclic loading of the lists of the terminal T_STATUS: cyclic posting of terminal status (also to activate the remote control via VNC)

		MDE: MDE is run (if the MDE processing is stopped, the quantities and shift changes are only recorded with delay). Return value: current status of the cyclic actions (e.g. "RELOAD=0 T STATUS=0 MDE=0")
	<pre>rc=scrSetData("ProcessMessage", "INIT=300 TEXT=Ausgangslo swechsel") rc=scrSetData("ProcessMessage", "TEXT=----- ") rc=scrSetData("ProcessMessage", "INIT=END")</pre>	Display of SplashScreen that is also shown when the terminal program is booted or when lists are reloaded. INIT=xxx: opening the window with the specified height TEXT=...: adding a text row (the rows are added at the bottom and disappear at the top edge of the window) INIT=END: closing the window.
	<pre>rc=scrSetData("SetGridAutofilter", "DLG=@ACTIVE CAPTION=S trukturefilter FONT=ARIAL SIZE=8 FOCUS=1")</pre>	Configuration of the auto filter field in the dynamic dialog CAPTION: Alternative text for "Filter" FONT/SIZE: setting for label and edit field FOCUS=1: set focus of dialog on the auto filter field in the grid (only AIP)
	<pre>rc=scrSetData("SetGridAutofilter", "DLG=@ACTIVE TEXT=Arti kel-Filter")</pre>	The text of the auto filter field of a grid in the dynamic dialog can be overwritten. (only AIP)
	<pre>rc=scrSetData("SetGridAutofilter", "DLG=@ACTIVE ACRO=AUNR ")</pre>	The acronym (filter field) of the auto filter field of a grid in the dynamic dialog is reset <uMpdvTnrScript.pas> (only AIP)
	<pre>rc=scrSetData("SetScriptResult", "1")</pre>	If a list button behind an input field is to be used for another function (i.e. no list display), this call must be set in the respective branch in DynDlgFieldListe... Also the function triggered by the button is implemented here. With CTWIN, you can add a function key at the respective position.
	<pre>rc=scrSetData("AllowRedraw", "MNR=FALSE ANR=FALSE")</pre>	Stop redrawing of grid in the main view Should be reactivated promptly afterwards using MNR=TRUE ANR=TRUE !! Application: Avoid line breaks / flicker when data is written to the list.
	<pre>rc=scrSetData("SetMessageDelayTime", "TIME=1")</pre>	On the AIP, the default display time of a message in the status row (top left) is 10 sec. Use this command to change the time. The previously valid default time is returned that should be reset directly after the message. Example: <pre>Function StatusBarTimedMsg(sMsg,sTyp,sTime) Dim rc,sDelay sDelay=scrSetData("SetMessageDelayTime", "TIME="+sTime) rc=scrStatusBarMsg(sMsg,sTyp,"1") sDelay=scrSetData("SetMessageDelayTime", "TIME="+sDelay) StatusBarTimedMsg=sDelay</pre>

		End Function
	rc=scrSetData("SetField","DLG=@ACTIVE AKRO={Hier ButtonKennung} BUTTON.CAPTION=XXX")	For ACTIONBUTTON configuration on the AIP Description of buttons that are configured in the dialog. An update in the dialog is performed.   rc=scrSetData("SetField","DLG=@ACTIVE AKRO=Z2.4 BUTTON.CAPTION="+Halo")
	rc=scrSetData("UpdateGrid","DLG=@ACTIVE SHOW=0")	Hide grid of a dynamic dialot (SHOW=1 to show) (only AIP)
	rc=scrSetData("ActivateMainButton","PANEL=MNR BTN=VNR_AN ,VNR_AB ACTIVE=-1")	Activate/deactivate keys in the main view of the AIP Values for ACTIVE: -1..disable 1..enable -2..hide 2..show (only AIP)
	rc=scrSetData("SetFocusToGrid","DLG=@ACTIVE FOCUS=FILTER ")	Set focus of dialog on the grid FOCUS=FILTER → Filter field FOCUS=GRID → Table area of the grid

		(only AIP)
	rc=scrSetData("AsBarcode","BAR=12345678 TYP=LEGIC COM=3")	Simulates a bar code. The terminal processes this bar code the same way as the scanned bar code. Note: Does not function with a call via @@TEST.
	rc=scrSetData("SetAttachedButton","DLG=@ACTIVE AKRO=BEM CAPTION=RUN WIDTH=150 GLYPH=ListButton.png")	Setting of label and symbol of a button that is attached to a field. This is useful if the button is not configured as list button. Dialog field configuration: Field attribute: BTN;;<Caption> List: FKT=<Scriptfkt> (only AIP)
	rc=scrSetData("SetSkinning","ON OFF SWAP GET")	ON: turn on skinning of the AIP OFF: turn off skinning of the AIP SWAP: Change skinning (ON <-> OFF) GET: returns the current status (ON or OFF)
(UE) + (DLG)	rc=scrExecute("LabelPrint",Parameter)	Print with Labelprt.exe Syntax for KME: rc=scrExecute("LabelPrint","/b<Barcode> [/d<Datum>] [/z<Zeit>] /s<Strang> /a<Ausschuß> [/o<Option>] [/f<BarFontName>]") (only CTWIN)
	rc=scrExecute("SendAutoStatus","MNR=4711")	Triggers sending of input quantities to the server (if available)
	rc=scrExecute("ChangeStatus","MNR=4711 MST=10")	Change status of a machine (send to server, local update, transfer to MDE)
	rc=scrExecute("BEEP","btOK")	Output of a standard audio signal As error signal: „btFalse2” Further available signals: btAgain,btFalse,btScreenshot,btWait,btShort,btReceiveBusEvent,btProcessBusEvent, btPrgNeustart
	rc=scrExecute("WinExec","SW_SHOWNORMAL ""c:\Programme\TextPad 4\TextPad.exe"" ""+DIR_SPOOL+"druck.000""")	Starting an application
	rc=scrExecute("WriteBufferToFile",sPrnFileName+" "+asDat)	Writing an ansi string in a file
	rc=scrExecute("CloseDynamicForm","DLG=TA_CAB_SOND MNR=ENTGRAT5 CloseActive=1")	Closing a dynamic dialog
	rc=scrExecute("OnOrderNotification","DLG=A_AN MNR=...")	Exactly calls the functions with A_AN or A_UN that are internally processed in the terminal program, once an order has been logged on or interrupted.
	rc=scrExecute('ShowOrderInfo','MNR=4711 ANR=TK1234 TAGINFOPAGE=AG_ZuInfo NANR.LST=LOADED ') [.. TAGINFOPAGE=AG_ZuInfo NANR.LST=LOADED ']	Display of OP info from terminal script Additional optional parameters - TAGINFOPAGE=AG_ZuInfo Transfer of the OP info page to be displayed - NANR.LST=LOADED stop double loading of <nanr.lst>

		- only use parameter if nanr.lst has been loaded in script
	rc=scrExecute("RequestReload","MNR,ANR,PNR")	Request reloading of lists. The list is only reloaded during the next run of the timer. In the script, the result cannot be waited for. Possible parameters: MNR,ANR,PNR,MSTAT,HZTYP,AGRD,LPKZ,BPOS,NCOM,PAINT,DLOSE,PPARAM,LOKVLIST,YSR,LICENSE,ZLO,TPE,CAQ_SEND,CAQ_RECV,QMS_TIMER,PROC_INT,PATHS,DNC_FAM,IOPRQ,AART,SKAL,MAT,RES
	rc=scrExecute("ResetRequestReload","MNR,ANR,PNR")	Reset reload request of lists
	rc=scrExecute("TopButtonClick","vbCAQ")	(only CTWIN) Click top button from a script vbPZE,vbMPL,vbCAQ, vbPDV,vbIHMLd, vbDNC, vbWRM, vbUser1... The specified button must be available!
	rc=scrExecute("RunWithAttachedPrg","FILE=c:\ctwin\spool\Infos.doc")	The file transferred is started using the application that is linked to the file extension in Windows. The complete path of the target file must be specified. Also network paths are allowed. Internet links do not work.
	rc=scrExecute("RunWithAttachedPrg2","FILE=c:\ctwin\spool\Infos.doc OPERATION=open")	Alternative function that also works on the terminal server (uses ShellExecute)
	rc=scrExecute("CtwinPasswordDialog","Titel=Debug-Menu Caption=Paßwort: Password=MOS6950 ExpertPassword=.. CaseSensitiv=TRUE ")	Open password dialog Transfer parameters: Titel = Password - Dialog - Header (Default „Titel“) Caption = Field designation (Default „Passwort:“) Password = Password (Default „MOS6950“) ExpertPassord = Expert Password (Default „“) CaseSensitiv = Groß/Kleinschreibung (Default „FALSE“) Return string: PWOK = Password entry correct PWEXPERT = Expert Password entry correct(only possible with Parameter <ExpertPassword>) PWFALSE = Password entry false or dialog interrupted Note: If < spBypassMPDVPASSWORD > is set, no dialog is opened
	sBarcode=scrExecute("InputBox","TITLE=Manuelle Barcode-Eingabe CAPTION=Barcode FORMAT=C10,150,TRUE")	Open a simple dialog to enter a string TITLE: Text in dialog title bar CAPTION: Field query text
	rc=scrExecute("ShowError","ERROR=90024 TIME=10 P1=10111 P2=10")	Show an error message (→errors.pas) Is required with release change to use existing error messages (no new translation required)
	sBarcode=scrExecute("ScanDialog","CAPTION=Barcode AKRO=ATTR:10 DEFAULT= LEN=18")	Open dialog to scan a bar code Characters in password presentation;

	Further options: OPT=PASSWORD_BARCODE OPT=PASSWORD_ENABLED CAPTION:TR=Barcode BKCOLOR=clPurple / BKCOLOR=\$005EFB9C	input only via bar code manual input is possible caption is translated color dialog background (constant or RGB value)
	rc=scrExecute("ShowVirtKeys", "DLG=@ACTIVE VISIBLE=0")	Hide/show virtual keyboard
	rc=scrExecute("SaveLineAsText", "CHARS=60 WORD_LF=1 FILE="+DIR_SPOOL+"\kommentar.txt TEXT="+sComment)	A text is divided into lines of maximum CHARS characters and saved in a FILE. With WORD_LF=1 the lines are only changed between entire words. Usage: a text file formatted this way can be displayed on the terminal using scrShowInfo("TEXT", "FILE="+DIR_SPOOL+"\kommentar.txt CAPTION=Kommentar").
	rc=scrExecute("AutoBatchChange", "MNR=4711")	Execution of an automatic output batch change without dialog. Requirement: automatic generation of output batch number.
	rc=scrExecute("WriteToMWP", "DLG=LLR_M_MST MNR=4711 MST=11 DAT=05/21/2010 ZEI=57600")	Writing a row to the MWP2 interface Return value: -1:error 0:new file created 1:added to existing file
	Rc=scrExecute("OpenMWPOut", "") .. rc=scrExecute("WriteMWPLine", "DLG=LLR_M_MST MNR=4711 MST=11 DAT=05/21/2010 ZEI=57600") .. rc=scrExecute("CloseMWPOut", "")	Writing of several data rows to the MWP2 interface Return value with OpenMWPOut: -1:error 0:new file created 1:added to existing file
	rc=scrExecute("CheckQueue", "")	This function tries to empty the queue. In the offline case, the offline timeout is not waited for. The terminal tries to send all records one after the other. If the function is successful, the function returns the value "0". If records remain in the queue, the return value matches the number of records with a minus sign put in front.
	rc=scrExecute("ChangeExtension", "FILE=C:\data\file.ctw NEW=.dat ChangeFile=1 DeleteExisting=1")	Changing a file extension (here: file.ctw → file.dat) ChangeFile=1: file is changed (otherwise only the changed name is returned) DeleteExisting=1: if the target file already exists, this file is replaced.
	rc=scrExecute("RunMDE", "")	Immediately perform an MDE run. Afterwards, the recorded quantities on the terminal are up-to-date.
	rc=scrExecute("MakeTextFile", "FILE=CAQTempDoc.txt TEXT:1=Hallo, TEXT:2= TEXT:3=Ende")	Creating a text with several rows based on the text parts transferred
	rc=scrExecute("CloseDelphiForm", "frmpanab")	Closing the dialog "Log on/off persons" - up to now only realized for this screen.

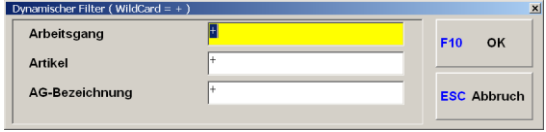
	<code>rc=scrExecute("RefreshDelphiFormGrid","frmpanab")</code>	Locally reload grid of dialog "Log on/off persons" - up to now only realized for this screen.
	<code>rc=scrExecute("ReloadPersons","")</code>	Immediately reload list of persons from server
	<code>rc=scrExecute("wrDruckdaten",sDate)</code>	Example: <code>xData=scrReplaceAllDDKennung("ANR.", "", aData, "")</code> order data <code>xData=xData + scrReplaceAllDDKennung("MNR.", "", mData, "")</code> machine data <code>xData=xData + scrReplaceAllDDKennung("DLG.", "", dData, "")</code> dialog data <code>xData=xData + Item("TRANSFERDIR_OUT", sTransDirOut)</code> output directory <code>xData=xData + Item("TRANSFERDIR_IN", sTransDirIn)</code> input directory <code>xData=xData + Item("TIMEOUT", sTimeOut)</code> Timeout in seconds <code>xData=xData + Item("OUTFILE", sOUTFILE)</code> output file name <code>xData=xData + Item("RESFILE", sRESFILE)</code> Result file name <code>rc=scrExecute("wrDruckdaten", xData)</code> Result file writes row RET=0 RET.INFO=here Result Text or RET=xx RET.INFO=error text RET=0 RET.INFO=OK if without result file if no result file is specified, only Outfile is written. Timeout only affects the result file. Note: Terminal waits for the result file or until timeout has elapsed If no TRANSFERDIR_IN is available, the input directory is set to the output directory
	<code>rc=scrExecute("DeleteIniKey", "FILE="+DIR_APP+"hcc_data.ini SECTION=xxxxxx IDENT=xxxx")</code>	Deleting a key in an INI file
	<code>rc=scrExecute("RefreshDelphiFormGrid", "LIST_MNR")</code> <code>rc=scrExecute("RefreshDelphiFormGrid", "", LIST_ANR)</code> <code>rc=scrExecute("RefreshDelphiFormGrid", "LIST3")</code>	Reread grids of the main view after a local update
	<code>rc=scrExecute("RunConsumption", "DLG=@ACTIVE FKT=VERBR AUCH:START DLG=A_UN MNR=4711 ANR=1222121211 ...")</code>	Calling the consumption dialog from script with free parameters
	<code>rc=scrExecute("ShowApplication", "NAME=Excel")</code> <code>rc=scrExecute("ShowApplication", "HANDLE=1234567")</code>	Moves another application in the foreground. NAME is the window title of the application or part of it. NAME is case sensitive.
	<code>rc=scrExecute("InheritLineCycle", "DLG=M_SZY MNR=4711 SZY=60")</code>	Line terminal: triggering transfer of cycle to aggregates. Note: If the send string passes SZY_LIN=0, then the inheritance of the target cycle to the aggregates can be stopped.
	<code>rc=scrExecute("CloseApplication", "HANDLE="+sHandle)</code>	Closing an application(WM_CLOSE is sent to the handle passed)
	<code>rc=scrExecute("SLEEP", "1000") '[ms]</code>	Waiting in the script [ms] - without processor load

	<code>rc=scrExecute("ReloadMachineList","REQUEST=TRUE LOCAL=TRUE")</code>	Immediately reload machine list LOCAL=TRUE → only reread file local in the grid REQUEST=TRUE → otherwise the file is only read during normal cycle
	<code>rc=scrExecute("ReloadOrderList","REQUEST=TRUE")</code>	Immediately reload order list REQUEST=TRUE → otherwise the file is only read during normal cycle
	<code>rc=scrExecute("SelectList3","P")</code>	AIP: Selection of a view in the third list M-material / P-persons / R-resources... (only AIP)
	<code>rc=scrExecute("GetLosProperties","FILE=..")</code>	Record batch attributes included in the file passed.
	<code>rc=scrExecute("OpenList","DLG=@ACTIVE FIELD=EGI:EGG:AUS")</code>	Automatically open list for a DIALOGLIST field (only AIP)
	<code>rc=scrExecute("WaitForEvent","EVENT=EDIT_EGG_AUS TIMEOUT=30")</code> <code>rc=scrSetData("SetEvent","EDIT_EGG_AUS")</code> <code>rc=scrSetData("ResetEvent","EDIT_EGG_AUS")</code> <code>s=scrGetInfo("EventWaitetFor","EDIT_EGG_AUS")</code>	WaitForEvent: program execution stops at this point and waits for other processings until the defined event has been set. Once the timeout has been reached, the execution is continued also without the event. SetEvent: Setting of the event that is waited for (via another code position) ResetEvent: Resetting event (for the next run) EventWaitetFor: Query if an event is waited for Using these functions requires expert knowledge. If the functions are used improperly, the complete background processing of the terminal can be stopped. Opening a script dialog results in the loss of the code position where the stop took place.
	<code>rc=scrExecute("UpdateHierarchy","DLG=@FIL=DLG=DNC")</code>	If the grid in the dialog is hierarchically structured, then the display can be updated using this function. (only AIP)
	<code>rc=scrExecute("ReInitHierarchy","DLG=@FIL=DLG=DNC")</code>	If the grid in the dialog is hierarchically structured, then the display can be rebuild once the list has been loaded. (only AIP)
	<code>rc=scrExecute("CreatePipeString","<Header> =###=<Dat aline>")</code>	Build a pipe string from its header row and a file row. Example: Header="MNR=Maschine MGRP=Gruppe MST=Status..." Dataline="4712 2 1 ..." Result:"MNR=4712 MGRP=2 MST=1 ..."
	<code>rc=scrExecute("ShowBatchInfo","CNR=PR1BH3411B")</code>	Display of batch information (required dialog: C_LOS_INFO)

	rc=scrExecute("PDMInit","HOST=10.10.60.87 USR=2200 HYPORT=10500 PROT=c:\prot.txt")	Reinitialization of the PDM interface Example: Section "Setting offline status for tests"
	rc=scrExecute("DialogTest","DLG=RES_WL TYP=TGRP DLGUSR=904")	Calling the dialog from the script (only part of the programmed functionality is available)
(UE) + (DLG)	rc=scrShowInfo("TEXT","FILE="+DIR_SPOOL+"ExcMagic.Log CAPTION=Fehlerprotokoll") rc=scrShowInfo("GRID","FILE="+DIR_SPOOL+"anr.lst CAPTION=Auftragsliste SECTION=Auftragsliste FILTER=AGNR=020 LAYOUTFILE="+DIR_APP+"ctwinlay.ini") scrShowInfo("AINFO","TYP=AddInfo ANR=TK0000000010 MNR=6010") scrShowInfo("AINFO","TYP=CompList ANR=TK0000000010 MNR=6010") rc=scrShowInfo("DOCUMENT","FILE="+DIR_SPOOL+"mwampel.html"+" ZOOM=1")	Display of a text file (advantage compared to scrMsgBox: window is bigger; can be operated using a CT760 keyboard) Display of a -lst file as grid. The parameters FILTER and LAYOUTFILE are optional Display of the additional info of an OP Display of the component list of an OP ZOOM=1: Showing zoom buttons in htmview3.exe (htmview3.exe 7.1.1.8)
	rc=scrExecute("AddBusQueueEvent","DLG=EVENT C:C007=1")	Simulation of PCC input data (use for test code)
	rc=scrExecute("DoFl2Debug","")	Triggering an exception error for test purposes
	rc=scrExecute("ButtonDialog","CAPTION=Auswahl upload=1;F1 activate protocols=2;F2 ")	Display of a dialog with several keys (similar to the debug functions)
	rc=scrExecute("ProcessMessages","")	Calling processing of the Windows queue! Only experts should use this function. In the LoopStop, the function can have the effect that the terminal hangs or crashes. In combination with "SLEEP", the use can be helpful elsewhere.
	rc=scrExecute("OpenErrorPopup","...")	Callback to open a ErrorPopupMeldung from a terminal script. For further information, refer to section "UserExitCustomErrorPopup" (only AIP)
	xV=scrExecute("GetQuotedDDItem",<ID>+" "+<VALUES>)	Identifies an item from a dialog string with masked DD items. Example: VALUES = „<MNR=m1 PARAM=MNR=pM1\ ANR=pA1 ANR=a1>“ Call <ANR ...> == „a1“ Call <PARAM ...> == „MNR=pM1 ANR=pA1 “
	xV=scrExecute("MakeQuotedDDItem",<xID>+" "+<xVALUE>)	Creates a masked dialog string item

(UE) + (DLG)	scrDeleteItems(asData,asAkros:string):string	Deleting acronyms from a dialog string (UserExitDynDlgBeforeSend) .. sDlg=VDlg("#GET#ALL#VALUES#") DLGSND="#DELETE#ALL#VALUES#" sDlg=scrDeleteItems(sDlg,"EGT:GUT EGT:AUS EGT:GES") DLGSND=sDlg
(UE) + (DLG)	// General VB script functions vbsTranslateDataValues(Items , Values)	➔ see <mpdv-system.scr> - Function vbsTranslateDataValues(..) - Translates the values of the transferred <items> in a - dialog data string <values> - e.g. vbsTranslateDataValues("MSTTXT",".. MSTTXT=#MST1 ..") -> ".. MSTTXT=PRODUKTION .." - e.g. vbsTranslateDataValues("S1 S2",".. S1=#S1 S2=#S2 ..") -> ".. S1=Spalte1 S2=Spalte2 .."
(UE)	scrComportDataWrite(string):string	With <HYREADER.DLL>, connection of external reader to write data to external reader with the ID <DATA2WRITE>. Relevant IDs to identify external readers are <COM> and <TYP> - <COM> is preferred -> Data is only written using the specified COMPORT - if only <TYP> is specified, the data is transferred to all instances of the <TYP> to write.
(UE)	scrComportEventResult(string):string	With <HYREADER.DLL>, connection of external reader to write an event result to an external reader with the ID <RET> and <RET.TXT> including a description in text form. Relevant IDs to identify external readers are <COM> and <TYP> - <COM> is preferred -> Data is only written using the specified COMPORT - if only <TYP> is specified, the data is transferred to all instances of the <TYP> to write.
	scrMDESetData(string)	MNR=<machine number> must be included For example, the following acronyms can be used: ACTION=MNR-UPDATE MNR=M24 MST=2 AGR:GUT=1 AGR:AUS=2 AGR:HUB=5 TLG=6 SZY=8000 PSPERRE=FALSE IMPF AKT=7 MST=3 DLG=MNR-UPDATE or ACTION= MNR-UPDATE
	scrMDEGetData(string):string	Machine info of the MDE is returned MNR=<machine number> must be included Result string= MNR=M24 MNR_ART=Z BUCH_ART=A PSP_ART=A MST=2 MSDAUER=360527 MSTTXT=RUESTEN IMPF AKT=2.00 TLG=8.00 SZY=10000.00 IZY=0 SKNR=3 SKSTA=SCH_NORM PSPERRE=TRUE PKENN=S S_GUT=0 S_AUS=12 S_LEN=0 S_HUB=0 Note: the shift counters -> S_GUT,S_AUS,...are no longer included with MDE72

(UE)	scrGWCUpdateResult(string):string	➔ only in UserExit < UserExitOnGatewayData > If the DD value <FT_ERROR> is set, the respective result is set for the calling external program. See section (1.4.3.1 Using < scrGWCUpdateResult > / < UserExitOnGatewayData >)
	ScrForceDirectories(DIR_SPOOL+"prnl\")	Creating a directory structure
	scrMDESetData("MNR="+{Maschinennr}+" ACTION=MNR-UPDATE MDE:ACTIONMODE=#SHIFTCOUNTER#CLEAR# ")	sets the shift counters in the MDE to 0 ("MNR="+{Maschinennr}+" ACTION=MNR-UPDATE MDE:ACTIONMODE=#SHIFTCOUNTER#CLEAR# ")
(DLG) + (UE)	scrDDSendRcv(oSnd:AnsiString;var pSnd:AnsiString;var pRcv:AnsiString):integer	Function to send dialog data (see < scrDDSend[WOErr] >) for application DLL interface! Parameter: <pre> oSnd „original send data“ var pSnd „actually sent data“ (MST without „;x“) var pRcv „actually received/set data“ (server result, DDQueue result, ..) </pre>
(DLG) + (UE)	scrLizenz(lizenz:String):boolean	Function to test a license Result: <pre> true / if license is active false / if license is not active </pre> For example: If scrLizenz("MPL-SNR") Then <pre> .. Else .. End If </pre>
(DLG) + (UE)	scrHyproduct(Prod:String;Version:String;iNoProdResult:integer):integer	Function to test a product version Result: <pre> -1 / if version passed is smaller than the active product version 0 / if version passed is equal to the active product version 1 / if version passed is greater than the active product version 2 / if product version is unknown </pre> For example: If scrHyproduct("MPL" , "7.2.5" , 2) >= 0 Then <pre> .. Else .. End If </pre>

	<pre>--Filter Dialog Aufrufen-- sRet=sScrExecute("mpDynDlg","DLG=DLG=DYNFLT; Dynamischer Filter (WildCard = +) ^DLG.UPPERKEY=TRUE^ANR=C24;Arbeitsgang;;#^ATK=C24;A rtikel;;#^AGBEZ=C24;AG- Bezeichnung;;# DEF=ANR=+^ATK=+^AGBEZ=+ ") --Set filter in GRID -- s=GStore("#GRID#FILTER#WILDCARD#",scrStrReplace(sRet, " "," & "))</pre>	Calling the dynamic filter in script and setting the filter in GRID as of CTWIN V7.2.5.61 
	DNC special functions <pre>rc=sScrExecute("DNCDocListView","RES=DNC-R- FMO011 RESTYP=DNC") rc=sScrExecute("DNCFileView","MNR=60610 RES=DNC-R- FMO011 RESTYP=DNC O_VORH=..") scrGetInfo("GetDNCFilPresetting","AKRO=MNR MNR=6011 A NR=12345678") rc=sScrSetData("ReloadDncRes","MOD=D MNR=6011 IDX:1=.. ..") scrGetInfo("GetDNCNameFromHost","RESFAMID=12 FIL1=606 11 ... FIL4=SG100") rc=sScrExecute("SendFileListToDNCInterface",asSend) scrGetInfo("DNCStatus","CMD=IsInfoDialogVisible") scrGetInfo("DNCStatus","CMD=IsTransferRunning MNR=606 10") rc=sScrSetData("DNC","CMD=ClearProtfile MNR="+VDlg("MN R")) rc=sScrSetData("DNC","CMD=DNCProt MNR=6010 MSG=Downloa d VISIBLE=1") rc=sScrExecute("GetDNCFiles",asRes) rc=sScrExecute("MakeDNCStatusList","RESFAM=.. RESTYP=. . [FILE=..]"</pre>	These functions are provided to realize the DNC functionality of ctaip. It is not recommended to use these functions elsewhere.

11.4.3 Notes on the script functions

11.4.3.1 Using < scrGWCUpdateResult(..) > / < UserExitOnGatewayData >

You may only execute this application callback function in the user exit < UserExitOnGatewayData > as of V# 7.2.3.45. You use this function with customer-specific GateWay events. The function passes the result of events with the ID <FT_ERROR> and sometimes also additional information on the error with the ID < FT_ERROR_TXT> to the calling external program.

The standard processing of a GateWay event is that the event is sent to the server with the addition <..|EVCOM=J|..>.

Possible error codes <FT_ERROR> with the default error description <FT_ERROR_TXT>.

<FT_ERROR> String identifier	Integer value	Default description <FT_ERROR_TXT>
fteOK	0	OK
fteTnrTmOt_NO_PLAUSI	1	NO_PLAUSI (TimeOut: TNR <-> DB)
fteDB_PLAUSERERROR	2	PLAUSERERROR (DB)
fteTNR_OFFLINE	3	OFFLINE (TNR-QUEUE)
fteTNR_Busy	4	TNR_BUSY (Process runs)
fteTIMEOUT_GW_TNR	5	TIMEOUT (GateWay <-> TNR)
fteGW_TNR_notINIT	7	CFG: GateWay -> TNR not init
fteTNRnotREADY	8	TNR not Ready for Process
fteTNR_GW_notCONFIG	9	CFG: TNR -> GateWay not init
fteDEFAULT_FT_ERROR	50	DEFAULT FT ERROR (is set if value of <FT_ERROR> cannot be identified)
fteDLG_UNDEF	90	unknown dialog
fteNO_DATA_TO_SND	99	No send data available
fteNO_VALID_DATA_FORMAT	100	Invalid data format
fteCLIENTSOCKET_GETDATA_EXCEPTION	101	FCT_EXCEPT (exception in processing function)
fteCLIENTSOCKET_GETDATA_UNDEF	102	FCT_UNDEF (undefined processing function)
fteTNR_MNR_NOT_CFG	900	MDE->MNR not config
fteWNR_NOT_CFG	901	ANR->WNR not runs
fteMNR_MST_NOT_CFG	902	MNR->MST not config

fteANR_MNR_NOT_RUNS	903	MNR->ANR not runs
fteAUSGRD_NOT_CFG	904	EGG:AUS not config or empty
fteBARCODE_LEN_ERROR	905	Undefined Barcode length [valid: 0,13,16]
fteMNR_MST_NOT_POSSIBLE_PSP_ACTIVE	906	MNR->MST not possible - PSPerre active
ftePLOCK_COUNT_OFF	907	P-Lock active without counting
fteMNR_KEINE_SCHICHT	950	Machine "no shift"
#DONE#	---	After calling the callback function < scrGWCUpdateResult(..) > in Userexit < UserExitOnGatewayData > , you must use the instruction UE_RET = Item("FT_ERROR", "#DONE#") to set <FT_ERROR> to the value <#DONE#> to prevent the standard processing from being run. ➔ In the modular environment, the notify event is sent to the modules nevertheless!

If the <FT_ERROR> has been set using the string identifier or the respective integer value in the UserExit < UserExitOnGatewayData > , the standard processing is not run any more (exception: the notification events of the modules).

If no <FT_ERROR> is set, the standard processing is run for the GateWay event.

(1) Example ➔ setting negative <FT_ERROR> via application (not with application callback scrGWCUpdateResult())

>>> Send string:

<DLG=TAKA_MST|MELDZEI=43200|MELDDAT=03/05/2008|BEARB=KFS|MNR=M000002|MST=1|CLI.SND.T=10:51:35.746|>

<<<< Receive string:

<DT:TNR=2,7970000170|FT_ERROR=906|FT_ERROR_TXT=MNR->MST not possible - PSPerre active (.An MDE->MNR >M000002< production lock is active. Machine status change is not permitted / UserExitOnGatewayData) [21]|COM.ID=4@|DLG=TAKA_MST|MELDZEI=43200|MELDDAT=03/05/2008|BEARB=KFS|MNR=M000002|MST=1|CLI.SND.T=10:51:35.746|TNR=17|DT:CLI=3,0000000726|>

(1) Example → setting positive <FT_ERROR=0> via application callback scrGWCUpdateResult())

>>> Send string:

<DLG=TAKA_MST|MELDZEI=43200|MELDDAT=03/05/2008|BEARB=KFS|MNR=M000002|MST=1|CLI.SND.T=12:07:45.011|>

<<<< Receive string:

<DT:TNR=0,2499999013|FT_ERROR=0|FT_ERROR_TXT=OK (.TAKA_MST verarbeitet / CallBack / scrGWCUpdateResult) [0]|COM.ID=5@|DLG=TAKA_MST|MELDZEI=43200|MELDDAT=03/05/2008|BEARB=KFS|MNR=M000002|MST=1|CLI.SND.T=12:07:45.011|FT_MODE=WAIT;2;150|TNR=17|DT:CLI=0,3590002656|>

→ other control identifier/variable relevant for the GateWay processing <FT_MODE>

Empty „“	is equal to <NORMAL> → after setting of result, there is not break in the processing. → The ClientServerThread sends the result, but the GateWay event processing in the terminal is not completed until the Windows queue of the application has been run through. → For example, if after the call < scrGWCUpdateResult > a server communication is performed, the terminal cannot receive a new GateWay command until this action is completed.
SLEEP;2;150	→ after setting the result, the application is stopped 2 times for 150 msec. (with 2 MessageBeep) → Default for the number <2> is 1. The default for the duration <150> is 200 MSec. → For notes on the processing, see < Empty „“ >
WAIT;2;150	→ after setting the result, the application is stopped 2 times for 150 msec. (with 2 MessageBeep and processing of the Windows queue using <i>ProcessMessages</i>) → The ClientServerThread sends the result and the GateWay processing in the terminal is completed. The terminal is therefore available for a new GateWay event.

General notes:

- 1) While a GateWay event is processed, the application program "ctwin.exe" shows the clock with a white background in the status bar.



- 2) Only one GateWay event can be processed. (display in application)

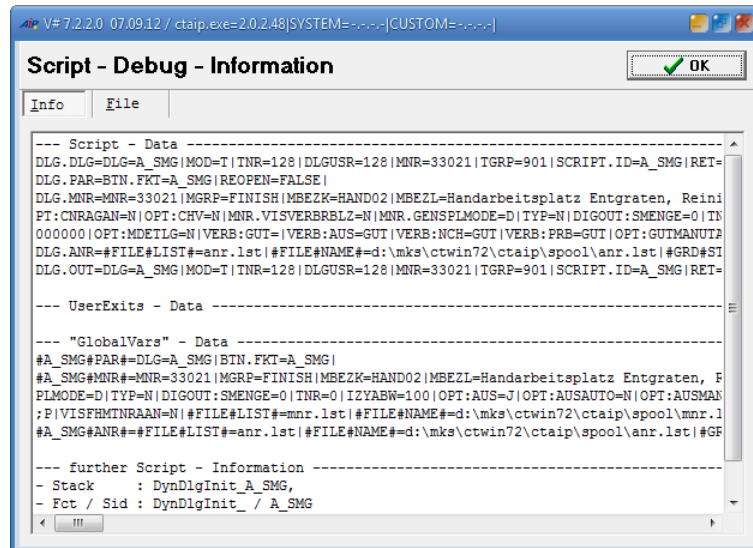


<<< Receive string for calling program

<FT_ERROR=4|FT_ERROR_TXT=TNR_BUSY (Process runs) [Client Event just runs] [4]|....>

11.4.3.2 Script - Debug – Dialog

Using the function „SYS_SCRIPT_DEBUG“ the Script – Debug – Information is displayed as follows as of terminal versions CTWIN V# 7.2.6.36 and CTAIP V# 2.0.2.49.



Tab "Info" includes the following information.

- „Script - Data“: Data/variables of the current/last dialog
- „UserExits - Data“: Data/variables of the current/last dialog user exit
- „GlobalVars - Data“: Global variables of the application
- „further Script - Information“: Information on the current script status.
 - Script call stack, i.e. name of the function that is executed
 - Script function + Dialog ID
 - Counter for active dialog and user exit functions

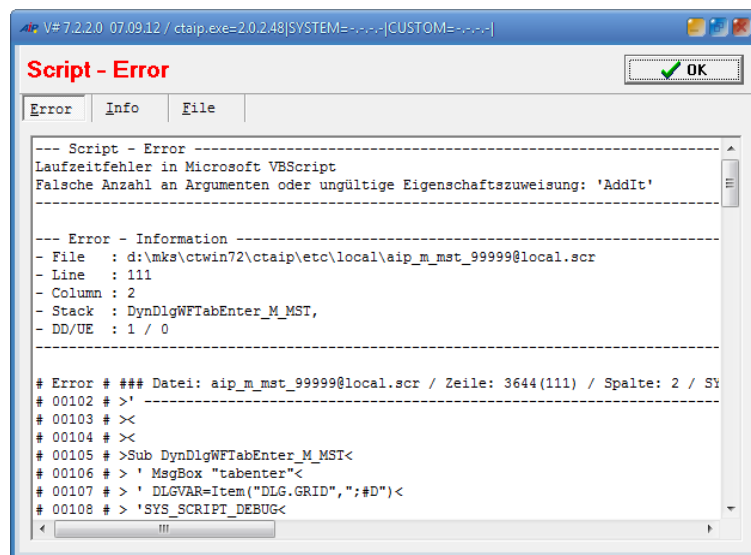
Tab "File" includes the following information.

- „Script - File - Infos“: Information in tabular form on the scripts currently loaded
- „Script - File - Overview“: Short information on the scripts loaded + zip files.
- „Script - Methods - Overview“: Information on the functions available in the system + dialog – script files of the scripts currently loaded.

11.4.3.3 Exception - Script – Dialog

In addition to the information described in section "Script - Debug – Dialog", this dialog includes the tab "Error" that displays the following data.

- Description of the error
- Additional error information :
 - Script file (with error)
 - Row in script file (with error)
 - Column in script file (with error)
 - Script call stack, i.e. name of the function that is executed
 - Counter for active dialog and user exit functions
- A script excerpt where the error is marked
- The script file that includes the error.



11.5 Special Fields of Application

11.5.1 Assignment of a script function to a key without DDLG

In **ctwinbut.ini** the identifiers must start with „@’ :

Ex. (WKP):

```
F9=@WKP_CNR_VA_DEL,Voranmld. Delete
F10=WKP_XXXX,WKP Dialog XXXX -> XXXX_WKP.SCR -> DynDlgInit_XXXX must be defined
```

Processing in the script **system_<customer>.scr**:

```
Sub UserExitButtonClick
    sFkt=VVar("UE:PAR","BTN.FKT")
    Select Case sFkt
        Case "@WKP_CNR_VA_DEL"
            sMNR=VVar("UE:MNR","MNR")
            sCNR=GetVLos(sMNR)
            if sCNR="" Then
                scrMsgBox("no advance logon for batch")
            Else
                Res=DlgJaNein("delete advance logon","really delete batch logged on
in advance ("&sCNR&")?")
                If Res="#JA#" Then
                    DeleteVLos(sMNR)
                End If
            End If
            UE_RET=Item("BTN.FKT","#FKT#->#EXIT#")
        End Select 'Case sFkt
    End Sub 'UserExitButtonClick
```

It is important to set the return value „BTN.FKT=#FKT#->#EXIT#“. Otherwise the error message "unknown button ID..." is displayed.

11.5.2 How to use the functions GSrce, VSrce

You can use GSrce to access a list file. The following parameters are possible:

- rc=GSrce ("LOAD", "FILE=.x.|RET=ROWCOUNT|") 'x. stands for the file including directory that is to be loaded.

With the addition "RET=ROWCOUNT" the file is reread from the hard disk.

- rc = GSrce ("FIRST", "XXX") 'XXX stands for an optional filter, e.g. MNR=102030

- rc = GSrce ("NEXT", "XXX") 'XXX stands for an optional filter, e.g. MNR=102030

First and Next provide a return code. If the return code is <> "#EOF#STORE#", then a further row has been found

VSrce is then used to access the current row, e.g. VSrce("MNR") is used to read the machine number of the current row.

- rc = GSrce ("CLOSE", "SAVE=[TRUE/FALSE]") with saving [Yes/No]

- sLine=GSrce("GETLINE", "") read row number

- rc=GSrce("SELECTLINE", sLine) delete row

- rc=GSrce("DELETELINE", "") delete current row

- rc=GSrce("DELETELINE", sLine) delete specific row

11.5.3 Important note when working with numbers

The following must be observed when working with numbers. There are different VB script engines of the Internet Explorer (that we use). When comparing <, >, ... , some script engines identify whether a variable is integer/floats, which variables are to be compared, and then convert them automatically.

Other versions do not identify this and might compare the variable with strings. For this reason, it is always recommended to make an explicit type cast in case of comparisons.

11.5.4 Important note for < IF > queries

The script engine performs all comparisons in < IF > queries, also with < AND > conditions.

With the following instruction, this leads to a run time error, if < sInt > is an empty string or cannot be converted.

➔ In Delphi / C, the second condition is normally not checked, if the first condition is < **false** >.

```
Sub ...
...
sInt = VDlg("FU:32") ' *** identify string
If IsNumeric( sInt ) And Int( sInt ) = 4 Then
...
Else
...
End If

' *** possible procedure without run time error
```

```

If IsNumeric( sInt ) Then
    If Int( sInt ) = 4 Then
        ...
    End If
Else
    ...
End If
...

```

11.5.5 Update grid at the push of a button

If you call `DLGVAR = AddIt("DLG.GRID","RELOAD","")` you can update a grid. Requirement: A command has already been assigned to the `GRD.CMD` to get the list. This can be done, e.g. in the user exit `DynDlgGridInit`

```

...
SCRVARS = "GRD.CMD=" + "DLG=LIST;104|MOD=U|MNR=<MNR>|ANR=<ANR>|"
...

```

11.5.6 Recording of MSS signals

All MSS events pass the **UserExitAutomaticQuantities**. This way, you can also catch MSS signals.

You implement a batch change for KME triggered by a PIN signal:

Configuration of the MSS in `ctwin.ini`:

```

[MSS-INIT]
;# buffered input - one entry per machine - 0=no channel
PIN=|1|
;# activate PIN recording
PIN_ERFASSUNG=on

```

UserExit:

```

Function UserExitAutomaticQuantities
    sMode=VVar("UE:PAR","MOD")
    Select Case sMode
        Case "LOCAL" ' *** EventToLocalData ***
            'SYS_SCRIPT_DEBUG
            sDlg=VVar("UE:SND","DLG")
            Select Case sDlg
                Case "PIN"
                    ' UE:SND=DLG=PIN|MNR=MR11|PINNR=1|PINCOUNT=1
                    Maschine=VVar("UE:SND","MNR")
                    PinNummer=VVar("UE:SND","PINNR") ' is not used
                    AnzEvents=VVar("UE:SND","PINCOUNT")
                    '*****
                    RunOrPufferBatchChange(Maschine,AnzEvents)
                    '*****
                Case "M_AST;ASYNC"
                    ' UE:SND=DLG=M_AST;ASYNC|MNR=MR11|DAT=10/18/2007|ZEI=36772|IZY=0
                    Maschine=VVar("UE:SND","MNR")
                    '*****
            End Select
            ExecuteBufferedBatchChanges(Maschine)
    End Select
End Function

```

```

'*****
End Select
End Select
End Function

```

The function **RunOrPufferBatchChange** must buffer the events if a batch change cannot be performed. As no timer is available for a delayed triggering of the buffered batch change, the cyclic machine status posting has been selected to trigger the batch change. The function **ExecuteBufferedBatchChanges** must check for the machine passed, if batch changes are buffered, and execute these changes if possible.

11.5.7 Call script when leaving "Log on/off person" using ESC

You can use the script to catch the event <<Leaving dialog "Log on/off person" (A_P_AN_AB) using ESC>>:

```

Sub UserExitButtonClick
  Select Case VVar("UE:PAR","BTN.FKT")
    Case "@@CANCEL.A_P_AN_AB"
      '#####
      If GVars("DLG","READY")="FALSE" Then
        UE_RET=Item("BTN.RET",-1) ` Dialog remains open
      End If
    End Select
  End Sub

```

11.5.8 Implementing a debug output

You can use the shortcut CTRL+ALT-<right cursor> to jump to the scripting. This is normally used to show customer-specific debug outputs.

Example:

```

Sub UserExitButtonClick
  Select Case VVar("UE:PAR","BTN.FKT")
    Case "@@TEST" ' Press Ctrl+Alt+<Cursor right>
      '*****
      ShowInfos
    End Select
  End Sub

Sub ShowInfos
  Dim sMsg,sMnr,sAnr,asAnr
  sMnr=GetSelectedMnr
  sAnr=GetSelectedAnr

  asAnr=scrQuickSearch(DIR_SPOOL+"anr.lst","ANR="+sAnr+"&MNR="+sMnr+"&AST=L|F")
  sMsg="MNR="+sMnr+" ... "+GetMaschTyp(sMnr)+CHR(13)+CHR(10)+CHR(13)+CHR(10)
  sMsg=sMsg+"ANR="+sAnr+" ...
  Ausgangslos="+scrDDItem("CNR",asAnr)+CHR(13)+CHR(10)+CHR(13)+CHR(10)
  sMsg=sMsg+"BASE_DLG="+GVars("SYSTEM","BASE_DLG")
  scrMsgBox(sMsg)
End Sub 'ShowInfos

```

11.5.9 Dynamic assignment of a button configuration in the OP info (AIP)

The configuration of the buttons in the tabs of the OP info is defined in ctaipbut.ini in sections like e.g. [A_INFO.DOKU-Page1]. In CTAIP, it is possible to change to another section in ctaipbut.ini during runtime. This change is made in UserExitButtonClick. The call is performed when you open the OP info and each time you change the tab.

```
Sub UserExitButtonClick
  Select Case VVar("UE:PAR", "BTN.FKT")
    Case "@@AINFO.PAGE"
      !*****
      ModifyPage
      UE_RET=Item("BTN.FKT", "#FKT#->#EXIT#")
    End Select

Sub ModifyPage
  Dim sPageName
  sPageName=GVars("#AINFO#", "PAGE")
  If sPageName="A_INFO.KOMP" Then      ' original button configuration
    If sAnr2<>sAnr Then
      GLOBALVARS="#AINFO#=PAGE=A_INFO.KOMP_F" ' alternative button
configuration 1
    End If
    GLOBALVARS="#AINFO#=PAGE=A_INFO.KOMP_L" ' alternative button
configuration 2
  End If
End Sub
```

```
Ctaipbut.ini:
[A_INFO.KOMP-Page1]
1=AI_CLOSE,L,AG-Information schließen,Cancel.png

[A_INFO.KOMP_F-Page1]
1=AI_CLOSE,L,AG-Information schließen,Cancel.png
2=@FIRST,R,First

[A_INFO.KOMP_L-Page1]
1=AI_CLOSE,L,AG-Information schließen,Cancel.png
2=@LAST,R,Last
```

11.5.10 Call scripts from DNC (only AIP)

Once a DNC transfer is finished, the button function "@@DNC.TRANSFER.FINISHED" is called. If the DNC transfer is stopped by the user, "@@DNC.TRANSFER.CANCEL" is called. Another event is created if the DNC progress display is left using "Cancel".

In the script, you can react as follows to the events:

```
Sub UserExitButtonClick
  Select Case VVar("UE:PAR", "BTN.FKT")
    Case "@@DNC.TRANSFER.FINISHED"
```

```

'*****
OnTransferFinished
'*****
UE_RET=Item("BTN.FKT", "#FKT#->#EXIT#")
Case "@@ DNC.TRANSFER.CANCEL"
'*****
OnTransferCancel
'*****
UE_RET=Item("BTN.FKT", "#FKT#->#EXIT#")
Case "@@DNC.INFO.CANCEL"
'*****
OnDNCInfoCancel
'*****
UE_RET=Item("BTN.FKT", "#FKT#->#EXIT#")
End Select
End Sub

```

11.5.11 Setting the display time of an LR error message (only AIP)

You can use a parameter in the send string to specify the display time of a possible server error message. Example:

```

DD_SND=Item("DLG", "M_MST")
DD_SND=Item("MNR", sMNR)
DD_SND=Item("MST", "2")
DD_SND=Item("RET.ERRDLGTIME", "20")
scrDDSnd

```

11.5.12 Editing the DNC communication

During DNC upload/download, commands are exchanged between the terminal program and the interface. In the terminal script, you can catch and edit the commands from the interface to the terminal program.

```

Sub UserExitButtonClick
Dim sDlg, sFile, sFullFileName
Select Case VVar("UE:PAR", "BTN.FKT")
Case "@@DATA_TO_DNC"
asCmd=GVars("#DNC.CMD#", "#GET#ALL#VALUES#")
sDlg=scrDDItem("DLG", asCmd)
If sDlg="RES_DOWNL" Then Exit Sub ' no changes for DNC-Download
sFile=scrDDItem("SPEICHORT:DATA", asCmd)
sFullFileName=DIR_APP+"dnc\"+sMnr+"\upload\"+sDatei+"."+sExt
...
End Select
End Sub 'UserExitButtonClick

```

The command sent by the driver is saved in the container "#DNC.CMD#". It can be changed using the script.

The DNC transfer can be interrupted using the following return value:

GLOBALVARS="#DNC.CMD#=RET=CANCEL"
(tested only with CTWIN)

11.5.13 Script events online/offline

If the terminal changes to the online or offline status, you can use the UserExitButtonClick to react to this change:

```
Sub UserExitButtonClick
  Select Case VVar("UE:PAR","BTN.FKT")
    Case "@@ONLINE"
      ' terminal switched to online
      ...
    Case "@@OFFLINE"
      ' terminal switched to offline
      ...
  End Select
End Sub 'UserExitButtonClick
```

11.5.14 Setting offline status for tests

You can use the below-mentioned function GoOnline(False) to set the terminal to the offline mode by setting a false host. This can take several seconds because the terminal tries to connect. Resetting to the online mode using GoOnline(True) is faster. To this end, the function directly retrieves the connection data.

```
Sub GoOnline(bOnline)
  If bOnline Then

asData=Item("HOST",rsIni(DIR_APP+"ctaip.ini","system","hostname","ERROR"))
asData=asData+Item("USR",SYS_USR)

asData=asData+Item("HYPORT",rsIni(DIR_APP+"ctaip.ini","system","hyport","0"))
asData=asData+Item("PROT","")
Else
asData=Item("HOST","OFFLINE")
asData=asData+Item("USR",SYS_USR)

asData=asData+Item("HYPORT",rsIni(DIR_APP+"ctaip.ini","system","hyport","0"))
asData=asData+Item("PROT","")
End If
'*****
rc=scrExecute("PDMInit",asData)
'*****
'scrMsgBox("2^GoOnline: "+rc)
End Sub 'GoOnline
```

11.5.15 Show comments in machine status log

A comment, which has been entered when the machine status was changed manually, can be added to the machine status log. To this end, you add an option to the list request. Use the following script function:

```

Sub UserExitModifyListCmd
  Dim asDat
  If VVar("UE:PAR","DLG")="SYSTEM.CALL" Then
    If VVar("UE:PAR","PROG")="hym_sprt72.scr" Then
      asDat=VVar("UE:PAR","#GET#ALL#VALUES#")
      UE_RET=asDat+Item("OPT:KOMMENTAR","J")
    End If
  End If
End Sub

```

The field BEM in the list m_status_protokoll.lst includes the comment entered. In ctaipplay.ini, the column "BEM" can be inserted in section [Machine status log-MDE72].

11.5.16 Script events with CAQ calls (only AIP)

You can use the script events listed in the following to react to the opening and closing of the CAQ dialog in the terminal script.

```

Sub UserExitButtonClick
  Dim sDlg,sFile, sFullFileName
  Select Case VVar("UE:PAR","BTN.FKT")
    Case "@@DLL_DLG.INIT"
      ' before requesting badge number
    Case "@@DLL_DLG.START"
      ' Opening the CAQ dialog
    Case "@@DLL_DLG.END"
      ' Closing the CAQ dialog
    Case "@@DLL_DLG.CANCEL"
      ' Cancelling badge number request
    Case "@@DLL_DLG.EXCEPT"
      ' Error handling branch
    ...
  End Select
End Sub 'UserExitButtonClick

```

11.5.17 Script events with line breaks in order and machine list

If you want to perform any operation in the script when changing the line in the machine list or order list of the main view, you can use the events @@SELCHANGE.MNR and @@SELCHANGE.ANR:

```

Sub UserExitButtonClick
  Dim sDlg,sFile, sFullFileName
  Select Case VVar("UE:PAR","BTN.FKT")
    Case "@@SELCHANGE.MNR"
      ' Line break in machine list (main view)
    Case "@@SELCHANGE.ANR"
      ' Line break in order list (main view)
  End Select
End Sub 'UserExitButtonClick

```

11.5.18 Read first row from list file

scrQuickSearch can process the parameter „FIRST“ instead of the filter:

```
asAuftrag=scrQuickSearch(DIR_SPOOL+"anr.lst","FIRST")
```

11.5.19 Processing of long 2D bar codes

If a bar code recorded has more than 200 characters, the complete bar code is stored in the global variable GVars("##BARCODE##","BARCODE"). Afterwards, the event "@@BIG_BARCODE" is sent to the script processing.

A very long bar code can be processed in the script as follows:

```
Sub UserExitButtonClick
..Select Case VVar("UE:PAR","BTN.FKT")
....Case "@@BIG_BARCODE"
.....scrMsgBox(GVars("##BARCODE##","BARCODE"))
.....GLOBALVARS="##BARCODE##=HANDLED=1"
..End Select
End Sub
```

11.5.20 Reloading ANR list script request (only AIP)

The event "@@RELOAD.ANR" is called when the "anr.lst" is reloaded:

```
Sub UserExitButtonClick
..sFkt = VVar("UE:PAR","BTN.FKT")
..Select Case sFkt..
....Case "@@RELOAD.ANR"....
.....asSelected = scrGetInfo("GetSelected","")
.....sMnr = scrDDItem("MNR",asSelected)
.....sFilter = rsCfg("Tnr Konfiguration","ORDER_FILTER","")
.....If sFilter = "" Then
.....UE_RET ..= Item("BTN.FKT", "#FKT#->#EXIT#")
.....Exit Sub
.....End If
.....' run for all machines
.....iLine = 1
.....Do
.....mData = scrGetDlgLstLine(DIR_SPOOL+"mnr.lst",CStr(iLine))
.....scrLog("Line "+CStr(iLine)+" --> "+mData)
.....If mData="" Then Exit Do
.....sMnr..=..scrDDItem("MNR",mData)
.....param = "MNR="+sMnr+"|AKRO=ANR_FU_30|FILTER="+sFilter
.....rc = scrSetData("HiddeAnrRow",param)
.....iLine=iLine+1
.....Loop
.....UE_RET ..= Item("BTN.FKT", "#FKT#->#EXIT#")
```

11.5.21 Assign function keys in the order sequencing list

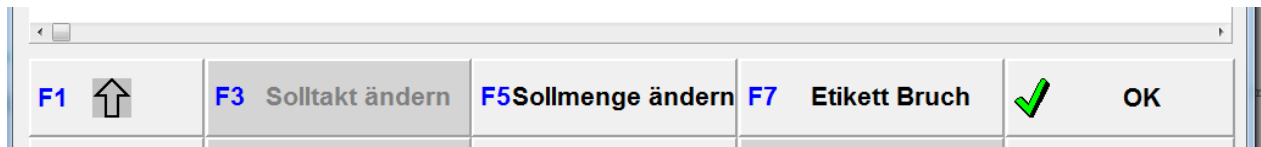
You can assign alternative functions in the script to the keys F3 to F8 in the order sequencing list. To this end, the following info is added in the dialog data:

*VLIST_BTN_F<n>=<function>,<caption>

This assignment can be made before the dialog initialization.

Example:

```
Sub UserExitDynDlgBeforeInitialize
    Select Case VOut("DLG")
        Case "A_AN", "A_P_AN"
            ' here, define key for sequencing list
            DLGVAR=Item("*VLIST_BTN_F7", "U_ETK_BRUCH, Etikett Bruch")
    End Select
End Sub
```



11.5.22 Script event when changing cell in the machine list

You can use the event "@@MNR.CELLCHANGE" to enable/disable buttons with reference to the selected machine.

```
Sub UserExitButtonClick
    Select Case VVar("UE:PAR", "BTN.FKT")
        ....Case "@@MNR.CELLCHANGE"
            ....CheckBlockButtonsActivation
    End Select
End Sub 'UserExitButtonClick

Sub CheckBlockButtonsActivation
    ..If GetFu29="J" Then
    ..rc=scrSetData("ActivateMainButton", "PANEL=MNR|BTN=VNR_AN,VNR_AB|ACTIVE=-1")
    ..Else
    ..rc=scrSetData("ActivateMainButton", "PANEL=MNR|BTN=VNR_AN,VNR_AB|ACTIVE=1")
    End If
End Sub 'CheckBlockButtonsActivation
```

(only AIP)

11.5.23 Focus in dialog grid (only AIP)

In DynDlgKeyDown and DynDlgFieldExit, the focus in the dialog grid is specified more precisely:

VDlg("DLG.FLD")="DLG.GRD" - the filter field is focused

VDlg("DLG.FLD")="DLG.GRD.GRD" - the grid is focused

The focus can also be set in the return string of the events:

```
DLGVAR="DLG.GRID=FOCUS|GRD.FOCUS=GRID"
```

or

```
DLGVAR="DLG.GRID=FOCUS|GRD.FOCUS=FILTER"
```

To prevent the filter from being deleted, you must pass an additional parameter:

```
DLGVAR="DLG.GRID=FOCUS|GRD.FOCUS=GRID|UPDATE_HARC=FALSE"
```

You can use the following function in any UserExit:

```
rc=scrSetData("SetFocusToGrid","DLG=@ACTIVE|FOCUS=FILTER")
```

11.5.24 Script event when loading additional info

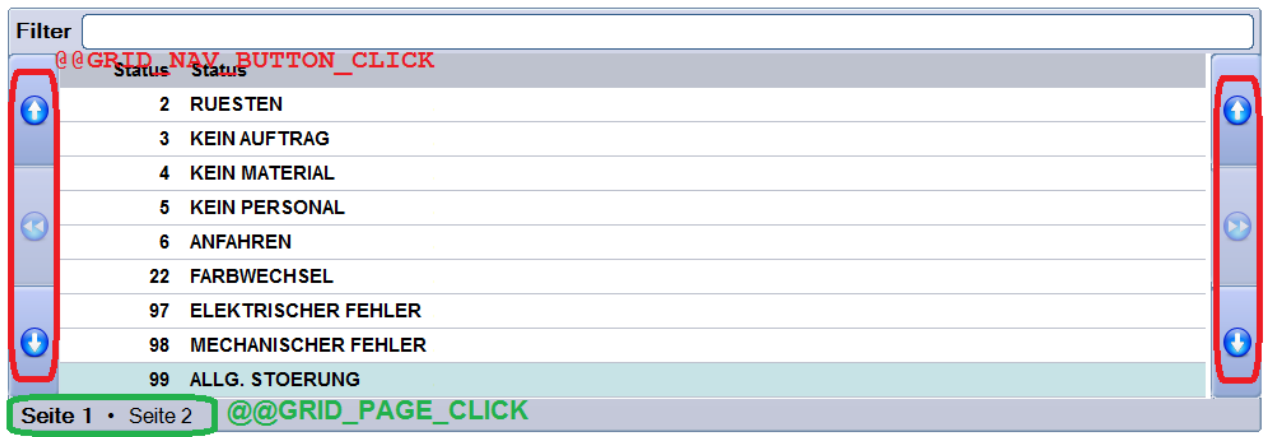
Once the additional OP info is loaded, an event is triggered. Here, the file can be manipulated from the script before it is read by the terminal program. The type of the additional info and the file name are passed in the global variable "#AINFO".

```
Sub UserExitButtonClick
  Select Case VVar("UE:PAR", "BTN.FKT")
    Case "A@AINFO.LOADED"
      If GVars("#AINFO#", "TYPE")="AI" Then
        If scrFileExists(GVars("#AINFO#", "FILE"))="0" Then
          ' change the file..
        End If
      End If
    End Select
  End Sub 'UserExitButtonClick
```

11.5.25 Script events when navigating in the dialog grid

If the keys are pressed to navigate in the grid, script events are triggered. For example, the focus can be changed.

```
Sub UserExitButtonClick
  Select Case VVar("UE:PAR", "BTN.FKT")
    Case "A@GRID_NAV_BUTTON_CLICK"
      ' Navigation button of grid was pressed
    Case "A@GRID_PAGE_CLICK"
      ' page button below grid was pressed
    End Select
  End Sub 'UserExitButtonClick
```



11.5.26 Extended customizing of label printing

The parameter **"PRN->PARAM"** is used for an extended customizing of a configured label with a posting event/dialog. Using this parameter, you can control if the "print order" is completely stopped or if only printing is stopped.

Parameter	Description
„PRN->PARAM=SKIP PRINTJOB“	Print order is completely stopped.
„PRN->PARAM=SKIP PRINTING“	Label printing is stopped. A configured Hydra script and a configured logging are performed. (Function available as of AIP V# 2.0.4.9 / CTWIN V# 7.2.8.40)

Example: Label printing is stopped or cancelled for the customer-specific posting event/dialog "Entry of quantities (U_MENGE)".

```

Sub DynDlgBeforeSend_U_MENGE
  Select Case VDlg("PRNMODE")
    Case "L"
      DLGSND=Item("PRN->PARAM", "SKIP PRINTING")
    Case "N"
      DLGSND=Item("PRN->PARAM", "SKIP PRINTJOB")
    Case Else
      ' processing configured print job
  End Select
End Sub 'DynDlgBeforeSend_U_MENGE

```

11.6 Terminal Dialog List Files (in alphabetical order)

For detailed information, refer to the standard PDM documentation

11.6.1 Static lists ()

<u>File name (/spool/..)</u>	<u>Description</u>	<u>Server command</u>
		<u>TNR = Terminal number (1 .. 999)</u>
		<u>USR = Hydra user (2001 .. 2999)</u>
aart.lst	Order types ➔ Includes all order types configured in the Hydra system (AUART)	DLG=LIST;87 ..
agrd.lst	Scrap reason list ➔ Includes scrap/yield reasons of the machines assigned to the terminal (ART=G,A,...)	DLG=LIST;84 MOD=T TNR=706 ..
anr.lst	Order list ➔ Includes all running order of all machines assigned to the terminal or logged on to the terminal.	LIST;11 MOD=L USR=2706 ..
bmkl.lst	List of RPA accounts ➔ Includes all Resource Performance Accounts configured in the Hydra system	DLG=BMK.LIST ..
bpos.lst	Machine operator positions ➔ Includes all operator positions of the machines assigned to the terminal.	DLG=LIST;14 USR=2706 ..
hyproduct.lst	Hydra – Product – List ➔ Includes all versions of all active Hydra products/modules (KERNEL,ADE,PZE,CAQ,HLS,ZKS,..)	DLG=LIST;121 ..
hztyp.lst	Semi-finished product types / material types (only loaded with a MPL machine) ➔ Includes all semi-finished product types and material types configured in the Hydra system (HZTYP)	DLG=LIST;21 ..
lizenl.lst	Hydra - licenses	DLG=LIST;48 ..

	➔ Includes all Hydra licenses	
mnr.lst	Machine list	LIST;10 USR=2706 ..
	➔ Includes all machines assigned to the terminal in a fixed form (=configuration) or dynamically (posting).	
mstat.lst	Machine status list	DLG=LIST;16 MOD=T USR=2706 ..
	➔ Includes all machine statuses of the machines assigned to the terminal in a fixed form (=configuration) or dynamically (posting possibly only after update from the server).	
paths.lst	Directory list	DLG=LIST;81 ..
	➔ Includes all paths of modules configured in the Hydra system (DNC,DOK,...)	
pnr.lst	List of persons	DLG=LIST;12 USR=2706 MOD=V ..
	➔ Includes all persons logged on that are logged on to a machine of the terminal.	
qrdcfg.lst	Label printing – configuration (requires license <HYD-ETD> or <'HYD-ETDRT>)	DLG=SYSTEM.CALL PROG=hyettlst.scr USR=2222 ..
	➔ Includes all labels that are active in the terminal and assigned to a dialog event. (DLG)	
	Note:	
	- the label definition is loaded in the file <qrdinit.rpb>	
	- in the local file <qrdinit.rpb.rpt>, all labels are listed	
schicht.lst	Shift list	DLG=LIST;38 MOD=T TNR=706 ..
	➔ Includes the shift configuration of the MDE machines assigned to the terminal	
tkenn.lst	Terminal label	DLG=LIST;45 TNR=706 ..
	➔ Includes the configured terminal label with some Hydra setup settings	
tnrmat.lst	Terminal - list of input material (only loaded with MPL machine)	DLG=LIST;13 MOD=T USR=2706 ..
	➔ Includes all input batches/materials logged on to a machine of the terminal.	
	Notes:	
	- this file is usually only updated via server synchronization.	
	- this file can also include logged on resources (project: EGO)	

tnrres.lst	Terminal – Resource list (as of WRM/ MDE >= 7.2.1) ➔ Includes all active resources of all machines of the terminal Notes: - this file is usually only updated via server synchronization.	DLG=LIST;129 MOD=T USR=2706 ..
tpe.lst	List of transport units ➔ List of all transport units configured in the Hydra system (container, pallet,...)	DLG=LIST;52 ..
vlpkz.lst	List of the wage/premium configuration	DLG=LIST;24 ..
zloueb.lst	List of the material buffers / target locations ➔ List of all material buffers/target locations of the Hydra system (or of the terminal)	DLG=LIST;49 TNR=706 ..

11.6.2 Temporary lists ()

<u>File name (/spool/..)</u>	<u>Description</u>	<u>Server command</u>
mat.lst	List of input material ➔ This file includes the component list last loaded of an order at a machine (the logged on input batches can also be included). Note: - this file is read ONLINE when it is displayed or processed (e.g. A_AN_MPL,CE_WL_MPL, ...) - this file is only managed locally for display purposes in a posting dialog (A_AN_MPL,CE_WL_MPL)	DLG=LIST;13 MOD=M MNR=DBCM1010 ANR=01001010010 DLG.DLGCFG=A_AN_MPL ..
nanr.lst	Order information ➔ This file includes the order info last loaded.	DLG=LIST;11 MOD=A ANR=010001010010 ..
vlist.lst	Order sequencing list of a machine ➔ MOD=V prepared/can be logged on for order logon (A_AN(_MPL,_RF),A_P_AN(_MPL,_RF)) ➔ MOD=L running	DLG=LIST;11 MOD=V MNR=DBCM1010 .. DLG=LIST;11 MOD=L MNR=DBCM1010 ..

for order interruption/logoff (A_UN(_MPL,_RF),A_AB(_MPL,_RF))

11.7 Exceptions via programmed standard development

11.7.1 Exception: Sending of ID < EGG:GUT >

You cannot send the ID < EGG:GUT > because, for program reasons, this ID is deleted from the string that is sent.

➔ Sending is only possible with an alternative ID.